

A Description of the Model for Prediction Across Scales, Atmosphere, Version 8

William C. Skamarock
Michael G. Duda
Joseph B. Klemp
Laura Fowler

***** DRAFT *****

Mesoscale and Microscale Meteorology Laboratory
NSF National Center for Atmospheric Research
Boulder, Colorado, USA

This is an incomplete **Draft** of the MPAS-Atmosphere Technical Note.
Chapters 6, 9 and 10 (Diagnostics, Physics, and Infrastructure) are incomplete.
This technical note should not be used to reference MPAS-Atmosphere in its current form.

NCAR TECHNICAL NOTES

The Technical Note series provides an outlet for a variety of NCAR manuscripts that contribute in specialized ways to the body of scientific knowledge but which are not suitable for journal, monograph, or book publication. Reports in this series are issued by the NCAR Scientific Divisions; copies may be obtained on request from the Publications Office of NCAR. Designation symbols for the series include:

- EDD: *Engineering, Design, or Development Reports*
Equipment descriptions, test results, instrumentation,
and operating and maintenance manuals.
- IA: *Instructional Aids*
Instruction manuals, bibliographies, film supplements,
and other research or instructional aids.
- PPR: *Program Progress Reports*
Field program reports, interim and working reports,
survey reports, and plans for experiments.
- PROC: *Proceedings*
Documentation of symposia, colloquia, conferences, workshops,
and lectures. (Distribution may be limited to attendees.)
- STR: *Scientific and Technical Reports*
Data compilations, theoretical and numerical
investigations, and experimental results.

The National Center for Atmospheric Research (NCAR) is operated by the University Corporation for Atmospheric Research (UCAR) and is sponsored by the National Science Foundation. Any opinions, findings, conclusions, or recommendations expressed in this publication are those of the authors and do not necessarily reflect the views of the National Science Foundation.

Contents

Acknowledgments	ix
1 Introduction	1
1.1 MPAS Atmosphere	1
1.2 Technical Note Purpose and Structure	2
1.3 MPAS-Atmosphere Code Overview	3
1.3.1 The init_atmosphere core	3
1.3.2 The atmosphere core	3
2 MPAS mesh	7
2.1 Horizontal Voronoi Mesh	7
2.1.1 Points on the MPAS horizontal mesh	8
2.1.2 Lines and arcs on the MPAS horizontal mesh	9
2.1.3 Connectivity	10
2.1.4 Horizontal velocity on the MPAS horizontal mesh	11
2.1.5 Areas on the MPAS horizontal mesh	11
2.2 MPAS Vertical Grid	12
2.2.1 Specifying the interface heights	13
2.2.2 Hybrid smoothed terrain-following coordinate	15
2.2.3 Vertical Interpolation	16
3 Equations and Time Integration	19
3.1 Continuous Equations	19
3.2 Time Integration Overview	20
3.3 RK3 Time Integration Scheme	22
3.3.1 Perturbation Equations Employed in the RK3 Solver	23
3.3.2 Operators in the RK3 Scheme	24
3.4 Acoustic Solver	24
3.4.1 Perturbation Equations Employed in the Acoustic Solver	24
3.4.2 Acoustic Timestep	28
3.4.3 Filters in the Acoustic Timestep	29
3.5 Scalar Transport Time Integration	30
4 Spatial Discretization	33
4.1 Mass Conservation Equation	33
4.2 Transport for Cell-Centered Variables	34

4.2.1	Horizontal Flux Divergences on the Voronoi Mesh	35
4.2.2	Vertical Fluxes	36
4.2.3	Boundary Conditions for the Vertical Fluxes	37
4.2.4	Positive-Definite and Monotonic Flux Renormalization	37
4.3	Horizontal Momentum Equation	38
4.3.1	Pressure and Kinetic Energy	38
4.3.2	Nonlinear Coriolis Term	40
4.3.3	Vertical Transport for the Horizontal Velocity	43
4.3.4	Horizontal Mass Flux Divergence	44
4.3.5	Vertical Momentum Equation	44
5	Filters	47
5.1	Horizontal Filters for Cell-Centered Variables	47
5.1.1	2nd-Order Horizontal Filter	47
5.1.2	4th-Order Horizontal Filter	48
5.2	Horizontal Momentum Equation: Horizontal Filters	49
5.3	Eddy Viscosities and Hyper-Viscosities	50
5.4	Offcentering in the Vertical Acoustic Integration	51
5.5	Gravity-Wave absorbing Layer	52
5.5.1	Rayleigh Damping for w	52
5.5.2	2nd-Order Horizontal Filtering	52
5.5.3	Rayleigh Damping of the Horizontal Momentum	52
6	Diagnostics	55
6.1	Zonal and Meridional Velocity	55
6.2	Frontogenesis Function	55
6.3	Other Diagnostics to Follow	55
7	Regional Configuration	57
7.1	Specified and Relaxation Boundary Regions	57
7.2	Time Integration	59
7.2.1	Divergence Damping in the Relaxation Zone	60
8	Initial Conditions	63
8.1	Initialization Commonalities	63
8.1.1	Reference States	64
8.1.2	Hydrostatic Balance	64
8.2	Initialization for Idealized Cases	64
8.2.1	Idealized Baroclinic Wave Case	65
8.2.2	Squall Line and Supercell Case	66
8.2.3	Mountain Wave Case	67
8.3	Initialization for Earth-Atmosphere Simulations	67
8.3.1	Initialization of the 3D State	68
8.3.2	Lateral Boundary Conditions for Regional Simulations	68
9	Physics	69

10 Infrastructure	71
A Vertically-Implicit Acoustic Solution	73
B Least-Squares Polynomial Computation	77
C Gradient Operators	81
C.1 Horizontal Derivatives	81
C.2 Horizontal deformation for the Smagorinsky scheme	84
C.3 Frontogenesis Function	85
References	87

List of Figures

1.1	A variable resolution MPAS global mesh.	1
1.2	A top level flow chart for MPAS-Atmosphere.	4
2.1	A portion of an MPAS-Atmosphere horizontal mesh with C-grid staggered velocities.	7
2.2	Horizontal mesh points and lengths used in MPAS.	8
2.3	Cell, edge and vertex neighbors of a cell. Cell indices are given in black, edge indices are red and vertex indices are given in blue.	10
2.4	Neighbors of edges and vertices. Cell indices are given in black, edge indices are red and vertex indices are given in blue.	11
2.5	Definition of the prognostic edge-normal horizontal velocity and the diagnostic horizontal edge-tangential velocity using the example mesh given in figure 2.4.	12
2.6	Horizontal mesh areas used in MPAS.	12
2.7	The MPAS-Atmosphere vertical grid with the locations of the staggered variables. The dashed red lines indicate the center of the layers, and solid lines are interfaces.	13
2.8	The terrain following coordinate used in MPAS. The left figure shows the traditional terrain following coordinate surfaces and the right figure shows the smoothed vertical coordinate.	16
3.1	The MPAS time integration methodology.	21
3.2	Runge-Kutta time integration response.	22
3.3	Integration timesteps in the split-explicit RK3 scheme.	25
7.1	Skamarock et al. (2018) Figure 2 - Regional MPAS meshes extracted from a global MPAS mesh by (a) specifying a circular region and (b) specifying a polygon whose faces are great-circle arcs on the sphere. The regional mesh specifications are given by the black lines, and the blue cells fall in the mesh. The boundary-zone cells appended to the outside of the specified mesh are also shown.	57
7.2	Skamarock et al. (2018) Figure 3 - Boundary zone for the MPAS horizontal mesh. The blue cells are the interior mesh, and the labeled cells and edges are in the boundary region.	58
7.3	Pseudo code describing the locations of lateral boundary condition applications in the MPAS timestep.	59
B.1	A limited region in an MPAS horizontal mesh. Adapted from Skamarock and Gassmann (2011) Figure 1.	77
B.2	Tangent plane for cell c_0	78
C.1	MPAS tangent-plane schematic.	82

C.2 Horizontal depiction of the tangent plane and cell-edge velocity fields for an arbitrary MPAS mesh cell.	83
--	----

List of Tables

Acknowledgments

The effort to develop, support and maintain the atmospheric component of MPAS-Atmosphere is led by the National Center for Atmospheric Research (NCAR), which is primarily supported by the National Science Foundation (NSF). The authors sincerely thank NCAR and NSF for their support of the MPAS effort. During the development of MPAS-Atmosphere, significant support in various forms has also been provided by: the National Oceanic and Atmospheric Administration, the U.S. Department of Defense, and the U.S. Department of Energy.

Chapter 1

Introduction

The Model for Prediction Across Scales (MPAS) is a collaborative project for developing atmosphere, ocean and other earth-system simulation components for use in climate, regional climate and weather studies. The primary development partners are the climate modeling group at Los Alamos National Laboratory (COSIM) and the National Center for Atmospheric Research. Both primary partners are responsible for the MPAS framework, operators and tools common to the applications; LANL has primary responsibility for the ocean and land ice models, and NCAR has primary responsibility for the atmospheric model.

The defining features of MPAS are the unstructured Voronoi meshes and C-grid discretization used as the basis for many of the model components. The unstructured Voronoi meshes, formally Spherical Centriodal Voronoi Tessellations (SCVTs), allow for both quasi-uniform discretization of the sphere and local refinement. The C-grid discretization, where the normal component of velocity on cell edges is prognosed, is especially well-suited for higher-resolution, mesoscale atmosphere and ocean simulations.

1.1 MPAS Atmosphere

The atmospheric component of MPAS, as with all MPAS components, uses an unstructured centroidal Voronoi mesh (grid, or tessellation) and C-grid staggering of the state variables as the basis for the horizontal discretization in the fluid-flow solver. The unstructured variable resolution meshes can be generated having smoothly-varying mesh transitions as illustrated in figure 1.1, and this capability ameliorates many issues associated with the traditional mesh refinement strategy of one-way and two-way grid nesting where the transitions are abrupt. Using the flexibility of the MPAS meshes, we are working towards applications in high-resolution numerical weather prediction (NWP) and regional climate, in addition to global uniform-resolution NWP and climate applications.

The MPAS atmosphere consists of an atmospheric fluid-flow solver (the dynamical core) and a subset of the Advanced Research WRF (ARW, [Skamarock et al. \(2021b\)](#)) model at-

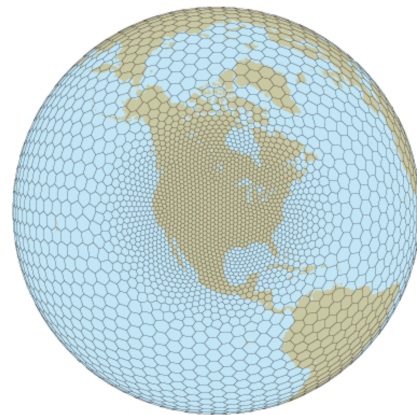


Figure 1.1: A variable resolution MPAS global mesh.

mospheric physics.

1.2 Technical Note Purpose and Structure

The purpose of this technical note is to provide a description of the MPAS-Atmosphere model. Importantly, this includes a complete description of the governing equations in the fluid-flow solver and their discretization. While aspects of the solver are described in the peer-reviewed literature, a comprehensive and detailed description appears only here in this technical note, along with updates to the techniques. It is our intent to keep this technical note up-to-date with the latest MPAS-Atmosphere release.

We begin in chapter 2 with a description of the unstructured horizontal MPAS centroidal Voronoi mesh and with the structured vertical component that make up the full 3-dimensional MPAS mesh. An understanding of this mesh is a prerequisite to understanding the discretization of the dynamical core (the MPAS-Atmosphere fluid-flow solver). Chapter 3 introduces the continuous fluid flow equations used in MPAS-A and describes the time integration of these equations. MPAS-A uses a split-explicit time integration technique (similar to that used in the WRF model (Skamarock et al., 2021b)) that requires defining mean and perturbation quantities for the prognostic variables and time-dependent equations for these variables that are used in specific sections of the solver. Chapter 4 describes the spatial discretization of the fluid-flow equations. Chapter 5 describes the discretization of the primary filters employed in MPAS-A, including 2nd and 4th order horizontal filters for cell-centered variables and for the horizontal velocity, and some aspects of filters applied to absorb vertically-propagating gravity waves. Chapter 6 describes the computation of the cell-centered zonal and meridional velocity using radial basis functions, and the horizontal frontogenesis function. Chapter 7 describes the regional configuration of MPAS focussing on the boundary conditions and how they are applied in the time integration. Chapter 8 describes MPAS-A initialization options, focussing on the real-data initialization but also providing an overview of available idealized cases. Initialization of an idealized or real atmospheric state that can be used in MPAS is accomplished by a separate MPAS model call *init_atmosphere_model*, in contrast to *atmosphere_model*. Chapter 9 provides a brief overview of physics available in MPAS-Atmosphere. The physics schemes available in MPAS are evolving quickly both in their formulation and in how MPAS-A accesses them. The final chapter in the technical note, chapter 10, outlines key aspects of the MPAS infrastructure that supports the atmosphere model.

In addition to describing the MPAS-A model and utilities, this technical note references the MPAS model code, and references to that code will appear in boxes with a pink background:

MPAS code: *Here is an example of how a statement referencing the MPAS code appears in this technical note. File names in the MPAS source code directory are given in blue. The main text is italicized to further distinguish it from the main body of the technote. Within an MPAS code section, subroutine names, variable names and named Fortran loops are given in red.*

In future MPAS-Atmosphere releases we will include comments in the MPAS source code referencing this technical note, and we will be adding additional markers in the code, and references to them in this technical note, that are easily found using simple search techniques.

1.3 MPAS-Atmosphere Code Overview

MPAS-Atmosphere is comprised of two main executables that can be built from the system contained in the MPAS-A Github repository, and these executables are called *cores* in the MPAS system. The *init_atmosphere* core is responsible for producing atmospheric states that the *atmosphere* core can use as an initial state that it integrates forward in time.

1.3.1 The *init_atmosphere* core

The *init_atmosphere* core is responsible for

- Interpolating *static* fields to the MPAS horizontal mesh, including terrain heights, land-use categorizations, etc.
- Creating the 3-dimensional MPAS mesh. See chapter 2.
- Interpolating analytic or an idealized initial state to the 3D MPAS mesh.
- Interpolating 3D analyses, or 3D states from other MPAS integrations, to the 3D MPAS mesh.
- Enforcing hydrostatic balance on the interpolated 3D state. See chapter 8 for information on interpolations and the MPAS-Atmosphere discrete hydrostatic balance.
- Interpolating sea-surface temperatures (SSTs) and other fields at times other than the initial state time for use by the *atmosphere* core to update these fields during MPAS-Atmosphere integrations.

MPAS code: The *init_atmosphere* core code can be found in *MPAS-Model/src/core_init_atmosphere/*. The main functionality that users access is contained in the source file *MPAS-Model/src/core_init_atmosphere/mpas_init_atm_cases.F*, that contains the routines that drive the idealized and real-data initializations.

1.3.2 The *atmosphere* core

The *atmosphere* core is responsible for integrating the MPAS-Atmosphere state forward in time. The integration is accomplished as depicted in this flow chart given in figure 1.2. The numbered tasks in the flow chart are explained as follows:

- (1) At startup necessary infrastructure is initialized, including I/O, parallel processing capabilities, clocking capabilities, etc.

MPAS code: The code that controls the sequencing of the flow chart items (2) through (5) can be found in *MPAS-Model/src/core_atmosphere/mpas_atm_core.F* and function *atm_core_init*.

- (2) There are two modes used to begin an integration - a restart mode (starting from a *restart* file produced by a previous simulation) or starting from an MPAS-Atmosphere initialization file produced by the MPAS *init_atmosphere* core. For more information see the MPAS-Atmosphere Users Guide.

MPAS code: Configuration as a restart run, as opposed to starting from an MPAS-A initialization, is controlled by the *namelist.atmosphere* configuration variable *config_do_restart* in the *&restart* namelist section.

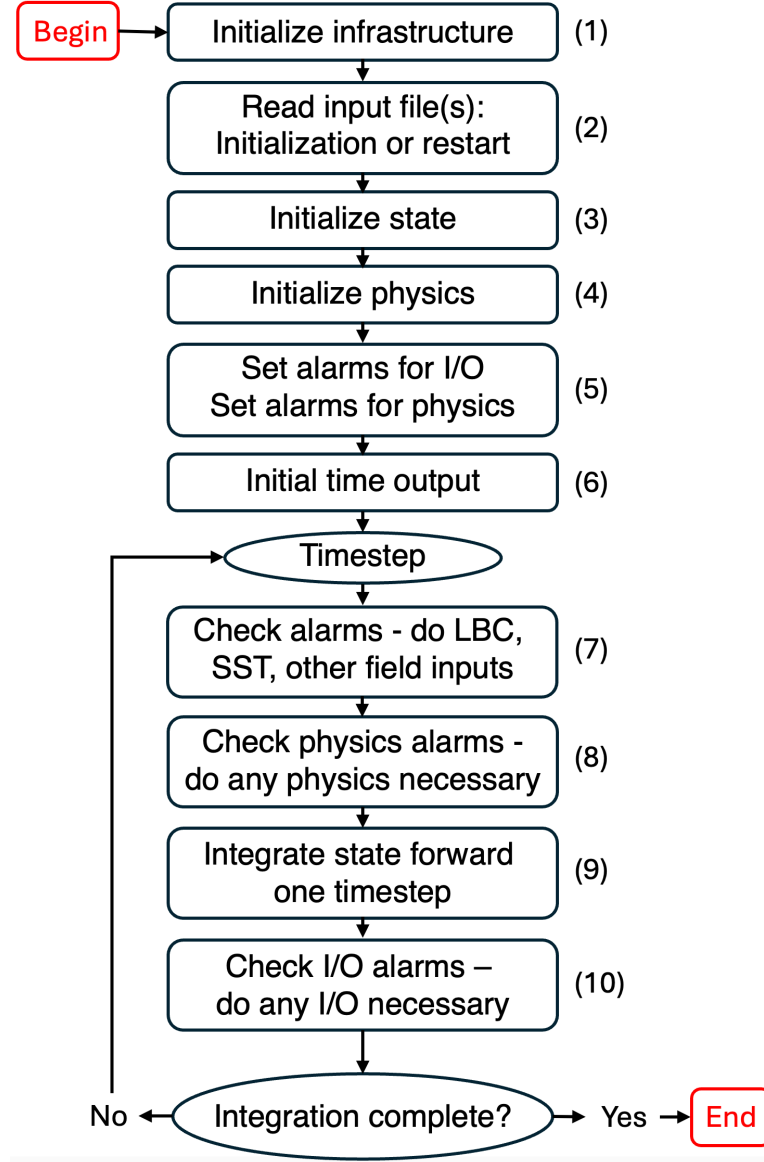


Figure 1.2: A top level flow chart for MPAS-Atmosphere.

(3) The MPAS-A initialization files only contained the uncoupled atmospheric state (e.g. the horizontal velocity u as opposed to the horizontal momentum ρu) and only fundamental mesh variables. Mesh metrics and other coefficients that can be precomputed are evaluated here. Additionally, the prognostic variables in the MPAS-A solver are the coupled variables (see section 3.1), and here the prognostic state is computed along with any state-dependent diagnostics. In a MPAS-A restart the coupled state would be read from the restart file.

MPAS code: The routine that does the coupling (*subroutine atm_init_coupled_diagnostics*) is found in *MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F*, and diagnostic variables (e.g. vorticity, etc) are computed in *subroutine atm_compute_solve_diagnostics*. These computations are described later in the technical note.

(4) Any aspects of the physics that need initialization is performed, including computing or reading from files any needed data (e.g. lookup tables used in radiation and microphysics) not included in the initialization or restart files.

MPAS code: *The MPAS-Atmosphere physics code is located in [MPAS-Model/src/core_atmosphere/physics/](#), including any code associated with the [init/run/finalize](#) structure of the MPAS-A model*

(5) MPAS-A uses [alarms](#) to determine when tasks that need to be executed periodically are to be performed during the integration. The bulk of these alarms are used to determine when I/O tasks need to be performed or when physics need to be called.

MPAS code: *The code that controls the sequencing of the flow chart items (6) through (10) can be found in [MPAS-Model/src/core_atmosphere/mpas_atm_core.F](#) and [function atm_core_run](#).*

(6) MPAS-A will produce stream output files at the initial time for a simulation starting from an MPAS-A initialization (i.e. a file produced by the MPAS-A [init_atmosphere](#) core). If the simulation is configured as a restart, then these files will not be produced as they will have been produced by the previous simulation that produced the restart file.

(7) At the start of each time step, alarms are checked to see if lateral boundary condition (LBC) data or other data need to be read in and fields updated.

(8) While forcings from most atmospheric physics are updated each MPAS-Atmosphere timestep, some physics forcings (e.g. radiation) are updated less frequently, hence the alarm checks.

(9) This is the main time step for the model, and a high-level overview can be found in [section 3.2](#).

(10) Output (restarts, history and diagnostic output) are performed here.

As described earlier in this chapter, the bulk of this technical note describes the spatial and temporal discretization utilized in the time integration. Atmospheric physics used in this integration is briefly described in [chapter 9](#). Aspects of the Infrastructure are described in [chapter 10](#). While some practical aspects of running the MPAS-Atmosphere model are given in the **MPAS code** callouts, a much more detailed and useful guide to running MPAS is found in the MPAS Users Guide:

https://www2.mmm.ucar.edu/projects/mpas/site/documentation/users_guide.html.

Additional descriptions of the MPAS systems and instructions for using it are given in the MPAS-Atmosphere tutorials, with tutorial presentations available at

<https://www2.mmm.ucar.edu/projects/mpas/tutorial/Virtual2024/agenda.html>.

Chapter 2

MPAS mesh

MPAS-A integrates the equations of motion that are spatially discretized employing a centroidal Voronoi mesh for its horizontal discretization with a hybrid vertical coordinate based on geometric height. For applications on the sphere, the horizontal mesh is defined using great-circle arcs as opposed to using a planer representation, thus the distances and areas are consistent in all spherical applications. In this chapter we briefly describe the centroidal Voronoi horizontal mesh and the vertical coordinate, along with the C-grid staggering of the MPAS-A prognostic variables. For detailed information on the centroidal Voronoi horizontal mesh readers are advised to consult [Ringler et al. \(2008\)](#) and references therein.

2.1 Horizontal Voronoi Mesh

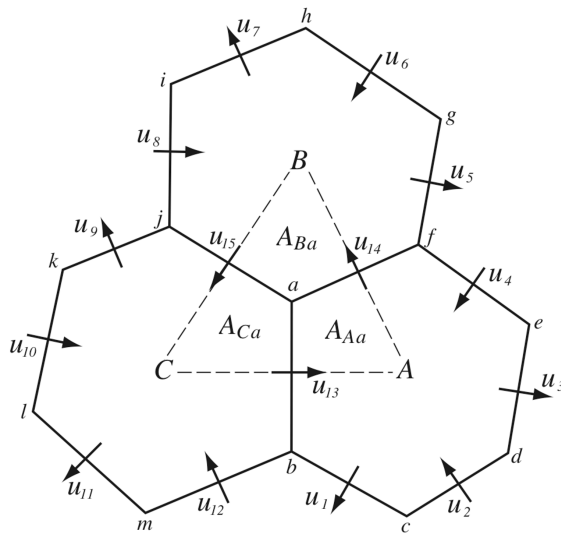


Figure 2.1: A portion of an MPAS-Atmosphere horizontal mesh with C-grid staggered velocities.

A schematic of a representative portion of the horizontal MPAS-A centroidal Voronoi mesh is given in figure 2.1. MPAS-A meshes are mostly comprised of hexagons, but may contain some pentagons and possibly heptagons, and mesh cells are referred to by their central points that

are located at the centroid (center of mass) of the cells. In figure 2.1 the cells (cell centers) are A , B and C . The dual of the centroidal Voronoi mesh is the Delaunay triangular mesh, and a Delaunay triangle ABC is shown in the figure. There are three properties of this centroidal Voronoi mesh that are critical to the MPAS-A discretization:

- (a) The centroidal property: The cell centers are centroids of the Voronoi cell, and any point in the cell is closer to the cell center than it is to any other cell center.
- (b) The lines connecting cell centers (edges of the dual Delaunay triangular mesh) are bisected by the cell edges: This follows from (a).
- (c) The lines connecting cell centers are perpendicular to cell edges at their intersection point. This also follows from (a).

Specifically, the fact that the horizontal line connecting the cell centers is orthogonal to the cell edge, and that the prognostic orthogonal velocity is coincident with that line at the the edge, greatly facilitates the discretization of the momentum equation, the transport of the cell-centered variables, and many of the filters, as described in chapters 4 and 5.

2.1.1 Points on the MPAS horizontal mesh

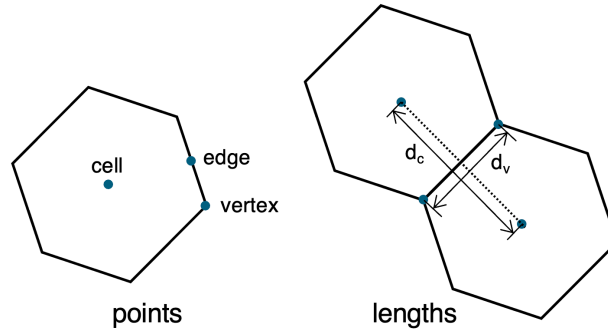


Figure 2.2: Horizontal mesh points and lengths used in MPAS.

The locations of the three points that are used to define the mesh are given in Figure 2.2:

1. The cell-center points, which are the centroids of the Voronoi cells. Note that they are also the vertices of the dual Delaunay triangular mesh cell (see figure 2.1).
2. The vertex points, which are the vertices of the Voronoi cells and the cell centers for the dual Delaunay triangular mesh cell.
3. The edge points, which are located where the lines connecting the cell centers intersect the cell edges (the lines connecting the cell vertices). The edge points bisect the lines connecting the cell centers. However, the edge points do not necessary bisect the cell edges.

Two horizontal meshes are supported in MPAS-Atmosphere, meshes on Cartesian planes and meshes on the sphere. Points are defined on the sphere by their locations in an (x, y, z) coordinate system with its origin at the center of the sphere. MPAS also keeps the latitude and longitude locations of the points on the sphere. The (x, y, z) locations of the points as a function of the latitude ϕ and the longitude λ are given by

$$\begin{aligned}x &= r \cos(\lambda) \cos(\phi), \\y &= r \sin(\lambda) \cos(\phi), \\z &= r \sin(\phi),\end{aligned}$$

where r is the radius of the sphere. Conversely,

$$\phi = \arcsin\left(\frac{z}{r}\right), \quad \lambda = \arctan\left(\frac{y}{x}\right)$$

The Cartesian plane is defined by $(x, y, z = 0)$, and the latitudes and longitudes are set to zero unless an $f - plane$ approximation is used.

MPAS code: Information about the MPAS mesh is defined in the MPAS [Registry.xml](#) file that can be found at [MPAS-Model/src/core_atmosphere/Registry.xml](#) for the atmospheric model and in [MPAS-Model/src/core_init_atmosphere/Registry.xml](#) for the atmospheric initialization utility. There are `nCells` cell-center points (and hence cells) in an MPAS mesh, and the locations of these points are defined in the Fortran arrays `xCell(nCells)`, `yCell(nCells)`, `zCell(nCells)` and `latCell(nCells)` and `lonCell(nCells)`. Similarly, there are `nEdges` edges and edge points defined in the arrays `xEdge`, `yEdge`, `zEdge` and `latEdge` and `lonEdge`. There are `nVertices` vertex points (and cells in the dual Delauney triangular mesh) defined in the arrays `xVertex`, `yVertex`, `zVertex`, and `latVertex` and `lonVertex`. The (x, y, z) points on the sphere have units of meters.

Note - unstructured MPAS mesh: The MPAS mesh is unstructured, and the order of the points on the mesh is arbitrary. For example, cell 42 will be located at (x, y, z) points `xCell(42)`, `yCell(42)`, `zCell(42)` and latitude and longitude `latCell(42)` and `lonCell(42)`. Likewise for edges and vertices. Neighbor cells (that share an edge) may have any other index between 1 and `nCells`. How to find neighbors, i.e. the connectivity of the mesh, is discussed in section 2.1.3.

2.1.2 Lines and arcs on the MPAS horizontal mesh

There are two classes of lines that define the MPAS horizontal mesh - the lines connecting the cell centers defining the dual Delauney triangular mesh, and the lines connecting vertices that define the cell edges. These lines are depicted in figure 2.2, and are defined by the points they connect and their length. The points and their definition in the sphere and plane have been discussed in section 2.1.1. The line lengths on the sphere are the lengths of the great circle arcs on the sphere connecting the points. The line lengths on the plane are the lengths of the straight lines connecting the points.

MPAS code: In `MPAS-Model/src/core_atmosphere/Registry.xml`, the line lengths for lines connecting cell centers across an edge are stored in the MPAS array `dcEdge(nEdges)`, and the cells at the endpoints of the line are stored in the array `cellsOnEdge(2,nEdges)`. The line lengths for lines connecting vertices (edge lengths) are stored in the MPAS array `dvEdge(nEdges)`, and the Vertices at the endpoints of the edge are stored in the array `verticesOnEdge(2,nEdges)`. The lengths have units of meters.

2.1.3 Connectivity

Components of a cell

Cells are composed of a cell center, edges, and vertices. The edges are bounded by vertices. As an illustration, figure 2.3 shows the cells surrounding cell 42 in the left panel, the edges in the center panel, and the vertices in the right panel. MPAS contains arrays that contain this information.

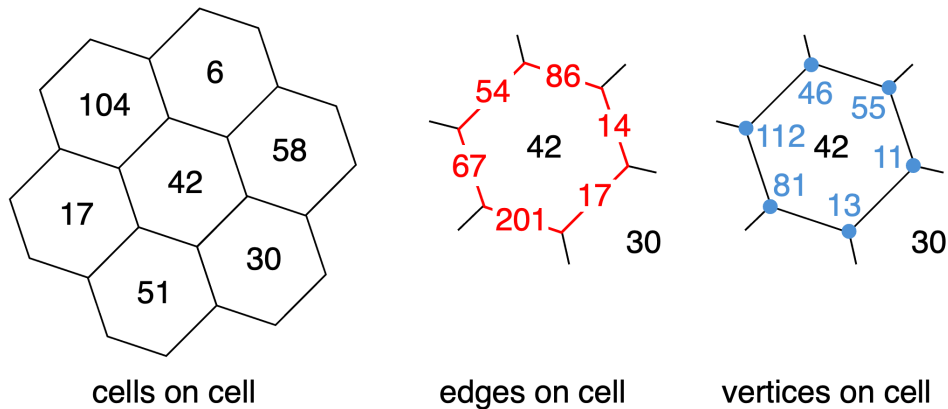


Figure 2.3: Cell, edge and vertex neighbors of a cell. Cell indices are given in black, edge indices are red and vertex indices are given in blue.

MPAS code: Following figure 2.3, the array containing the neighbors of cell 42 has the values `cellsOnCell(1:6,42) = (30, 58, 6, 104, 17, 51)`. By convention the order is counterclockwise around the cell. Likewise, the edges around the cell are given as `edgesOnCell(1:6,42) = (17, 14, 86, 54, 67, 201)`. By convention the edges are taken in the same order and start with the edge shared by the center cell and the first cell in the `cellsOnCell` list, i.e. `cellsOnCell(1,42) = 30`. Lastly, vertices for a cell are given as `verticesOnCell(1:6,42) = (13, 11, 55, 46, 112, 81)` and follow the counterclockwise order convention.

Edge and vertex neighbors

Each edge in an MPAS mesh is shared by two cells and connects two vertex points. Each vertex has three edges that meet at the vertex and three cells that share the vertex.

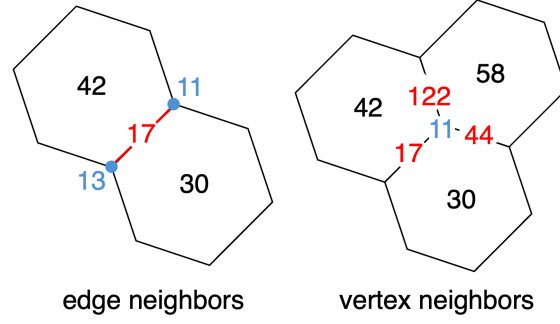


Figure 2.4: Neighbors of edges and vertices. Cell indices are given in black, edge indices are red and vertex indices are given in blue.

MPAS code: In the example given in the left figure in figure 2.4, the cells sharing edge 17 are given by $\text{cellsOnEdge}(1:2,17) = (30, 42)$. By convention the lower number cell is given first followed by the higher number cell. Similarly, the vertices on the edge are given by $\text{verticesOnEdge}(1:2,17) = (11, 13)$. This also follows a convention that will be discussed in section 2.1.4. The right figure in figure 2.4 shows the neighbor components of vertex 11. The cells sharing vertex 11 are given by $\text{cellsOnVertex}(1:3,11) = (30, 58, 42)$ and the edges meeting at the vertex are given as $\text{edgesOnVertex}(1:3,11) = (17, 44, 122)$.

2.1.4 Horizontal velocity on the MPAS horizontal mesh

Two horizontal velocities are defined on the MPAS mesh, the edge-normal velocity, denoted as u in the MPAS code, and the edge-tangential velocity that is denoted as v . These velocities are defined following the convention given in figure 2.5. By convention, a positive edge-normal velocity u at edge $Edge$ points from the cell index $\text{cellsOnEdge}(1,Edge)$ to cell index $\text{cellsOnEdge}(2,Edge)$. Also by convention, $\text{cellsOnEdge}(1,Edge)$ is the smaller the two cell indices. A positive tangential velocity v points from $\text{verticesOnEdge}(1,Edge)$ to $\text{verticesOnEdge}(2,Edge)$. These velocities follow a right-hand rule - the cross product of the unit vectors at the edge points corresponding to positive u and v point outward from the sphere or Cartesian plane.

2.1.5 Areas on the MPAS horizontal mesh

Three areas are defined on the MPAS mesh and they are depicted in figure 2.6: The Voronoi mesh cell areas, the areas of the Delauney triangular (dual) mesh, and the areas formed by the intersection of a Voronoi cell and a Delauney triangle, denoted as kite areas. The areas are on the plane for the Cartesian plane configuration of MPAS, and the area on the surface of the sphere for the spherical configuration of MPAS.

MPAS code: The arrays containing the areas for the MPAS mesh are defined in the *Registry.xml* file, [MPAS-Model/src/core_atmosphere/Registry.xml](#), and are contained in the arrays $\text{areaCells}(nCells)$, $\text{areaTriangle}(nVertices)$ and $\text{kiteAreasOnVertex}(3,nVertices)$. The kiteAreasOnVertex follow the same ordering as cellsOnVertex , e.g. the kite area $\text{kiteAreasOnVertex}(1,Vertex)$ will be the kite area associated with cell $\text{cellsOnVertex}(1,Vertex)$, and likewise for 2 and 3.

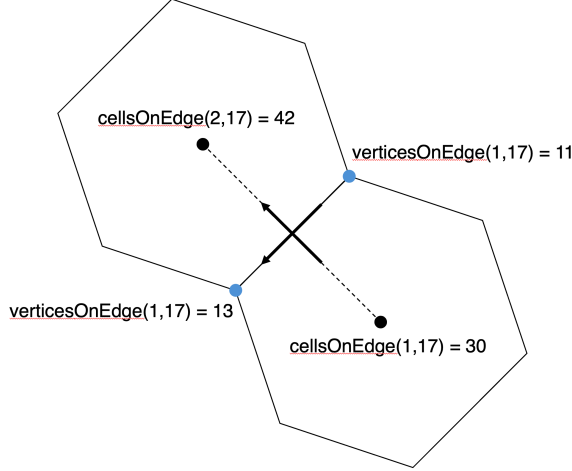


Figure 2.5: Definition of the prognostic edge-normal horizontal velocity and the diagnostic horizontal edge-tangential velocity using the example mesh given in figure 2.4.

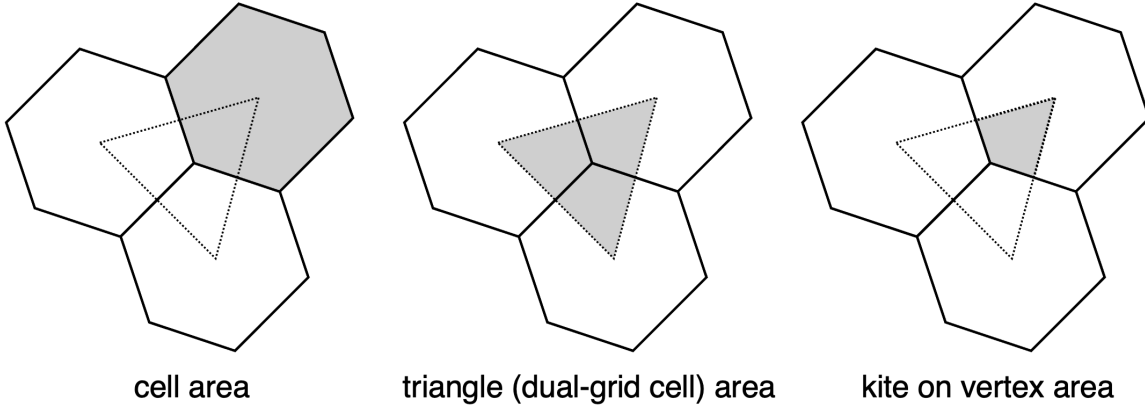


Figure 2.6: Horizontal mesh areas used in MPAS.

2.2 MPAS Vertical Grid

In this section we provide an overview of the computational vertical coordinate used in MPAS and its relation to the physical heights of layers and interfaces in the column, along with a description of the hybrid smoothed 3D mesh that is used in MPAS-Atmosphere.

MPAS-Atmosphere uses a computational vertical coordinate ζ that represents geometric height in the absence of topography, in contrast to the scaled hydrostatic pressure (mass) vertical coordinate used in WRF (Skamarock et al., 2021a). The Jacobian $\partial\zeta/\partial z$ is used to transform the governing equations cast in terms of z to those using ζ (see chapter 3).

The vertical grid in MPAS is structured - the vertical coordinate axis points outward from the center of the earth (or upward from the Cartesian plane), and quantities defined on the vertical axis have neighbors adjacent in the arrays in which they are stored. The MPAS vertical grid defines variables at *layers* and at layer *interfaces*. The interfaces are where the vertical velocities Ω and w , and the height above sea level z (or above the Cartesian plane) are defined.

The layers are where all other variables (u, ρ, θ, q , etc) are defined, and by definition the layer heights are halfway between the interface heights. For example, referring to figure 2.7:

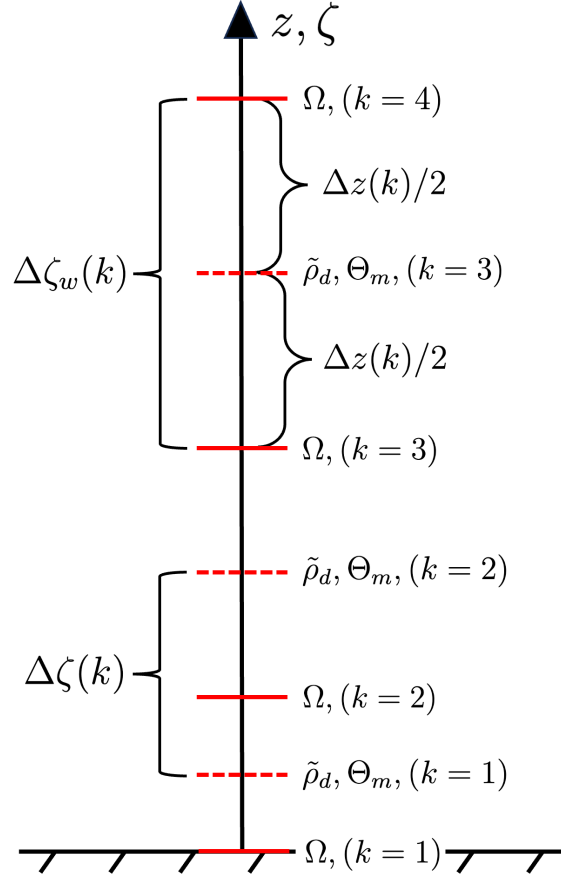


Figure 2.7: The MPAS-Atmosphere vertical grid with the locations of the staggered variables. The dashed red lines indicate the center of the layers, and solid lines are interfaces.

$$z_{layer}(2) = [z_{int}(3) + z_{int}(2)] / 2.$$

2.2.1 Specifying the interface heights

The computational vertical coordinate ζ represents the heights of the layer interfaces and layer midpoints in the absence of topography. As noted earlier, the layer heights are just the average of the bounding interface heights, so ζ is completely specified by the interface heights. In the following, we describe options to specify the heights ζ available in the real data configurations; idealized configurations have different (usually hardwired) configurations that will be discussed in chapter 8.

One option in the MPAS initialization sequence is to specify the interface heights in an ascii file supplied by the user.

MPAS code: User specified levels are enabled by supplying an ascii file with the levels and setting the configuration variable `config_specified_zeta_levels` in the `namelist.init_atmosphere` to the name of the ascii file. If no file is specified (the default configuration) then an analytic specification of the interface level distribution is used. Also see the MPAS Users Guide available at https://mpas-dev.github.io/atmosphere/atmosphere_download.html

The MPAS model also provides analytic functions to define the interface layer heights ζ . For a given number of interface levels n , where level 1 is the surface, $\zeta(1) = 0$ and $\zeta(n) = z_{top}$, $\zeta(k)$ is given by

$$\zeta(k) = z_{top} \left(\frac{k-1}{n-1} \right)^{1.5} \quad k = 1 \dots n \quad (2.1)$$

MPAS code: Use of (2.1) is enabled by setting the `namelist.init_atmosphere` variable `config_tc_vertical_grid = false`. The variable `z_top` is available through the `namelist.init_atmosphere` variable `config_ztop` and the units are meters.

Another option for distributing the ζ levels given using the following function form:
For $(k-1)/(n-1) < \zeta_l$

$$\begin{aligned} \zeta(k) = z_{top} \Bigg[& a_s \left(\frac{k-1}{n-1} \right) \\ & + [3(1-a_t) + 2(a_t-a_s)\zeta_l] \left(\frac{1}{\zeta_l} \frac{(k-1)}{(n-1)} \right)^2 \\ & - [2(1-a_t) + (a_t-a_s)\zeta_l] \left(\frac{1}{\zeta_l} \frac{(k-1)}{(n-1)} \right)^3 \Bigg]. \end{aligned} \quad (2.2)$$

else

$$\zeta(k) = z_{top} \left(1 - a_t(1 - \zeta_l) + a_t \left[\left(\frac{k-1}{n-1} \right) - \zeta_l \right] \right). \quad (2.3)$$

MPAS code: Use of (2.2) and (2.3) is enabled by setting the `namelist.init_atmosphere` variable `config_tc_vertical_grid = true`, otherwise (2.1) is used. The coefficients $(a_s, a_t, \zeta_l) = (0.075, 1.23, 0.31)$ produce a ζ distribution similar to that given in the default WRF model configuration using 41 interface levels, and these values are used for configurations having less than 55 interface levels. For 55 or greater vertical interface levels the value $a_t = 1.70$ is used and the other coefficients are unchanged. The analytic formula are available for real-data cases and are used in the initialization code [MPAS-Model/src/core_init_atmosphere/mpas_init_atm_cases.F](#) in `subroutine init_atm_case_gfs`.

We note here that users could replace any of these analytic functions for ζ with their own function if desired.

2.2.2 Hybrid smoothed terrain-following coordinate

Given the 1D computational coordinate ζ , the the physical heights $z(\zeta, \vec{x})$ of the interface levels must be set. To accomplish this MPAS-Atmosphere employs a hybrid smoothed terrain-following vertical coordinate described in [Klemp \(2011, equation 4\)](#):

$$z = \zeta + Ah_s(\zeta, \vec{x}), \quad (2.4)$$

where z is the geometric height, ζ is the vertical computational coordinate where $0 \leq \zeta \leq z_{top}$ for a model top (lid) at height z_{top} , and $\vec{x}$ is a horizontal position vector. $A(\zeta)$ controls the rate at which the coordinate transitions from terrain following at the surface toward constant height surfaces aloft,

$$\begin{aligned} A(\zeta) &= \cos^6 \left(\frac{\pi}{2} \frac{\zeta}{z_H} \right) & \text{for } \zeta < z_H \\ A(\zeta) &= 0 & \text{for } \zeta \geq z_H, \end{aligned} \quad (2.5)$$

where $z_H = 30$ km is the default value for MPAS-Atmosphere. $h_s(\zeta, \vec{x})$ represents the terrain influence in the vertical-coordinate, and smoother versions of the terrain are used for increasing ζ . The smoother representations of the terrain are generated using multiple passes of a horizontal Laplacian filter. Denoting the actual terrain elevation as h , and the discrete model level ζ at the surface as model level $n = 1$, we can cast the smoothing operator for increasing level n as

$$\begin{aligned} h_s^{(1)} &= h \\ h_s^{(n)} &= \left[1 + \beta(\zeta, \vec{x}) \tilde{\nabla}_\zeta^2 \right]^m h_s^{(n-1)}. \end{aligned} \quad (2.6)$$

The horizontal (on ζ levels) dimensionless Laplacian is discretized as

$$\tilde{\nabla}_\zeta^2 h_s^{(n-1)} = \sum_{e_c} \frac{d_v}{d_c} \delta_e(h_s^{(n-1)}).$$

In the current MPAS-Atmosphere release, the coefficient β multiplying the Laplacian is dependent on its horizontal and vertical position on the computational mesh:

$$\beta(\zeta, \vec{x}) = \max \left[0.01, \frac{1}{8} \min \left(1.0, \frac{3000}{\overline{d_c}(\vec{x})} \right) \right] \times \min \left[\left(\frac{3\zeta}{z_H} \right)^2, 1.0 \right]. \quad (2.7)$$

This formulation includes a function of the horizontal cell spacing $d_c(\vec{x})$ that decreases the smoothing coefficient as the cell spacing increases that is not present in the original [Klemp \(2011\)](#) specification. This factor provides some compensation for the increased smoothing that is inherent in interpolating the actual terrain to a coarser horizontal mesh, and provides more consistency in applying the smoothed coordinate over a variable resolution MPAS mesh. Finally, the number of smoothing passes at each vertical coordinate level is defined as $m(n) = N_s + n$ with the default value of N_s set at 30. This also differs from [Klemp \(2011\)](#) where m was specified as a constant. Since the smaller scale terrain features are removed more efficiently by the Laplacian filter, increasing m with height allows greater smoothing of larger larger scale terrain influences that become more dominate at higher levels.

At each application $i = 1, m$ of the smoother (2.6) the resulting heights are bounded so as to limit the minimum spacing between levels in a given column. Given a smoothed preliminary value of $h_s(k)$ at level k , the value is checked to see if the level spacing is above a minimum value:

$$[z(k) - z(k-1)]_{min} < \gamma [\zeta(k) - \zeta(k-1)]. \quad (2.8)$$

where the default value for $\gamma = 0.3$. On a given iteration i of the smoother, if (2.8) is not satisfied the increment to z is not applied and the iteration is halted.

Figure 2.8 shows an example of the coordinate surfaces from a traditional terrain following coordinate where $A(\zeta)$ is a linear function with a value of 1 at the surface and 0 at the model top and $h_s = h$, and the smoothed hybrid terrain following coordinate.

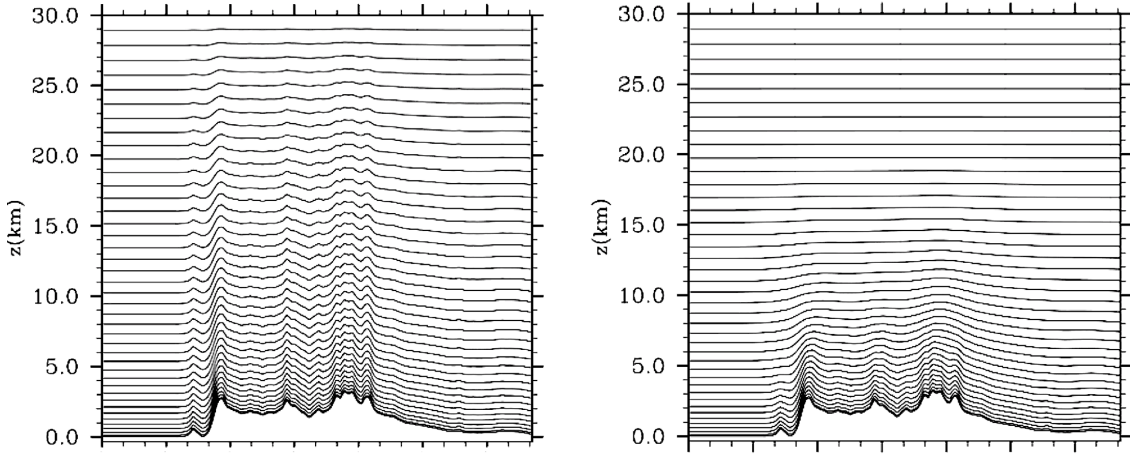


Figure 2.8: The terrain following coordinate used in MPAS. The left figure shows the traditional terrain following coordinate surfaces and the right figure shows the smoothed vertical coordinate.

MPAS code: For earth applications, the generation of the full 3D mesh and its coordinate surfaces occurs in the initialization *subroutine* `init_atm_case_gfs` in `MPAS-Model/src/core_init_atmosphere/mpas_init_atm_cases.F`. The height where the coordinate surface transitions from hybrid terrain following to constant height at z_H is set in the initialization code to 30 km.

2.2.3 Vertical Interpolation

There is a need to interpolate layer variables to interfaces, and to interpolate interface variables to layers. For the latter, we take the average of the interface variables values from the two surrounding interface to set the layer value given that a layer lies halfway between the interfaces. We have two methods to interpolate layer values to an interface. The first is to do a linear in ζ interpolation of the layer value to the interface. Taking into account that the layers lie halfway between interfaces, we can write this interpolation as

$$\phi_{int}(k) = \frac{\Delta\zeta_w(k-1)}{\Delta\zeta_w(k) + \Delta\zeta_w(k-1)} \phi_{layer}(k) + \frac{\Delta\zeta_w(k)}{\Delta\zeta_w(k) + \Delta\zeta_w(k-1)} \phi_{layer}(k-1). \quad (2.9)$$

The second approach is to vertically integrate the quantity between the layers containing the interface and divide by the height difference between the two layers. We assume the quantity is constant between the interfaces, and in this case the result simply switches the weights in (2.9) between the two layer values, i.e.

$$\phi_{int}(k) = \frac{\Delta\zeta_w(k)}{\Delta\zeta_w(k) + \Delta\zeta_w(k-1)} \phi_{layer}(k) + \frac{\Delta\zeta_w(k-1)}{\Delta\zeta_w(k) + \Delta\zeta_w(k-1)} \phi_{layer}(k-1). \quad (2.10)$$

MPAS code: *The weights for the vertical interpolation of values from a layer to an interface are computed and stored during model initialization, where a user can choose between a linear interpolation (2.9) or the integrated average value (2.10). In MPAS the weights are stored in the arrays `fzm` and `fzp`, where the weight `fzm` multiplies $\phi_{layer}(k)$ in (2.9) and (2.10), and likewise `fzp` multiplies $\phi_{layer}(k-1)$. In some places in the MPAS code the array `fnm` points to `fzm` and `fnp` points to `fzp`.*

Chapter 3

Equations and Time Integration

The MPAS-A dynamics solver integrates the compressible, nonhydrostatic Euler equations and a brief description of the solver is presented in [Skamarock et al. \(2012\)](#). The equations are cast in flux form using variables that have conservation properties, following the philosophy of [Ooyama \(1990\)](#). The equations are formulated using geometric height as an independent variable. The vertical coordinate is terrain following, using a hybrid $\sigma - z$ formulation that includes a filtering of the terrain effect for higher coordinate surfaces following [Klemp \(2011\)](#) as described in section [2.2](#).

In this chapter we present the moist flux-form equations cast in coordinates for the sphere, and we describe the time integration scheme used in the MPAS-A solver. The current MPAS-A release uses equations employing the shallow-atmosphere approximation, and we focus on these equations in the next two chapters. The perturbation equations we employ in MPAS-A are formulated to facilitate the integration scheme we employ, particularly the acoustic splitting used in the MPAS-A time integration. This motivates our presentation of the equations and time integration together in this chapter.

3.1 Continuous Equations

MPAS uses the spherical centroidal Voronoi tessellation (mesh) (SCVT) for horizontal tiling of the sphere which is described in Chapter [2](#). Contrary to a latitude-longitude grid, this mesh is not based on a global coordinate hence equations are cast in general form. For the vertical coordinate, MPAS employs a height-based terrain-following vertical coordinate ζ following [Klemp \(2011\)](#) as described in section [2.2](#). The coupled prognostic variables in this coordinate are defined as

$$\tilde{\rho}_d = \rho_d / \zeta_z, \quad (3.1)$$

$$\mathbf{V}_H = \tilde{\rho}_d \mathbf{v}_H, \quad (3.2)$$

$$W = \tilde{\rho}_d w, \quad (3.3)$$

$$\Theta_m = \tilde{\rho}_d \theta [1 + (R_v/R_d)q_v], \quad (3.4)$$

$$Q_j = \tilde{\rho}_d q_j, \quad (3.5)$$

where the vertical derivative of the computational coordinate $\zeta_z = \partial\zeta/\partial z$, ρ_d is the dry-air density, $\mathbf{v}_H$ and w represent the horizontal and vertical velocities ($u\mathbf{i}, v\mathbf{j}, w\mathbf{k}$) where $\mathbf{i} \times \mathbf{j} = \mathbf{k}$,

θ is the potential temperature, q_v is the water vapor mixing ratio with respect to the dry-air density ρ_d , q_j are the mixing ratios of other moisture constituents and other scalars, and R_d and R_v are the gas constants for dry air and water vapor, respectively. The full continuous equations for both deep- and shallow-atmosphere configurations can be cast as

$$\begin{aligned} \frac{\partial \mathbf{V}_H}{\partial t} = & -\frac{\rho_d}{\rho_m} \left[\nabla_\zeta \left(\frac{p}{\zeta_z} \right) + \frac{\partial}{\partial \zeta} \left(\frac{\zeta_H p}{\zeta_z} \right) \right] - \eta \mathbf{k} \times \mathbf{V}_H - \tilde{\rho}_d \nabla_\zeta K \\ & - \mathbf{v}_H \nabla_\zeta \cdot \mathbf{V} - \frac{\partial \Omega \mathbf{v}_H}{\partial \zeta} + \mathbf{F}_{V_H} - \beta_d \tilde{\rho}_d \left(ew \mathbf{i} + \frac{\mathbf{v}_H w}{r} \right), \end{aligned} \quad (3.6)$$

$$\frac{\partial W}{\partial t} = -\frac{\rho_d}{\rho_m} \left[\frac{\partial p}{\partial \zeta} + g \left(\frac{r_o}{r} \right)^2 \tilde{\rho}_m \right] - (\nabla \cdot \mathbf{V} w)_\zeta + F_W + \beta_d \tilde{\rho}_d \left(\frac{\mathbf{v}_H \cdot \mathbf{v}_H}{r} + ew_H \cdot \mathbf{i} \right), \quad (3.7)$$

$$\frac{\partial \Theta_m}{\partial t} = -(\nabla \cdot \mathbf{V} \theta_m)_\zeta + F_{\Theta_m}, \quad (3.8)$$

$$\frac{\partial \tilde{\rho}_d}{\partial t} = -(\nabla \cdot \mathbf{V})_\zeta, \quad (3.9)$$

$$\frac{\partial Q_j}{\partial t} = -(\nabla \cdot \mathbf{V} q_j)_\zeta + F_{Q_j}. \quad (3.10)$$

The equation of state,

$$p = p_o \left(\frac{R_d \Theta_m}{p_o} \right)^{c_p/c_v} \quad (3.11)$$

is used to diagnose pressure. The moist density $\rho_m = \rho_d(1 + \sum q_m)$, where q_m are mixing ratios for water vapor and all hydrometeors. $\beta_d = 1$ for the deep-atmosphere equations, and the traditional shallow-atmosphere approximation is recovered when $\beta_d = 0$ and the radial distance from the center of the sphere r is approximated by the constant sphere radius r_o . The gravity g is defined by its value at the sphere surface. The horizontal momentum equation (3.6) is not cast in conservative (flux) form. In (3.6) the absolute vertical vorticity $\eta = \mathbf{n} \cdot \nabla \times \mathbf{v}_H + f$, where $\mathbf{n}$ is the unit vector normal to a ζ surface, and $K = |\mathbf{v}_H|^2/2$ is the horizontal kinetic energy. In the curvature and Coriolis terms in (3.6) and (3.7), $f = 2\Omega_e \sin \psi$, $e = 2\Omega_e \cos \psi$, ψ is the latitude, Ω_e is the angular rotation rate of the Earth. The terms $\mathbf{F}_{V_H}$, F_W , F_{Θ_m} and F_{Q_j} represent sources and sinks from physics, sub-grid models and filters. The mass-flux normal to the horizontal coordinate surface is $\Omega = \mathbf{V} \cdot \nabla \zeta$ and the corresponding normal velocity is $\omega = \dot{\zeta}$. The flux divergence

$$(\nabla \cdot \mathbf{V} \phi)_\zeta = \nabla_\zeta \cdot (\mathbf{V}_H \phi) + \frac{\partial(\Omega \phi)}{\partial \zeta},$$

where $\mathbf{V}_H$ and $\mathbf{v}_H$ are the horizontal momentum and velocity, respectively.

The current release of MPAS, Version 8, does not contain the solver for the the deep atmosphere (full) equations, rather MPAS-Atmosphere employs the shallow atmosphere approximation in the release. A MPAS-Atmosphere solver for the full equations (3.6) - (3.10) has been developed and tested as reported in Skamarock et al. (2021a).

3.2 Time Integration Overview

MPAS integrates the equations of motion using a Runge-Kutta (RK3) time integration scheme, described in Wicker and Skamarock (2002), within which a split-explicit time integration tech-

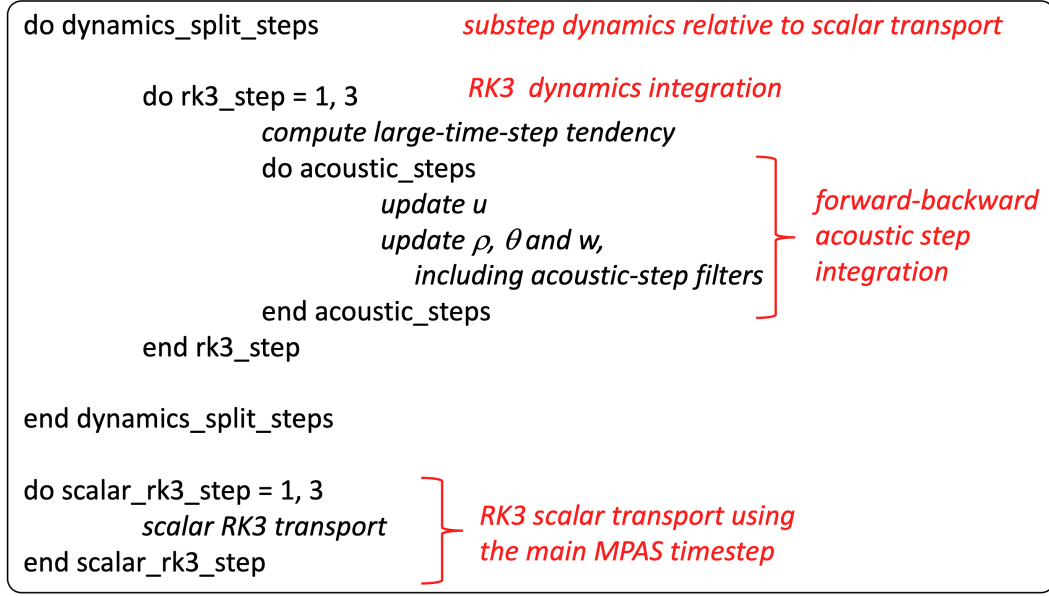


Figure 3.1: The MPAS time integration methodology.

nique is employed to integrate acoustic and gravity wave modes (Klemp et al., 2007). For the dry dynamics, prognostic equations (3.6)-(3.9) for the density-coupled horizontal velocity, vertical velocity, potential temperature Θ_m , and the dry air density are integrated in the RK3 split-explicit dynamics time step. Other scalars, e.g. water vapor, precipitation species, number concentrations in microphysics, etc, are integrated in a separate Runge-Kutta step after the dynamics are advanced.

The MPAS time step integration sequence is depicted in figure 3.1. The main MPAS time step, *config_dt* in the namelist.atmosphere, is the time step for the scalar transport which is performed as the final process in the time integration, and this scalar integration appears as the final loop in the pseudo-code in figure 3.1. The dynamics can be integrated with a smaller time step than the main MPAS time step, and this time step is specified through the namelist.atmosphere parameters as *config_dt/config_dynamics_split*, where *config_dynamics_split* is an integer. The loop for the dynamics steps is the first loop in figure 3.1, and within it are the RK3 substeps within which appear a loop for the acoustic time steps that integrate each RK3 substep.

MPAS code: The time step integration is performed in *subroutine atm_srk3* in *MPAS-Model/src/core_atmosphere/dynamics/MPAS_atm_time_integration.F*. The dynamics RK3 integration is in the named loop *rk3_dynamics*. The number of dynamics steps taken each transport step is given by the namelist.atmosphere variable *config_dynamics_split* and the loop executing these steps is the named loop *dynamics_substeps*. The acoustic steps inside the dynamics integration are controlled by the named loop *dynamics_acoustic_steps*. The scalar RK3 transport is performed in the named loop *rk3_split_transport*.

In the following sections we will describe the RK3 time integration scheme and the perturbation equations it employs, followed by a description of the splitting of the acoustic and gravity-wave modes (the acoustic steps within the dynamics integration), a summary of the dynamics step, and a summary of the scalar transport time integration.

3.3 RK3 Time Integration Scheme

The RK3 time integration scheme, described in [Wicker and Skamarock \(2002\)](#), is comprised of three steps to advance a solution over a time step Δt . To advance a variable ϕ from time t to time $t + \Delta t$ using RK3, where $\phi_t = L(\phi)$,

$$\phi^* = \phi^t + \frac{\Delta t}{3} L(\phi^t), \quad (3.12)$$

$$\phi^{**} = \phi^t + \frac{\Delta t}{2} L(\phi^*), \quad (3.13)$$

$$\phi^{t+\Delta t} = \phi^t + \Delta t L(\phi^{**}). \quad (3.14)$$

The scheme (3.12)-(3.14) is third-order accurate in time for linear systems and second-order for nonlinear systems. A variant of this scheme available in MPAS replaces the coefficient $\Delta t/3$ in (3.12) with $\Delta t/2$, and the resulting scheme is second-order accurate for linear and nonlinear systems. The default MPAS configuration uses the second order variant because it allows for a larger time step as shown in linear analysis results given in figure 3.2.

MPAS code: The RK3 time-step coefficients are set in two places in the main MPAS time integration driver in

[MPAS-Model/src/core_atmosphere/dynamics/MPAS_atm_time_integration.F](#), in *subroutine atm_sr3*. The first location is before the main Runge-Kutta loop for integrating the dry dynamics, and the coefficients are used for the named loop *rk3_dynamics*. The second location is immediately before the named loop *rk3_split_transport* where, as the name implies, an RK3 integration of the scalar transport is accomplished.

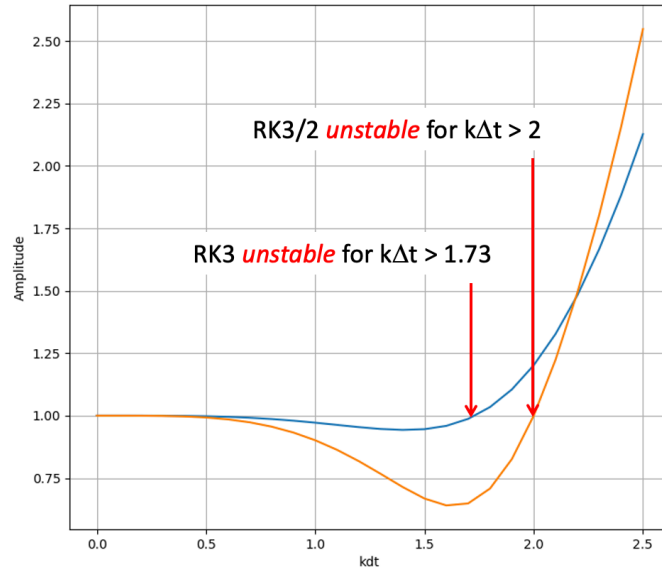


Figure 3.2: Runge-Kutta time integration response.

3.3.1 Perturbation Equations Employed in the RK3 Solver

The shallow atmosphere approximation is recovered by setting $\beta_d = 0$ and $r = r_o$ in (3.6) - (3.10). We describe the discrete solver for the shallow atmosphere equations in the following chapters, so for clarity and ease of reference we will describe the shallow-atmosphere equations and solver in the rest of this note.

Following Klemp et al. (2007), two sets of perturbations are used in the MPAS-A solver. Perturbation variables are defined about a hydrostatically-balanced thermodynamic state that are only a function of the geometric height z : $\rho_d = \bar{\rho}_d(z) + \rho'_d$, and $\Theta_m = \bar{\rho}_d(z)\bar{\theta}_m(z) + \Theta'_m$. The reference state ($\bar{\rho}_d$ and $\bar{\theta}_m$) satisfies the equation of state (3.11) that defines the reference pressure $\bar{p}$ and hence a perturbation pressure $p' = p - \bar{p}(z)$. The thermodynamic reference state is for a dry atmosphere ($\bar{q}_m = 0$). Incorporating these perturbation variables into (3.6) - (3.10), making the shallow atmosphere approximation, and temporally discretizing the equations (3.6)-(3.10) yields:

$$\begin{aligned} \delta_t \mathbf{V}_H = & \left[-\frac{\rho_d}{\rho_m} \left(\nabla_\zeta \left(\frac{p'}{\zeta} \right) + g z_H \tilde{\rho}'_m \right) - \eta \mathbf{k} \times \mathbf{V}_H - \tilde{\rho}_d \nabla_\zeta K \right. \\ & \left. - \mathbf{v}_H \nabla_\zeta \cdot \mathbf{V} - \frac{\partial \Omega \mathbf{v}_H}{\partial \zeta} \right]_{t_1} \\ & + [\mathbf{F}_{V_H}]_{t_2}, \end{aligned} \quad (3.15)$$

$$\delta_t W = \left[-\frac{\rho_d}{\rho_m} \left(\frac{\partial p'}{\partial \zeta} + g \tilde{\rho}'_m \right) - (\nabla \cdot \mathbf{V} w)_\zeta \right]_{t_1} + [F_W]_{t_2}, \quad (3.16)$$

$$\delta_t \Theta'_m = \left[-(\nabla \cdot \mathbf{V} \theta_m)_\zeta \right]_{t_1} + [F_{\Theta_m}]_{t_2}, \quad (3.17)$$

$$\delta_t \tilde{\rho}'_d = \left[-(\nabla \cdot \mathbf{V})_\zeta \right]_{t_1}, \quad (3.18)$$

$$\delta_t Q_j = \left[-(\nabla \cdot \mathbf{V} q_j)_\zeta \right]_{t_1} + [F_{Q_j}]_{t_2}, \quad (3.19)$$

where the discrete time difference over the timestep Δt for an arbitrary variable ϕ is

$$\delta_t \phi = \frac{\phi^{t+\Delta t} - \phi^t}{\Delta t}, \quad (3.20)$$

the bracketed terms $[]_{t_*}$ denote terms evaluated at time t_* , and

$$\tilde{\rho}'_m = \tilde{\rho}'_d + \tilde{\rho}_d \sum q_j. \quad (3.21)$$

We have also recast the vertical pressure gradient correction in the horizontal momentum equation by its hydrostatic approximation:

$$\frac{\partial p'}{\partial \zeta} = -g \tilde{\rho}'_m. \quad (3.22)$$

which has been found to improve the accuracy in computing the horizontal pressure gradient by reducing truncation errors in the numerics (Klemp and Skamarock, 2021)

3.3.2 Operators in the RK3 Scheme

The operator $L(\phi)$ in (3.12)-(3.14) includes all the terms except for the partial derivatives with respect to time in the prognostics equations for the dynamics (3.15)-(3.18), i.e. all the terms to the right-hand-side of the equal signs. However, the dynamics integration using $L(\phi)$ in (3.12)-(3.14) involves evaluating only some of the RHS terms in (3.15)-(3.18) in each of the RK3 substeps. Specifically, the bracketed terms $[]_{t_1}$ in (3.15)-(3.18) are the terms evaluated from the states ϕ^t , ϕ^* , and ϕ^{**} . The other RHS terms in (3.15)-(3.18), given as bracketed terms $[]_{t_2}$, are evaluated at prescribed times during the integration. These other terms include atmospheric physics, whose tendencies may be updated either once per time step or even less frequently (see Chapter 9), and subgrid turbulence models and MPAS model filters, that usually are evaluated at the beginning of a time step at time t and held fixed over the time step. Additionally, some filters are evaluated on the acoustic timestep.

MPAS code: The RK3 tendencies for the dynamics, the bracketed terms $[]_{t_1}$ in (3.15)-(3.18), are computed in a call to *subroutine atm_compute_dyn_tend* from the main MPAS time integration routine *atm_sr3* in *MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F*.

For the $[]_{t_2}$ terms in (3.15)-(3.18), the terms associated with MPAS model filters and subgrid transport, aside from the column physics, are computed in *subroutine atm_compute_dyn_tend* during the first RK3 substep and saved. The tendencies from the column physics are acquired in the call to *subroutine physics_get_tend* in routine *atm_sr3* at the beginning of the main MPAS timestep. The dynamics tendencies, the saved physics tendencies and saved model filter and subgrid transport tendencies are summed in *subroutine atm_compute_dyn_tend* to construct $L(\phi)$ in each RK3 substep (3.12)-(3.14).

3.4 Acoustic Solver

Within the RK3 substeps, smaller acoustic steps are taken where only the terms responsible for acoustic modes are integrated while the terms responsible for the slower atmospheric modes are held fixed to their RK3 evaluation. Thus the Runge-Kutta time integration takes place within the acoustic steps where the precomputed RK3 tendencies are added to the prognostic variables. The combined timesteps are illustrated in figure 3.3 which includes the graphical relation of the acoustic timestep to the RK3 timestep in each RK3 substep. The RK3 substeps ($\Delta t/3, \Delta t/2, \Delta t$) are accomplished over the acoustic steps within them. In the following section we present the perturbation equations for the acoustic steps along with gravity wave and acoustic filters that are employed in MPAS-A.

3.4.1 Perturbation Equations Employed in the Acoustic Solver

The fully compressible equations contain acoustic modes that can severely limit the allowable timesteps in an explicit numerical time integration. MPAS employs a split-explicit vertically

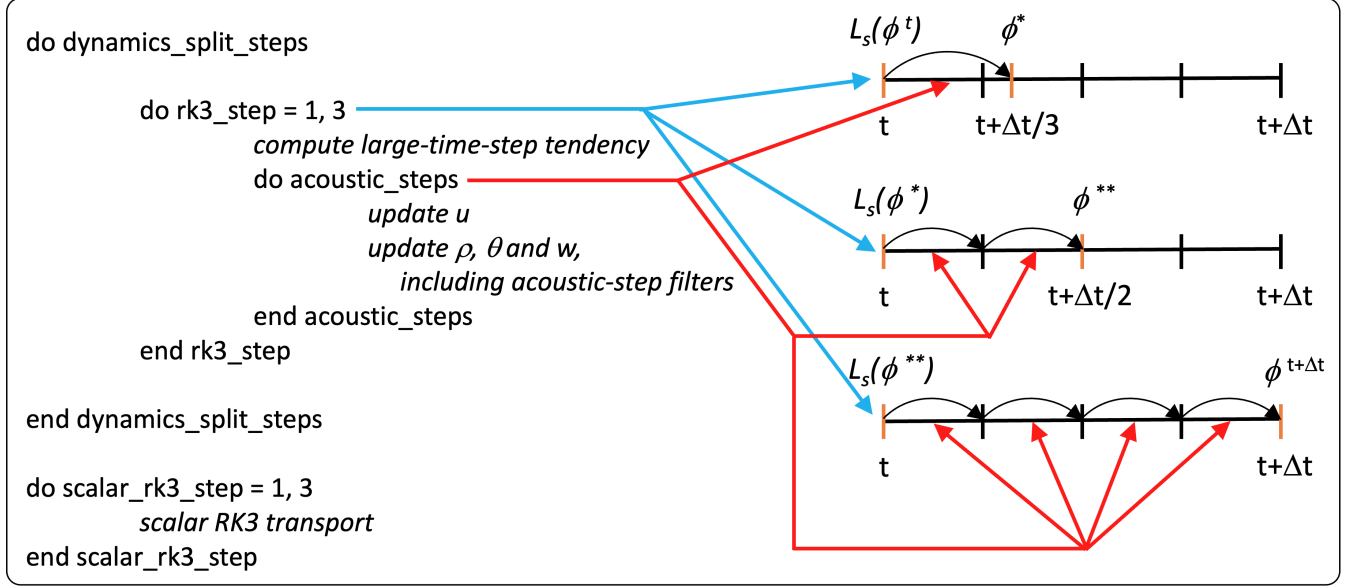


Figure 3.3: Integration timesteps in the split-explicit RK3 scheme.

implicit time integration scheme to advance the acoustic modes with smaller timesteps. This scheme is described in detail in [Klemp et al. \(2007\)](#). The acoustic step advances the dynamics variables defined as perturbations from their respective values at the beginning of the acoustic time steps at time t :

$$\begin{aligned}
 \mathbf{V}_h'' &= \mathbf{V}_h - \mathbf{V}_h^t \\
 W'' &= W - W^t \\
 \Theta_m'' &= \Theta_m' - \Theta_m'^t \\
 \tilde{\rho}_d'' &= \tilde{\rho}_d' - \tilde{\rho}_d'^t
 \end{aligned} \tag{3.23}$$

Additionally, we convert the vertical momentum equation for the coupled contravariant vertical velocity W'' to an equation for the covariant vertical velocity $\Omega = \Omega^t + \Omega''$ (i.e. the coupled velocity normal to the hybrid coordinate horizontal surfaces), defined as

$$\Omega = V_h \cdot \nabla \zeta + \zeta_z W, \tag{3.24}$$

so that we can use the upper and lower boundary conditions $\Omega = \Omega'' = 0$ at the top and bottom of the domain during the acoustic step integration. The acoustic integration follows [Klemp et al. \(2007\)](#) and employs a forward-backward time integration of the system, first advancing the horizontal momentum $\mathbf{V}_h''$ followed by advancing $\Omega'' = V_h'' \cdot \nabla \zeta + \zeta_z W''$, Θ_m'' and $\tilde{\rho}_d''$ with semi-implicit numerics, using the updated values of $\mathbf{V}_h''$. In terms of these double-prime perturbation variables, the acoustic step equations are very similar to equations (13)-(16) in [Klemp et al. \(2007\)](#), except we substitute the equation for W'' with that for Ω'' and we employ a slightly

different linearization as discussed below.

$$\delta_\tau \mathbf{V}_h'' = -\frac{\rho_d^t}{\rho_m^t} [\gamma R_d \pi^t \nabla_\zeta \Theta_m''^\tau + g z_H \tilde{\rho}_d''^\tau] + \mathbf{R}_{V_h}^t \quad (3.25)$$

$$\delta_\tau \Omega'' + \zeta_z \frac{\rho_d^t}{\rho_m^t} \left[\gamma R_d \pi^t \partial_\zeta (\zeta_z \overline{\Theta_m''}^\tau) - g \tilde{\rho}_m^t \frac{R_d}{c_v} \frac{\overline{\Theta_m''}^\tau}{\Theta_m^t} \right] + \zeta_z g \tilde{\rho}_d''^\tau = \mathbf{R}_{V_H}^t \cdot \nabla_h \zeta + R_w^t \zeta_z \quad (3.26)$$

$$\delta_\tau \Theta_m'' + \nabla_\zeta \cdot (\mathbf{V}_h''^{\tau+\Delta\tau} \theta_m^t) + \partial_\zeta (\overline{\Omega''}^\tau \theta_m^t) = R_{\Theta_m}^t \quad (3.27)$$

$$\delta_\tau \tilde{\rho}'' + \nabla_\zeta \cdot \mathbf{V}_h''^{\tau+\Delta\tau} + \partial_\zeta (\overline{\Omega''}^\tau) = R_{\tilde{\rho}_d}^t \quad (3.28)$$

The operator

$$\overline{\phi}^\tau = \frac{1+\epsilon}{2} \phi^{\tau+\Delta\tau} + \frac{1-\epsilon}{2} \phi^\tau \quad (3.29)$$

is a time averaging of ϕ and represents the vertically semi-implicit time differencing of the vertically-propagating acoustic modes and buoyancy oscillation modes in the compressible non-hydrostatic system. For the system (3.28)-(3.26) the vertically semi-implicit scheme will be second-order accurate and neutral (non-damping) for $\epsilon = 0$ and will be damping for $\epsilon > 0$. This damping mechanism is discussed further in section 5.4. The terms $\mathbf{R}_{V_h}^t$, R_w^t , $R_{\Theta_m}^t$, and $R_{\tilde{\rho}_m}$ in (3.25)-(3.28) are the RHS terms in (3.15)-(3.18), that is

$$\begin{aligned} \mathbf{R}_{V_h}^t = & \left[-\frac{\rho_d}{\rho_m} \left(\nabla_\zeta \left(\frac{p'}{\zeta_z} \right) + g z_H \tilde{\rho}_m' \right) - \eta \mathbf{k} \times \mathbf{V}_H - \tilde{\rho}_d \nabla_\zeta K \right. \\ & \left. - \mathbf{v}_H \nabla_\zeta \cdot \mathbf{V} - \frac{\partial \Omega \mathbf{v}_H}{\partial \zeta} \right]_{t_1} + [\mathbf{F}_{V_h}]_{t_2}, \end{aligned} \quad (3.30)$$

$$R_W^t = \left[-\frac{\rho_d}{\rho_m} \left(\frac{\partial p'}{\partial \zeta} + g \tilde{\rho}_m' \right) - (\nabla \cdot \mathbf{V} w)_\zeta \right]_{t_1} + [F_W]_{t_2}, \quad (3.31)$$

$$R_{\Theta_m}^t = \left[-(\nabla \cdot \mathbf{V} \theta_m)_\zeta \right]_{t_1} + [F_{\Theta_m}]_{t_2}, \quad (3.32)$$

$$R_{\tilde{\rho}_d}^t = \left[-(\nabla \cdot \mathbf{V})_\zeta \right]_{t_1}. \quad (3.33)$$

Derivation of the Acoustic Step Momentum Equations

These acoustic-step equations are readily obtained (for the most part) by substituting the perturbation form of the variables (3.23) into (3.15)-(3.18) and collecting all of the terms with double-prime variables on the left hand side of the equations (3.25)-(3.28) and all of the terms at time t on the right hand side (3.30)-(3.33). In these equations we have recast the horizontal and vertical pressure gradients in terms of the prognostic variables in order to gain computational efficiency in the acoustic integration where we treat terms responsible for the vertical propagation of acoustic modes in an implicit manner. These acoustic equations are constructed beginning from the RK3 equations where the pressure gradients in the momentum equations

(3.15) and (3.16) are modified such that they are cast in terms of the prognostic variable Θ_m using the equation of state (3.11);

$$\nabla p = \gamma R_d \pi \nabla \Theta_m, \quad (3.34)$$

where $\pi = (p/p_o)^{R_d/c_p}$ is the Exner function and $\gamma = c_p/c_v$ is the ratio of the specific heats. Using this representation, the perturbation form of vertical pressure gradient and buoyancy terms in the vertical momentum equation (3.16) can be written as

$$\begin{aligned} \frac{\partial p'}{\partial \zeta} + g\tilde{\rho}'_m &= \gamma R_d(\bar{\pi} + \pi') \frac{\partial}{\partial \zeta} \left[\zeta_z(\bar{\Theta}_m + \Theta'_m) \right] + g(\tilde{\rho}_d + \tilde{\rho}'_m) \\ &= \gamma R_d \left[\pi \frac{\partial}{\partial \zeta} (\zeta_z \Theta'_m) + \pi' \frac{\partial}{\partial \zeta} (\zeta_z \bar{\Theta}_m) \right] + g\tilde{\rho}'_m \\ &= \gamma R_d \pi \frac{\partial}{\partial \zeta} (\zeta_z \Theta'_m) - g\tilde{\rho}_d \frac{\pi'}{\bar{\pi}} + g\tilde{\rho}'_m. \end{aligned} \quad (3.35)$$

The pressure gradients (3.34) and (3.35) correspond those used in the horizontal and vertical momentum equations (11) and (12) in Klemp et al. (2007).

Substitution of the acoustic perturbation variables (3.23) into the modified RK3 equations, with some additional approximations, leads to the acoustic equations (3.25) - (3.28). For the horizontal pressure gradient cast in terms of Θ_m (3.34), π is linearized about its value at time t in (3.25) for the integration over the acoustic steps. The representation of the vertical pressure gradient in the vertical momentum equation merits some further explanation [the terms within the large square brackets in (3.26)]. It is obtained by substituting the perturbation representation of Θ_m and π into (3.35):

$$\begin{aligned} \frac{\partial p'}{\partial \zeta} &= \gamma R_d(\pi^t + \pi'') \frac{\partial}{\partial \zeta} [\zeta_z(\Theta_m^t + \Theta_m'')] - g\tilde{\rho}_d \frac{\pi^t + \pi''}{\bar{\pi}} \\ &= \frac{\partial p^t}{\partial \zeta} + \gamma R_d \pi^t \frac{\partial}{\partial \zeta} (\zeta_z \Theta_m'') + \left[\gamma R_d \pi^t \frac{\partial}{\partial \zeta} (\zeta_z \Theta_m^t) - g\tilde{\rho}_d \frac{\pi^t}{\bar{\pi}} \right] \frac{\pi''}{\pi^t} \\ &= \frac{\partial p^t}{\partial \zeta} + \gamma R_d \pi^t \frac{\partial}{\partial \zeta} (\zeta_z \Theta_m'') + \left[\frac{\partial p^t}{\partial \zeta} + g\tilde{\rho}_d \frac{\pi^t}{\bar{\pi}} - g\tilde{\rho}_d \frac{\pi^t}{\bar{\pi}} \right] \frac{R_d}{c_v} \frac{\Theta_m''}{\Theta_m^t} \\ &= \frac{\partial p^t}{\partial \zeta} + \gamma R_d \pi^t \frac{\partial}{\partial \zeta} (\zeta_z \Theta_m'') - g\tilde{\rho}_m^t \frac{R_d}{c_v} \frac{\Theta_m''}{\Theta_m^t}. \end{aligned} \quad (3.36)$$

In deriving this expression, we have employed an approximation using a perturbation form of the equation of state $\pi''/\pi^t \simeq (R_d/c_v)(\Theta_m''/\Theta_m^t)$. Also, in simplifying the coefficient of the Θ_m''/Θ_m^t term we have used a hydrostatic approximation for the vertical pressure gradient $p_\zeta^t \simeq -g\tilde{\rho}_m^t$, as we did in (3.22). With this representation for $\partial p'/\partial \zeta$, the coefficient of the Θ_m''/Θ_m^t term in (3.26) differs slightly from that in equation (14) in Klemp et al. (2007), and represents a somewhat more consistent derivation for the double-prime perturbation variables. We also note that in their current form the coefficients of all of the double-prime terms in (3.25)–(3.28) are exactly the same as in the corresponding acoustic equations obtained directly from (3.6)–(3.9) using the full variables for p , $\tilde{\rho}_d$, and Θ_m without expressing them as perturbations from a specified reference sounding profile.

3.4.2 Acoustic Timestep

We define an acoustic timestep as advancing $\mathbf{V}_h''$, Ω'' , Θ_m'' , and $\tilde{\rho}_d''$ from time τ to time $\tau + \Delta\tau$, where $\Delta\tau$ is the acoustic timestep. The acoustic timestep begins with advancing the horizontal momentum equation (3.25) using the known values of Θ_m'' and $\tilde{\rho}_d''$ at time τ . Having the values $\mathbf{V}_h''$ at time $\tau + \Delta\tau$, and using the time-weighting operator (3.29) and the time derivative operator (3.20), we can rearrange the acoustic update equations for Ω'' , Θ_m'' and $\tilde{\rho}_d''$, (3.26), (3.27) and (3.28), as

$$\Omega''^{\tau+\Delta\tau} + \Delta\tau \frac{1+\epsilon}{2} \left\{ \zeta_z \frac{\rho_d^t}{\rho_m^t} \left[\gamma R_d \pi^t \partial_\zeta (\zeta_z \Theta_m''^{\tau+\Delta\tau}) - g \tilde{\rho}_m \frac{R_d}{c_v} \frac{\Theta_m''^{\tau+\Delta\tau}}{\Theta_m^t} \right] + \zeta_z g \tilde{\rho}_d''^{\tau+\Delta\tau} \right\} = F_{\Omega^\tau} \quad (3.37)$$

$$\Theta_m''^{\tau+\Delta\tau} = -\Delta\tau \frac{1+\epsilon}{2} \partial_\zeta (\Omega''^{\tau+\Delta\tau} \theta_m^t) + F_{\Theta_m^\tau} \quad (3.38)$$

$$\tilde{\rho}_d''^{\tau+\Delta\tau} = -\Delta\tau \frac{1+\epsilon}{2} \partial_\zeta (\Omega''^{\tau+\Delta\tau}) + F_{\tilde{\rho}^\tau} \quad (3.39)$$

where the right-hand-side terms F_{Ω^τ} , $F_{\Theta_m^\tau}$, and $F_{\tilde{\rho}^\tau}$ are known:

$$F_{\Omega^\tau} = \Omega''^\tau - \Delta\tau \frac{1-\epsilon}{2} \left\{ \zeta_z \frac{\rho_d^t}{\rho_m^t} \left[\gamma R_d \pi^t \partial_\zeta (\zeta_z \Theta_m''^\tau) - g \tilde{\rho}_m \frac{R_d}{c_v} \frac{\Theta_m''^\tau}{\Theta_m^t} \right] + \zeta_z g \tilde{\rho}_d''^\tau \right\} + \Delta\tau \left[\delta_\tau \mathbf{V}_h''^{\tau+\Delta\tau} \cdot \nabla_h \zeta + \zeta_z R_w^t \right] \quad (3.40)$$

$$F_{\Theta_m^\tau} = \Theta_m''^\tau + \Delta\tau \left\{ -\nabla_\zeta \cdot (\mathbf{V}_h''^{\tau+\Delta\tau} \theta_m^t) - \frac{1-\epsilon}{2} \partial_\zeta (\Omega''^\tau \theta_m^t) + R_{\Theta_m}^t \right\} \quad (3.41)$$

$$F_{\tilde{\rho}^\tau} = \tilde{\rho}_d''^\tau + \Delta\tau \left\{ -\nabla_\zeta \cdot \mathbf{V}_h''^{\tau+\Delta\tau} - \frac{1-\epsilon}{2} \partial_\zeta (\Omega''^\tau) + R_{\tilde{\rho}_d}^t \right\} \quad (3.42)$$

Implicit Acoustic Solver

Equations (3.37)-(3.39) form a coupled system for $(\Omega'', \Theta_m'', \rho_d'')$ at time $\tau + \Delta\tau$. The solution of this system is obtained using (3.38) and (3.39) to eliminate $\Theta_m''^{\tau+\Delta\tau}$ and $\tilde{\rho}_d''^{\tau+\Delta\tau}$ from (3.37), resulting in an equation for the single unknown variable $\Omega''^{\tau+\Delta\tau}$. The variables Θ_m'' and $\tilde{\rho}_d''$ are vertically staggered relative to Ω'' , and interpolation of these variables, along with vertical derivatives, results in an equation for $\Omega''^{\tau+\Delta\tau}$ where the new value at a level depends on the new values above and below it thus requiring us to solve a tridiagonal matrix. A description of the formulation of the tridiagonal system and solver in the acoustic step is given in Appendix A. Subsequent to the tridiagonal solution for $\Omega''^{\tau+\Delta\tau}$, vertical velocity damping is applied to damp vertically-propagating gravity waves (see section 3.4.3), after which $\Theta_m''^{\tau+\Delta\tau}$ and $\rho_d''^{\tau+\Delta\tau}$ are computed by back substitution of $\Omega''^{\tau+\Delta\tau}$ into (3.38) and (3.39).

MPAS code: The acoustic timestep is accomplished in *subroutine atm_advance_acoustic_step*. The tridiagonal solver coefficients are computed in *subroutine atm_compute_vert_imp_coefs*. The tendency for Ω involving R_w and R_{V_H} , the right-hand-side of (3.26), is computed in *subroutine atm_set_smlstep_pert_variables*, while other pieces involving multiplication by ζ_z are picked up directly in *subroutine atm_compute_dyn_tend*. All of these routines can be found in [MPAS-Model/src/core/atmosphere/dynamics/mpas_atm_time_integration.F](#).

3.4.3 Filters in the Acoustic Timestep

There are two filters that are applied on each acoustic timestep - a gravity wave filter following [Klemp et al. \(2008\)](#) that damps the vertical velocity and serves as an absorber to prevent wave reflection at the rigid MPAS model top, and an acoustic mode filter as described by ([Klemp et al., 2018](#)).

Gravity-Wave Absorbing Layer

The upper boundary of MPAS is a rigid lid ($\Omega = 0$) through which there is no flow. Vertically-propagating gravity waves can reflect off the rigid upper boundary of MPAS. In order to prevent spurious gravity-wave reflection that can contaminate the solution as the reflected waves propagate downward, MPAS employs the filter described in [Klemp et al. \(2008\)](#) where the vertically velocity is damped in an implicit manner as part of each acoustic step.

The implicit damping term is included after solving the tridiagonal equation for Ω'' (3.37). Denoting the solution to (3.37) as Ω''^* , the final solution for $\Omega''^{\tau+\Delta\tau}$ is computed by adding the implicit damping term for Ω , recognizing that the $\Omega^\tau = \Omega^t + \Omega''^\tau$:

$$\Omega''^{\tau+\Delta\tau} = \Omega''^{\tau*} - \Delta\tau R_\Omega \left[\Omega^t + \Omega''^{\tau+\Delta\tau} \right]. \quad (3.43)$$

(3.43) is equivalent to equation (9) in [Klemp et al. \(2008\)](#) except cast in terms of Ω'' instead of w . R_Ω is the damping coefficient. It typically has a value of zero (no damping) from the surface through most of the atmosphere and uses the square of a sine function to increase from 0 to a specified positive value over a user-specified range below the model top. (3.43) is trivially solved:

$$\Omega''^{\tau+\Delta\tau} = \frac{\Omega''^{\tau*} - \Delta\tau R_\Omega \Omega^t}{1 + \Delta\tau R_\Omega}. \quad (3.44)$$

The damping coefficient R_Ω is specified as

$$R_\Omega = \nu \sin^2 \left[\frac{\pi}{2} \frac{(z - z_d)}{(z_t - z_d)} \right] \quad \text{for } z_d \leq z \leq z_t, \quad \text{otherwise } z = 0,$$

where z is the height of an Ω point, z_d is the height above which the absorbing layer is active, z_t is the height of the model top, and ν is the damping rate and has units T^{-1} . After the velocity $\Omega''^{\tau+\Delta\tau}$ is computed using (3.44), $\Theta_m''^{\tau\Delta\tau}$ and $\rho_d''^{\tau\Delta\tau}$ are computed by back substitution of $\Omega''^{\tau+\Delta\tau}$ into (3.38) and (3.39).

MPAS code: The gravity wave filter application takes place in *subroutine atm_advance_acoustic_step* in *MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F*. The coefficient R_Ω is set in subroutine *atm_compute_damping_coefs* in *MPAS-Model/src/core_atmosphere/mpas_atm_core.F* and the values are stored in the array variable named *dss*. The maximum value of the damping coefficient R_Ω is given by the *namelist.atmosphere* configuration variable *config_xnutr* and the filtering begins at *namelist.atmosphere* configuration variable *height_config_zd* (m).

Acoustic Filter

The acoustic filter in MPAS follows (Klemp et al., 2018) and damps 3D divergence to filter acoustic waves. (Klemp et al., 2018) propose that the appropriate divergence to damp is $\nabla \cdot (\mathbf{V} \Theta_m)$ where $\mathbf{V} = (\mathbf{V}_H, \Omega)$. In MPAS, we use the time-rate-of-change of Θ_m'' , given in (3.27), to define the divergence which includes diabatic heating. After (3.25)-(3.28) are advanced along with the gravity-wave filter, the acoustic filter is applied as an additional update to the horizontal momentum $\mathbf{V}_H''^*$:

$$\mathbf{V}_H''^{\tau+\Delta\tau} = \mathbf{V}_H''^* + \frac{\gamma_D \Delta x}{\Theta_m^t} \delta_\tau \Theta_m'', \quad (3.45)$$

where $\delta_\tau \Theta_m'' = (\Theta_m''^{\tau+\Delta\tau} - \Theta_m''^\tau) / \Delta\tau$ as computed in the acoustic step. The damping coefficient γ_d has a default value of 0.1. See (Klemp et al., 2018) for an analysis of this approach, specifically the discussion following their equations (21)-(26).

MPAS code: The acoustic filter application takes place in *subroutine atm_divergence_damping_3d* in *MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F*. The damping coefficient is *config_smdiv* in *namelist.atmosphere*.

3.5 Scalar Transport Time Integration

The time integration of the scalar transport occurs after the dynamics steps, and the scalars are held fixed during the dynamics steps. As noted earlier, the dynamics timestep may be smaller than the scalar timestep, with the constraint that the scalar timestep be an integer multiple of the dynamics timestep. This is enforced in the specification of the dynamics timestep as being the scalar timestep divided by an integer.

Scalar transport is integrated using the RK3 time integration described in section 3.3 where the flux divergence and physics forcing terms are evaluated as the RK3 operator $L(\phi)$. Scalar transport (the flux divergence term in (3.19)) is integrated with a shape-preserving (montonic) scheme that employs limiters on the final RK3 substep to enforce shape preservation. The limiters do not take into account the forcing terms F_{Q_j} in (3.19), so we advance the solution

using the RK3 scheme as follows:

$$Q_j^* = Q_j^t + \frac{\Delta t}{3} L(Q_j^t); \quad L(Q_j^t) = [-\nabla \cdot \mathbf{V} q_j^t] + F_{Q_j}, \quad (3.46)$$

$$Q_j^{**} = Q_j^t + \frac{\Delta t}{2} L(Q_j^*); \quad L(Q_j^*) = [-\nabla \cdot \mathbf{V} q_j^*] + F_{Q_j}, \quad (3.47)$$

$$Q_j^{***} = Q_j^t + \Delta t L(Q_j^{**}); \quad L(Q_j^{**}) = F_{Q_j}, \quad (3.48)$$

$$Q_j^{t+\Delta t} = Q_j^{***} + \Delta t L(Q_j^{***}); \quad L(Q_j^{***}) = [-\nabla \cdot \mathbf{V} q_j^{***}]. \quad (3.49)$$

To accommodate the limiters, on the third RK3 substep we first update the scalars with the physics tendency, and then we apply the final transport step starting from the physics-updated values but using the Q_j^{**} values in the flux divergence. If the physics forcing $F_{Q_j} = 0$, this scheme defaults to the (3.12) - (3.14) where the final step uses an operator that includes the monotonicity constraint. The spatial discretization of the flux divergence for scalars and the monotonicity constraints are described in chapter 4.

MPAS code: *There are two routines that perform scalar advection in MPAS. Substeps (3.46) and (3.47) are performed in `subroutine atm_advance_scalars`, and substeps (3.48) and (3.49) are performed in a single call to `subroutine atm_advance_scalars_mono`. These routines are found in `MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F`.*

Chapter 4

Spatial Discretization

In chapter 3 we examined the time integration of the MPAS-Atmosphere temporally discrete prognostic equations (3.15)-(3.19) which include the shallow atmosphere approximation. We also explored the vertical discretization of the acoustic-step equations for Ω , Θ_m and $\tilde{\rho}$. In this chapter we describe the spatial discretization of the right-hand-side terms in (3.15)-(3.19).

4.1 Mass Conservation Equation

The continuous equation describing conservation of dry air mass can be expressed as

$$\frac{\partial \tilde{\rho}_d}{\partial t} = -\nabla \cdot \mathbf{V} \quad (4.1)$$

where $\mathbf{V} = (\mathbf{V}_H, \Omega) = (\tilde{\rho}_d \mathbf{v}_H, \tilde{\rho}_d \omega)$. The finite volume formulation employed in MPAS integrates (4.1) over a cell volume,

$$\int_D \left[\frac{\partial \tilde{\rho}_d}{\partial t} = -\nabla \cdot \mathbf{V} \right] dV,$$

and applies the divergence theorem to turn the volume integral into an area integral over the cell sides:

$$\frac{\partial \overline{(\tilde{\rho}_d)}}{\partial t} = -\frac{1}{V} \int_{\Sigma} \mathbf{V} \cdot \mathbf{n} \, d\sigma, \quad (4.2)$$

where the left hand-side-term is the time rate of change of the cell averaged value of $\tilde{\rho}_d$ and the right-hand-side is an integral of the integral form of the mass-flux divergence, i.e. the mass flux through the cell boundaries. $\mathbf{n}$ is the outward-pointing unit vector normal to a cell edge. Note that (4.2) is still exact.

The left-hand-side term in (4.2) is discretized by approximating the mass flux as the instantaneous value of the momentum normal to the cell faces (these are the MPAS prognostic variables $\mathbf{V}_H$ and Ω) multiplied by the area of the cell face divided by the cell volume. Thus the discrete form of the flux divergence for cell i, k denoting the i th horizontal cell at the k th level:

$$-\frac{1}{V_{k,i}} \left[\sum_{n_{e_i}} (A_{e_i} \mathbf{V}_{H, \mathbf{e}_i} \cdot \mathbf{n}) + A_c (\Omega_{k+1,i} - \Omega_{k,i}) \right], \quad (4.3)$$

where $V_{i,k}$ is the volume of the cell, and A_{e_i} and $\mathbf{V}_{\mathbf{H},\mathbf{e}_i,k}$ are the area of the hexagon face and normal mass flux through cell face e_i of cell i at level k , respectively. For the shallow-atmosphere approximation, (4.3) can be written as

$$-\frac{1}{A_i} \sum_{n_{e_i}} (L_{e_i} \mathbf{V}_{\mathbf{H},\mathbf{e}_i} \cdot \mathbf{n}) + \frac{\Omega_{k+1,i} - \Omega_{k,i}}{\Delta\zeta_w(k)}, \quad (4.4)$$

where L_{e_i} is the length of the horizontal cell edge e_i , and $\Delta\zeta_w$ is cell thickness at level k . This is the form used in the MPAS-A code.

MPAS code: The mass flux divergence (4.4) appears in a number of locations in MPAS-A. For example, it is used to compute $R_{\tilde{\rho}_d}^t$ in (3.33) (the array `rho_tend` in the code), and this occurs early in `subroutine atm_compute_dyn_tend` in `MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F`. The quantity $1/\Delta\zeta_w$ is stored in the array `rdzw` in the MPAS code. (4.3) and (4.4) are identical for the shallow atmosphere approximated equations on the discrete MPAS mesh.

We repeat here that the momentum $\mathbf{V} = (\mathbf{V}_{\mathbf{H}}, \Omega) = (\bar{\rho}_d^{c_e} \mathbf{v}_{\mathbf{H}}, \bar{\rho}_d^k \omega)$, where here $\bar{\rho}_d^{c_e}$ denotes the horizontal average of $\tilde{\rho}_d$ to the cell edge and $\bar{\rho}_d^k$ is interpolation of $\tilde{\rho}_d$ to the interface level k . This represents a simple 2nd order projection of $\tilde{\rho}_d$ to the cell faces, and we use it because the density is a very smoothly varying field while other scalar mass fields are often not smooth. In the following section we describe transport for cell-centered scalars other than the dry-air mass, where a higher order projection is described.

4.2 Transport for Cell-Centered Variables

For the flux divergence evaluations in the transport of scalars in (3.10), in the potential temperature equation (3.8), in the equation for W in (3.7), and for the vertical flux divergence needed to horizontal momentum in (3.6), we use a higher-order projection of the relevant variable to the cell edges to produce a more-accurate approximation to the flux divergence.

The transport equation for a scalar ϕ_i in cell i ,

$$\frac{\partial(\tilde{\rho}_d \phi)}{\partial t} = -\nabla \cdot \mathbf{V} \phi, \quad (4.5)$$

is discretized using the same finite-volume approach used for the dry-air conservation equation (4.1). The discrete flux divergence becomes

$$-\nabla \cdot \mathbf{V} \phi \approx -\frac{1}{A_i} \sum_{n_{e_i}} [L_{e_i} (\mathbf{V}_{\mathbf{H},\mathbf{e}_i} \cdot \mathbf{n} \phi_{e_i})] + \frac{\Omega_{k+1,i} \bar{\phi}^{k+1,i} - \Omega_{k,i} \bar{\phi}^{k,i}}{\Delta\zeta_w(k)}. \quad (4.6)$$

In the following subsections we describe the horizontal flux divergence operators and the vertical flux divergence operator, i.e. we describe how the cell-edge projections of ϕ_{e_i} and $\bar{\phi}^k$ are computed.

4.2.1 Horizontal Flux Divergences on the Voronoi Mesh

The evaluation of horizontal flux divergence on the Voronoi mesh and its performance in the RK3 scheme for transport tests are described in [Skamarock and Gassmann \(2011\)](#). This scheme generalizes the flux formula used in [Wicker and Skamarock \(2002\)](#) and first presented in [Hunds-dorfer et al. \(1995\)](#). The mass flux per unit area through cell face e_i is given by

$$(\mathbf{V}_{\mathbf{H},\mathbf{e}_i} \cdot \mathbf{n} \phi_{e_i}) = (\mathbf{V}_{\mathbf{H},\mathbf{e}_i} \cdot \mathbf{n}) \left[\frac{1}{2}(\phi_{c_0} + \phi_{c_i}) - \Delta x_{e_i}^2 \frac{1}{12} \left\{ \left(\frac{\partial^2 \phi}{\partial x_{e_i}^2} \right) \Big|_{c_i} + \left(\frac{\partial^2 \phi}{\partial x_{e_i}^2} \right) \Big|_{c_0} \right\} \right. \\ \left. + \text{sign}(\mathbf{V}_{\mathbf{H},\mathbf{e}_i} \cdot \mathbf{n}) \Delta x_{e_i}^2 \frac{\beta}{12} \left\{ \left(\frac{\partial^2 \phi}{\partial x_{e_i}^2} \right) \Big|_{c_i} - \left(\frac{\partial^2 \phi}{\partial x_{e_i}^2} \right) \Big|_{c_0} \right\} \right], \quad (4.7)$$

where a positive flux velocity flows from cell c_0 to cell c_i through edge e_i , Δx_e is the distance between the cell centers of cells c_0 and c_i , and the derivative ∂x_e is the horizontal derivative in the $\overrightarrow{c_0 c_i}$ direction. In (4.7), the values at the cell centers ϕ_{c_0} and ϕ_{c_i} are known, but their second derivatives at the cell centers in the direction $\overrightarrow{c_0 c_i}$ are not known. β is a coefficient scaling the magnitude of the upwinding in the scheme; $\beta = 0$ results in a 4th-order scheme and $\beta = 1$ is the third-order scheme from [Hunds-dorfer et al. \(1995\)](#). As described in [Skamarock and Gassmann \(2011\)](#), we compute a least-squares fit polynomial for ϕ using values at the cell center and at the neighboring cell for each of the two cells sharing the cell edge e_i , and we compute the second derivative of the polynomials in the direction $\overrightarrow{c_0 c_i}$ and use this in (4.7). The least-squares polynomial computation is described in Appendix B.

Horizontal Transport Coefficients

As noted in Appendix B, the least-squares-polynomial computation (B.2) requires a matrix inversion, but given that the matrix depends only on the geometry of the Voronoi mesh it is computed and stored as part of the initialization process by the MPAS initialization core *init_atmosphere*. The cells that contribute to the projection of a mixing ratio for flux on a cell edge requires values from the two cells sharing the edge and all of the neighbors of these two cells (see figure B.1 in Appendix B). To streamline the transport calculation in the MPAS integration, we assemble all the coefficients needed to compute the flux (4.7) as part of the start-up of the MPAS model integration such that the flux computation is a vector-vector multiply given the mixing ratios and previously assembled weights.

MPAS code: The coefficients used in the transport routines are precomputed (and stored) in *subroutine atm_adv_coef_compression* found in

MPAS-Model/src/core_atmosphere/mpas_atm_core.F

For a given edge, a list of cells needed in the flux computation is constructed and stored. For example, the list would consist of cells c_0 through c_9 for edge e_1 in figure B.1. Two set of coefficients are computed and stored. The set corresponding to the 4th-order flux for edge e_1

$$L_{e_1} \left[\frac{1}{2}(\phi_{c_0} + \phi_{c_i}) - \Delta x_{e_i}^2 \frac{1}{12} \left\{ \left(\frac{\partial^2 \phi}{\partial x_{e_1}^2} \right) \Big|_{c_1} + \left(\frac{\partial^2 \phi}{\partial x_{e_1}^2} \right) \Big|_{c_0} \right\} \right]$$

are stored in the array *adv_coef(source cells, edge)*. The set corresponding to the 3rd-order upwind piece,

$$L_{e_1} \Delta x_{e_i}^2 \frac{\beta}{12} \left\{ \left(\frac{\partial^2 \phi}{\partial x_{e_1}^2} \right) \Big|_{c_1} - \left(\frac{\partial^2 \phi}{\partial x_{e_1}^2} \right) \Big|_{c_0} \right\}, \quad (4.8)$$

are stored in the *adv_coef_3rd(source cells, edge)*. The source cells for each edge flux computation are stored in the array *advCellsForEdge(cell,iEdge)*. In the MPAS code, the coefficient β in (4.8) is included in a subsequent call to *subroutine atm_couple_coef_3rd_order*.

4.2.2 Vertical Fluxes

The vertical transport in (4.6) requires the computation of the vertical fluxes $\Omega\phi$ located at the (w, Ω) points of the MPAS mesh. We use a similar operator as used for the horizontal fluxes except that we make use of the structured vertical coordinate. The vertical flux at level k in (4.6) is

$$\Omega_{k,i} \bar{\phi}^{k,i} = \Omega_{k,i} \left[\frac{1}{2}(\phi_{k,i} + \phi_{k-1,i}) - \frac{1}{12}(\delta_\zeta^2 \phi_{k,i} + \delta_\zeta^2 \phi_{k-1,i}) + \text{sign}(\Omega) \frac{\beta}{12}(\delta_\zeta^2 \phi_{k,i} - \delta_\zeta^2 \phi_{k-1,i}) \right],$$

where $\delta_\zeta^2 \phi_{k,i} = \phi_{k+1,i} - 2\phi_{k,i} + \phi_{k-1,i}$. In the MPAS code the flux is equivalently written as

$$(\Omega\phi)_{k,i} = \Omega_{k,i} \left\{ \frac{1}{12} \left[7(\phi_{k,1} + \phi_{k-1,i}) - (\phi_{k+1,i} + \phi_{k-2,i}) \right] + \text{sign}(\Omega_{k,i}) \frac{\beta}{12} \left[(\phi_{k+1,i} - \phi_{k-2,i}) - 3(\phi_{k,i} - \phi_{k-1,i}) \right] \right\}. \quad (4.9)$$

As with the horizontal fluxes, the parameter β controls the level of upwinding, and hence damping, from this component of the transport. This form of the flux divergence originally appear in (Hundsdofer et al., 1995) and was first implemented in the RK3 transport solver in Wicker and Skamarock (2002). It is also used in the WRF model (Skamarock et al., 2021b). In the referenced descriptions, the coefficient $\beta = 1$. For $\beta = 0$ the flux divergence is 4th-order accurate and neutral in the integration. $\beta > 0$ introduces damping in the flux divergence. In MPAS $\beta = 0.25$ is the default; see (Skamarock and Gassmann, 2011) for details.

MPAS code: The vertical flux divergence calculations for scalars occur in the scalar transport routines `atm_advance_scalars` and `atm_advance_scalars_mono` found in `MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F`. Given the horizontal and vertical flux formulae described in sections 4.2.1 and 4.2.2, the updates in the first two substeps of the RK3 integration, (3.46) and (3.47), can be computed, and the fluxes are also needed for the final monotonic update. The vertical flux divergence for potential temperature (the remaining cell-centered prognostic variable) is computed in subroutine `atm_compute_dyn_tend`. The quantity $1/\Delta\zeta_w$ in the flux divergence (4.6) is stored in the array `rdnw` which is computed in the initialization routines for various cases in `MPAS-Model/src/core_init_atmosphere/mpas_init_atm_cases.F`.

4.2.3 Boundary Conditions for the Vertical Fluxes

The vertical flux divergence employs a no-flux condition ($\Omega = 0$) at the surface and at the top of the domain. Referring to the vertical mesh in figure 2.7, the zero flux condition at the lower boundary is at the $k = 1$ level for Ω . To evaluate the flux divergence for all the layer variables (u, θ_m, q), vertical fluxes are needed at the interfaces except for the lower and upper boundaries. To compute a vertical flux at interface k using (4.9), scalar values at layers $k + 1$, k , $k - 1$, and $k - 2$ are needed. i.e. from two adjacent layers on either side of the interface where the flux is computed. For the case of the interfaces closest to the lower boundary (at $k = 2$) and interface closest to the upper boundary (the MPAS-A model top), there are not sufficient layer values to fill out (4.9). In this case the value of the fluxed quantity is interpolated to the interface from the two layers. The interpolation is either linear in the computation coordinate ζ or it is the average value of the integral of the quantity between the layers as described in section 2.2.3

4.2.4 Positive-Definite and Monotonic Flux Renormalization

MPAS uses a shape-preserving (monotonic) flux renormalization adapted from Zalesak (1979). In the time integration scheme (3.46) - (3.49), the flux renormalization is applied on the final update (3.49). The update, including the flux renormalization, is accomplished as follows. Beginning with the intermediate solution that has been updated with the physics tendencies, (3.48), $q_j^{***} = Q_j^{***}/\tilde{\rho}_d^t$.

- (1) All the fluxes needed to update the cell values, i.e. the fluxes for both the horizontal faces and vertical faces, are computed and saved.
- (2) For each cell, possible cell maximum and minimum values are determined by finding the maximum and minimum values of the scalar ϕ_j^{***} from the cell center and all neighbor cells (cells that share an edge).
- (3) First-order upwind fluxes are used to update the solution. The upwind update is

$$\tilde{\rho}_d^{t+\Delta t} \phi_j^{t*} = \tilde{\rho}_d^t \phi_j^{***} - \Delta t \left[\frac{1}{A_i} \sum_{n_{e_i}} L_{e_i} \mathbf{V}_{\mathbf{H}, \mathbf{e}_i} \cdot \mathbf{n} \phi_{upwind}^{***} + \frac{1}{\Delta\zeta_k} (\Omega_{k+1,i} \phi_{upwind}^{***} - \Omega_{k,i} \phi_{upwind}^{***}) \right] \quad (4.10)$$

(4) Perturbation fluxes are formed by subtracting the upwind fluxes from the full fluxes computed in step (1). For the upwind fluxes, ϕ_{upwind} is the value from the cell upwind of the velocity on the cell face.

(5) A test update is computed using the outgoing perturbation fluxes for each cell. If the value produced is less than the minimum cell value obtained in step (2) a scalar factor ($scale_out(cell)$) is computed that multiplies the fluxes such that the minimum value is obtained. The $scale_out$ factor initially set to one before the scaling check, and it is bounded - $0 \leq scale_out \leq 1$.

(6) A test update is computed using the incoming perturbation fluxes for each cell. If the value produced is greater than the maximum cell value obtained in step (2), a scalar factor ($scale_in(cell)$) is computed that multiplies the fluxes such that the maximum value is obtained.

(7) The perturbation fluxes on an edge are scaled (renormalized) with the minimum value of the $scale_out$ from the upwind cell and $scale_in$ from the downwind cell.

(8) To complete the time step, the values are updated, starting from upwind-updated values ϕ_j^{t*} from (4.10), with the scaled perturbation fluxes.

MPAS code: *The shape-preserving (monotonic) transport scheme is contained in subroutine `atm_advance_scalars_mono` found in `MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F`. This routine is called to execute updates (3.48) and (3.49) in the described in section 3.5. The first action in this subroutine is to compute the update (3.48) by adding the physics tendencies to the scalars to produce the intermediate values q_j^{***} . The code then follows steps 1 through 8 which is the update (3.49) with the flux renormalization. The main loops in the code often include calculations from multiple steps. In contrast to the scalar updates in the first two substeps, (3.46) and (3.47), here each scalar is updated individually given the need to store the fluxes and the scaling factors.*

4.3 Horizontal Momentum Equation

The horizontal momentum equation (3.6) in MPAS is cast in vector-invariant form. In the derivation of this vector invariant form of the equation, beginning with the advective form of the horizontal momentum equation [using u_t as opposed to $(\rho u)_t$], the horizontal transport of the horizontal velocity, $(-\mathbf{v}_H \cdot \nabla_H \mathbf{v}_H)$, is recast as $-\eta \mathbf{k} \times \mathbf{v}_H - \nabla_\zeta K$. Coupling the horizontal velocity with the dry-air density brings in the additional term $\mathbf{v}_H \nabla_\zeta \cdot \mathbf{V}$. Also note that the vector-invariant form does not include the vertical transport term which still appears in (3.6).

MPAS uses a C-grid staggering of the horizontal velocity, thus the prognostic velocity is defined in the cell edge and is normal to the cell edge as illustrated in figure 2.1. In the follow sections we review the spatial discretization of the terms in the various forms of the horizontal momentum equation used in MPAS-A.

4.3.1 Pressure and Kinetic Energy

There are two places where the pressure gradient term in the horizontal momentum equation needs to be evaluated, first in the RK3 integration using (3.15), and also in the acoustic in-

tegration using (3.25). The kinetic energy gradient is only needed for the RK3 in integration (3.15). Consider the velocity u_{13} on edge ab given in figure 2.1. The line connecting cell centers A and C is perpendicular to the cell edge and is bisected by the cell edge, thus the difference of the cell center values divided by the distance between the cell centers will give a second-order accurate representation of the gradient at the u_{13} edge point. With this gradient formula, and with pressure Θ_m , ρ , and the diagnostic quantities K (horizontal kinetic energy), pressure, and exner function defined at the cell centers, we can write the discrete terms as

spatially continuous

spatially discrete

$$\frac{\rho_d}{\rho_m} \left(\nabla_\zeta \left(\frac{p'}{\zeta} \right) + g z_H \tilde{\rho}'_m \right) \rightarrow \left(\frac{\rho_d}{\rho_m} \right) \Big|_{c_e} \left[\delta_{c_e} \left(\frac{p'}{\delta_z(\zeta)} \right) + g (\delta_{c_e} z) \overline{\tilde{\rho}'_m}^{c_e} \right], \quad (4.11)$$

$$\tilde{\rho}_d \nabla_\zeta K \rightarrow \overline{\tilde{\rho}_d}^{c_e} \delta_{c_e} K, \quad (4.12)$$

$$\frac{\rho_d^t}{\rho_m^t} [\gamma R_d \pi^t \nabla_\zeta \Theta_m''^\tau + g z_H \tilde{\rho}_d''^\tau] \rightarrow \left(\frac{\rho_d}{\rho_m} \right) \Big|_{c_e} [\gamma R_d \overline{\pi^t}^{c_e} \delta_{c_e} (\Theta_m''^\tau) + g (\delta_{c_e} z) \overline{\tilde{\rho}_d''^\tau}^{c_e}]. \quad (4.13)$$

(4.11) and (4.12) are found in the RK3 integration (3.15), and (4.13) is found in the acoustic integration (3.25). The gradient operator δ_{c_e} is defined as the difference across the edge e of the cell-centered values from the two cells sharing the edge. Referring to figure 2.1 and velocity u_{13} , where a positive value of u_{13} indicates flow from cell C to cell A , the gradient operator is

$$\delta_{c_e}(\phi) = \frac{\phi_A - \phi_C}{|\overrightarrow{CA}|}, \quad (4.14)$$

where $|\overrightarrow{CA}|$ is the distance between the cell centers (the great-circle arc distance on the sphere). The averaging operator $\overline{\phi}^{c_e}$ as the arithmetic average of the cell centers sharing the edge e :

$$\overline{\phi}^{c_e} = \frac{\phi_A + \phi_C}{2}. \quad (4.15)$$

The density ratio is computed at the edge in MPAS as follows:

$$\left(\frac{\rho_d}{\rho_m} \right) \Big|_{c_e} = \left(\frac{1}{1 + \sum q_j} \right) \Big|_{c_e} \rightarrow \left(\frac{1}{1 + \sum q_j^{c_e}} \right). \quad (4.16)$$

MPAS code: The horizontal pressure gradient and kinetic energy operators (4.11) and (4.12) for the RK3 timestep can be found in *subroutine atm_compute_dyn_tend* in *MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F*. The acoustic step horizontal pressure gradient operator (4.13) can be found in *subroutine atm_advance_acoustic_step* in the same source file. The coefficients ρ_d/ρ_m for the cell edges (4.16) are computed in *subroutine atm_compute_moist_coefficients* and the array storing the coefficients is *cqu(levels, edge)*. The cell order in the gradient operator is given by the integer array *cellsOnEdge(2, edge)*, where the positive flow direction across an edge is always from *cellsOnEdge(1, edge)* to *cellsOnEdge(2, edge)* by convention.

Cell-Centered Kinetic Energy Evaluation

The discrete kinetic gradient in (4.12) uses a kinetic energies defined at the cell centers. Ringler et al. (2010) showed that for the shallow water equations on the sphere using an SCVT, an energy conserving form of the cell-center KE used with the vector invariant equations is

$$KE_{ce} = \frac{1}{A_c} \sum_{n_{e_i}} \frac{u_e^2 l_e d_e}{4}, \quad (4.17)$$

where l_e is the length of edge e , d_e is the length between the cell centers sharing edge e , and A_c is the area of cell c .

Testing during the development of MPAS-A revealed that noise would develop in 3D atmospheric solutions in idealized baroclinic wave simulations. Skamarock et al. (2012) demonstrate that an augmented form of (4.17) would remove this noise. This formulation uses a definition of the kinetic energy at the vertices of a cell

$$KE_v = \frac{1}{A_v} \sum_{n_{e_v}} \frac{u_e^2 l_e d_e}{4}, \quad (4.18)$$

where A_v is the area of the dual-mesh triangle containing the vertex, and the summation is over the three edges n_{e_v} meeting at the vertex. A weighted sum of the KE_v is next used to compute a cell-center kinetic energy:

$$KE_{cv} = \frac{1}{A_{c_v}} \sum_{n_{v_c}} A_{c_v} KE_v \quad (4.19)$$

Using this definition the kinetic energy at the cell center KE_c is computed as the weighted sum of (4.17) and (4.19):

$$KE_c = \alpha KE_{ce} + (1 - \alpha) KE_{cv}. \quad (4.20)$$

MPAS-A uses a value $\alpha = 0.375$ in its default configuration.

MPAS code: The kinetic energy is computed each RK3 substep in *subroutine atm_compute_solve_diagnostics*. The coefficient α in (4.20) is set in the code before (4.20) is evaluated. The subroutine is found in *MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F*.

4.3.2 Nonlinear Coriolis Term

The nonlinear Coriolis term in (3.6), $-\eta \mathbf{k} \times \mathbf{v}_H$, requires the reconstruction of the velocity tangent to the cell edge at the points where the prognostic normal component of velocity is defined. Additionally, the full vertical vorticity η also must be computed. The reconstruction of the tangential velocity at a edge is described in Thuburn et al. (2009), and the reconstruction represented a solution to a longstanding problem centered on using a C-grid staggering of the velocities on icosahedral meshes. Using this reconstruction, Ringler et al. (2010) derives fully nonlinear forms of the nonlinear Coriolis term for a shallow water equations discretization possessing various conservation properties. In the following subsections we describe the Thuburn et al. reconstruction, the full vertical vorticity calculation, and the construction of the nonlinear Coriolis term that we have adopted from Ringler et al.

Tangential Velocity Reconstruction

Consider the mesh depicted in figure 2.1. The edge tangent velocities $v_{13} = u_{13}^\perp$, v_{14} and v_{15} are the normal velocities on the dual-mesh triangular cell (dashed lines) with vertices A , B , and C . The Thuburn et al. (2009) reconstruction is designed so that the divergence on a dual mesh triangular cell is equal to the area-weighted sum of the divergences from the three CVT (primary mesh) cells sharing the vertex or triangular cell. Thuburn et al. determined that the reconstruction of the tangential velocity at an edge satisfying this constraint could be constructed from the weighted sum of the edge-normal velocities from the cells sharing the edge. Returning to figure 2.1, the tangential velocity v_{13} , for example, is composed of the weighted sum of velocities on the edges of cells A and C :

$$v_{13} = (w_{1,13} u_1 + w_{2,13} u_2 + w_{3,13} u_3 + w_{4,13} u_4 + w_{14,13} u_{14}) + (w_{15,13} u_{15} + w_{9,13} u_9 + w_{10,13} u_{10} + w_{11,13} u_{11} + w_{12,13} u_{12}). \quad (4.21)$$

Note that the normal velocity at the edge, u_{13} , does not contribute to the tangential velocity given that it is perpendicular to the edge. The first term in parentheses on the right-hand-side of (4.21) are the contributions from the edges of cell A and the second are from cell C .

The general formula for the weights $w_{e,e'}$, where e is an edge on the cells sharing the edge e' where the tangential velocity is reconstructed, is given in Thuburn et al. equation (33):

$$w_{e,e'} t_{e,v} = \left(\sum_v R_{i,v} - 1/2 \right) n_{e,i}. \quad (4.22)$$

This formula is for tangential velocities defined by the right-hand-rule where $\mathbf{k} \times \mathbf{n}_e$ gives the direction for the positive tangential velocity ($\mathbf{n}_e$ is the positive unit vector for the edge normal velocity). In (4.22) the indicator function $n_{e,i}$ is equal to 1 if the edge normal unit vector (direction of positive flow) points outward from cell i for edge e , and it is equal to -1 if the normal points inward. The indicator function $t_{e,v}$ is equal to 1 when the vertex v is on the left end of edge e and is equal to -1 when it is on the right edge. The quantities $R_{i,v}$ in the summations are the area of each *kite* associated with vertex v in the cell i divided by the area of the cell i . Consider cell C in figure 2.1. The kite areas associated with a vertex for a given cell is the area of the dual-mesh triangle centered at that vertex that is contained in a given cell. Thus $R_{C,a} = A_{C,a}/A_C$, $R_{C,j} = A_{C,j}/A_C$, and likewise for the remaining vertices k , l , m , and b for cell C . The summation of $R_{i,v}$ over all the vertices for a given cell will equal 1. In (4.22), the summation for a given edge e is over the vertices traversed to reach the edge e starting from edge e' where the tangential velocity is located. The direction traversing the cell from e' to e can be clockwise or counterclockwise - both will give the same result.

The reader is advised to consult Thuburn et al. (2009) for a detailed description of the derivation of (4.22) and the constraints underlying it.

MPAS code: The weights $w_{e,e'}$ are computed as one of the final steps in the MPAS mesh generation; the evaluation of (4.22) is not part of the MPAS release. We are considering putting the calculation into the release in the future. The reconstruction of the tangential velocity, e.g. (4.21), takes place in *subroutine atm_compute_solve_diagnostics* found in *MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F*. The weights $w_{e,e'}$ are stored in the array *weightsOnEdge(e,e')*, the source edges e are stored in the integer array *edgesOnEdge(e,e')* and the number of edges contributing to each reconstruction is stored in the array *nEdgesOnEdge(e')*.

Vertical Vorticity

The discrete vertical vorticity is evaluated at the vertices of the MPAS CVT mesh. Referring to figure 2.1, the vertical vorticity at vertex a is computed using the circulation theorem applied to the MPAS dual triangular mesh. The relative vertical vorticity at vertex a is computed as

$$\zeta_a = \frac{u_{13} |\overrightarrow{CA}| t_{13,a} + u_{14} |\overrightarrow{AB}| t_{14,a} + u_{15} |\overrightarrow{BC}| t_{15,a}}{A_a}, \quad (4.23)$$

where A_a is the area of the triangle centered at vertex a . The indicator function $t_{e,v}$ corrects for the direction of a positive velocity from the first cell center to the second in the circulation calculation. In the example (4.23), the indicator functions are all equal to 1. In contrast, the indicator functions would be equal to -1 for these same velocities contributing to the circulation about vertices b , f and j . Also note that these computations are performed on MPAS horizontal surfaces. Finally, the absolute vorticity at the vertices $\eta_a = \zeta_a + 2\Omega \sin(\phi)$, where Ω is the angular velocity of the earth's rotation and ϕ is the latitude.

MPAS code: The vertical vorticity is computed in *subroutine atm_compute_solve_diagnostics* found in *MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F*. The relative vertical vorticity at a vertex ζ_v is stored in the array *vorticity(level,vertex)*. The indicator function $t_{e,v}$ is stored in the array *edgesOnVertex-sign(edge,vertex)*. The absolute vorticity at the vertices η_v is stored in the array *pv_vertex(level,vertex)*. The name goes back to MPAS-A's initial implementation of a shallow water model.

For the discrete nonlinear Coriolis term, the vorticity is needed on the edges where the horizontal velocities are defined. There are two options for defining this edge vorticity. The first option is to average the vorticities from the vertices at the ends of an edge. For example, for the edge containing u_{13} in figure 2.1, we would average the vorticities from the vertices a and b . The second option is to use an upwind transport estimate of the vorticity (Ringler et al., 2010). The process of producing the upwind estimate begins by defining a cell-center vorticity:

$$\eta_c = \frac{1}{A_c} \sum_v \eta_v A_{c_v}$$

The sum is over the vertices v of a cell c where A_c is the area of cell c and A_{c_v} are the kite areas in cell c associated with vertex v . The edge absolute vorticity is then updated with an advective-form upwind transport:

$$\eta_e = \overline{\eta}_v^e - \beta \Delta t [u_e \delta_{c_e} \eta_c + v_e \delta_{v_e} \eta_v], \quad (4.24)$$

where δ_{ce} is the discrete derivative across the edge between cell centers (4.14) and δ_{cv} is the discrete derivative along the edge between the cell vertices. For example, for the edge containing u_{13} in figure 2.1, the upwinding is

$$\eta_{13} = \frac{\eta_a + \eta_b}{2} - \beta \Delta t \left[u_{13} \frac{\eta_A - \eta_C}{|\vec{CA}|} + v_{13} \frac{\eta_a - \eta_b}{|\vec{ba}|} \right]. \quad (4.25)$$

This upwind estimate has a damping effect on the absolute vorticity and it was first considered by Sadourny and Basdevant (1985) and is called the Anticipated Potential Vorticity Method (AVPM). The damping is most appropriate when the flow is characterized by a downscale enstrophy cascade characteristic of large-scale atmospheric flow and ocean dynamics. See (Ringler et al., 2010) and references therein for further information. The default value for β is 0.5 in MPAS-A, although we often configure applications with $\beta = 0$ without significant impact on the solutions. It should be noted that on variable-resolution meshes some edge lengths may be very small, hence the discrete derivative along the edge between the vertices may violate stability constraints for the estimate (4.24). In these cases we set $\beta = 0$.

MPAS code: The vertical vorticity at edges are computed in *subroutine atm_compute_solve_diagnostics* found in *MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F*. The absolute vertical vorticity is stored in the array *pv_edge(level,edge)*. The upwind estimate is also computed in this routine where first *pv_cell(level,cell)* is computed and then (4.24) is evaluated.

MPAS-A Formulation for the Nonlinear Coriolis Term

(Ringler et al., 2010) derived a discretization of the nonlinear Coriolis term that conserved potential vorticity in a shallow water model using the SCVT mesh on the sphere. We have adapted the shallow-water formulation to a 3D nonhydrostatic equations used in MPAS:

$$[\eta \mathbf{k} \times \mathbf{V}_H]_{e'} = \overline{\rho_d}^{e'} \sum_e \frac{1}{2} (\eta_{e'} + \eta_e) w_{e,e'} u_{e,e'}. \quad (4.26)$$

Here the summation is over the edges that contribute to the tangential velocity reconstruction, and the absolute vorticity that multiplies each component of the reconstructed velocity is the average of the absolute vorticities at the edge contributing to the reconstruction and the edge of the reconstruction.

MPAS code: The nonlinear Coriolis term is computed in subroutine *atm_compute_dyn_tend* in *MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F*. The absolute vertical vorticity is computed in *subroutine atm_compute_solve_diagnostics* as noted earlier. The nonlinear Coriolis term is added directly to the tendency for the momentum at the edge. Note that the reconstructed tangential velocity at the edges is not directly used in (4.26).

4.3.3 Vertical Transport for the Horizontal Velocity

The vertical flux divergence for the horizontal velocity is evaluated at the edges MPAS-A mesh where the edge-normal velocities are defined. The flux divergence is written in a similar way to

that for scalars (4.6), except the vertical mass flux Ω is averaged to the cell edges, i.e.

$$(\tilde{\rho}_d u)_k^{new} = (\tilde{\rho}_d u)_k^{old} \dots - \Delta t \frac{(\overline{\Omega^e u})_{k+1} - (\overline{\Omega^e u})_k}{\Delta \zeta_w(k)}, \quad (4.27)$$

The flux is evaluated as in (4.9) with u replacing ϕ and $\overline{\Omega^e}$ replacing Ω , and of course replacing cell i with edge e . As in the scalar transport case, a nonzero positive value of β introduces damping into the scheme. The boundary conditions for the vertical flux are handled in the same manner as in the scalar transport described in section 4.2.3.

MPAS code: The vertical flux divergence for the horizontal velocity is computed in *subroutine atm_compute_dyn_tend* found in *MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F*.

4.3.4 Horizontal Mass Flux Divergence

The horizontal mass flux divergence term in the horizontal momentum equation (3.6) is discretized as

$$-\mathbf{v}_H \nabla_\zeta \cdot \mathbf{V} \rightarrow -\mathbf{v}_{He} \frac{1}{A_i} \sum_{n_{e_i}} \overline{(L_{e_i} \mathbf{V}_{H,e_i} \cdot \mathbf{n})}^{c_e}. \quad (4.28)$$

Here the horizontal mass flux divergence in the two cells sharing the edge is averaged to the edge and multiplied by the edge velocity.

MPAS code: The discrete term (4.28) is computed in *subroutine atm_compute_dyn_tend* found in *MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F*.

4.3.5 Vertical Momentum Equation

Pressure Gradient and Body Force Term

The vertical discretization of the acoustic perturbation equations is discussed in Appendix A. The RK3 integration requires evaluation of the pressure gradient, body force and transport terms, as given in (3.31), for the acoustic integration, and it's form is analogous to the perturbation terms in the acoustic-step discretization:

$$\frac{\rho_d}{\rho_m} \left(\frac{\partial p'}{\partial \zeta} + g \tilde{\rho}'_m \right) \rightarrow \overline{\left(\frac{\rho_d^t}{\rho_m^t} \right)}^{k'} \left(\frac{p(k) - p(k-1)}{\Delta \zeta(k)} + g \overline{\tilde{\rho}'_m^k} \right). \quad (4.29)$$

The operator $\overline{\phi}^k$ is an interpolation operator that maps ϕ to the interface level k using either the linear interpolation (2.9) or a cell average (2.10). The operator quantity $\overline{(\rho_d^t/\rho_m^t)}^{k'}$ is handled in an analogous way to how it is discretized in the horizontal momentum equation (4.16) except we use a vertical average of the summed moist mixing ratios:

$$\overline{\left(\frac{\rho_d}{\rho_m} \right)}^{k'} = \overline{\left(\frac{1}{1 + \sum q_j} \right)}^{k'} \rightarrow \left(\frac{1}{1 + \overline{\sum q_j^k}} \right), \quad (4.30)$$

where the interpolation of the moist variables to the interface levels uses (2.9) or (2.10).

MPAS code: The pressure gradient and body force terms (4.29) are computed in *subroutine atm_compute_dyn_tend* in

MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F. The moist coefficient (4.30) is computed in *subroutine atm_compute_moist_coefficients*.

Transport for the Vertical Velocity

The 3 dimension transport of the vertical velocity in (3.31) requires evaluating the term $(\nabla \cdot \mathbf{V}w)_\zeta$, and this is accomplished in a manner similar to the transport of scalar variables described in section 4.2. The 3D flux divergence for scalars (4.6) is modified to account for the staggered vertical velocity w and mass flux Ω :

$$-(\nabla \cdot \mathbf{V}w)_\zeta \approx -\frac{1}{A_i} \sum_{n_{e_i}} \left[L_{e_i}(\overline{\mathbf{V}_{\mathbf{H}, \mathbf{e}_i} \cdot \mathbf{n}}^k w_{e_i}) \right] + \frac{\overline{\Omega}^{k,i} \overline{w}^{k,i} - \overline{\Omega}^{k-1,i} \overline{w}^{k-1,i}}{\Delta \zeta_k}. \quad (4.31)$$

Here the terms $\overline{\Omega}^{k,i}$ and $\overline{\Omega}^{k-1,i}$ in the vertical flux divergence denote an average of Ω to the k and $k-1$ layers, i.e. the layers above and below the interface k (see figure 2.7). In this case the flux is computed as

$$\begin{aligned} \overline{\Omega}^{k,i} \overline{w}^{k,i} = \overline{\Omega}^{k,i} \left\{ \frac{1}{12} \left[7(w_{k+1,i} + w_{k,i}) - (w_{k+2,i} + w_{k-1,i}) \right] \right. \\ \left. + \text{sign}(\overline{\Omega}^{k,i}) \frac{\beta}{12} \left[(w_{k+2,i} - w_{k-1,i}) - 3(w_{k+1,i} - w_{k,i}) \right] \right\}. \end{aligned}$$

The evaluation of the horizontal portion of the flux divergence occurs in the exact same manner as that for scalars as described in section 4.2.1 except that the horizontal velocities are vertically interpolated to the interface k , as denoted by the notation $\overline{\mathbf{V}_{\mathbf{H}, \mathbf{e}_i} \cdot \mathbf{n}}^k$. The vertical interpolation uses either the linear interpolation (2.9) or the cell integrated average (2.10). The boundary conditions for the vertical flux are handled in the same manner as in the scalar transport described in section 4.2.3.

MPAS code: The horizontal and vertical transport for the vertical velocity is computed in *subroutine atm_compute_dyn_tend* in

MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F.

Chapter 5

Filters

In this chapter we describe the discretization of filters employed in MPAS. Not covered in this chapter are filters described elsewhere:

- Vertical velocity damping in the gravity-wave absorbing layer: See section 3.4.3.
- Acoustic wave filter: See section 3.4.3.
- Upwinding in the horizontal transport scheme for θ , w and scalars: See section 4.2.1.
- Upwinding in the vertical transport: Scalars and potential temperature in section 4.2.2; w in section 4.3.5; horizontal velocity $\mathbf{v}$ in section 4.3.3.

5.1 Horizontal Filters for Cell-Centered Variables

There are two horizontal filters available in MPAS for filtering cell-centered variables, a 2nd-order filter employing a horizontal Laplacian and a 4th-order filter that employs two applications of the Laplacian. Both of these filters are discretized on the ζ surfaces of the MPAS-Atmosphere mesh, and we use the term "horizontal" to describe the smoothing on these coordinate surfaces. In the current MPAS release the 2nd-order filter can be applied to the potential temperature θ and the vertical velocity w . Horizontal filters are not applied to other scalars (e.g. q_v , etc) because monotonic transport supplies any necessary filtering.

5.1.1 2nd-Order Horizontal Filter

The continuous form for the second-order horizontal filter is given by the RHS term for the update equation for a scalar ϕ :

$$\frac{\partial(\tilde{\rho}_d \phi)}{\partial t} = \dots + \nabla_\zeta \cdot \tilde{\rho}_d K \nabla_\zeta \phi, \quad (5.1)$$

where the variable K is an eddy viscosity with units $m^2 s^{-1}$. The discrete form of Laplacian in (5.1) for θ (a variable that is not vertically staggered) is

$$\nabla_\zeta \cdot \tilde{\rho}_d K \nabla_\zeta \theta \rightarrow \frac{1}{A_i} \sum_{e_i} L_{e_i} \overline{\tilde{\rho}_d K} (\mathbf{n}_{e_i} \cdot \nabla \theta). \quad (5.2)$$

The overbar terms in (5.2) represent an average of a quantity from the cell centers of the two cells sharing the edge e_i . The gradient operator $\nabla \theta$ is the gradient of the quantity θ normal to

the edge in the direction outward from the cell ($\mathbf{n}_{e_i}$ points outward away from cell i). Compared to the horizontal mass flux used for transport as given in (4.6), the mass flux $\mathbf{V}_{\mathbf{H}, \mathbf{e}_i}$ is replaced by $\tilde{\rho}_d K$ and the transported scalar θ replaced by its gradient normal to the cell edge $\mathbf{n}_{e_i} \cdot \nabla \theta$. This turbulent flux divergence is constructed such that a constant value of θ remains constant because $\nabla \phi \equiv 0$. In addition, the filters conserved the quantity $\tilde{\rho}_d \theta$ locally and hence globally.

The 2nd-order horizontal filter can also be applied to the vertical velocity w . In this case the discrete of the turbulent flux divergence (5.2) is given by

$$\nabla_\zeta \cdot \tilde{\rho}_d K \nabla_\zeta w \rightarrow \frac{1}{A_i} \sum_{e_i} L_{e_i} \overline{\tilde{\rho}_d}^{H,z} \overline{K}^{H,z} (\mathbf{n}_{e_i} \cdot \nabla w), \quad (5.3)$$

where the averaging of $\tilde{\rho}_d$ and K is over the horizontal (between the cells sharing the edge) and vertical (between the model layers sharing the interface where w is defined).

5.1.2 4th-Order Horizontal Filter

The continuous form for the fourth-order horizontal filter is given by the RHS term for the update equation for a scalar ϕ :

$$\frac{\partial(\tilde{\rho}_d \phi)}{\partial t} = \dots - \nabla \cdot (\tilde{\rho}_d \nu_4 \nabla (\nabla_\zeta \cdot \nabla_\zeta \phi)), \quad (5.4)$$

where the hyperviscosity ν_4 has units $m^4 s^{-1}$. Defining a discrete turbulent flux divergence operator $T_2(\phi)$ as

$$\nabla_\zeta \cdot \nabla_\zeta \phi \rightarrow \frac{1}{A_i} \sum_{e_i} L_{e_i} (\mathbf{n}_{e_i} \cdot \nabla \phi) = T_2(\phi), \quad (5.5)$$

we can write the discrete 4th-order filter term as

$$\nabla \cdot (\tilde{\rho}_d \nu_4 \nabla (\nabla_\zeta \cdot \nabla_\zeta \phi)) \rightarrow \frac{1}{A_i} \sum_{e_i} L_{e_i} \overline{\tilde{\rho}_d} \nu_4 (\mathbf{n}_{e_i} \cdot \nabla T_2(\phi)). \quad (5.6)$$

The hyperviscosity ν_4 is constant in the current MPAS implementation, thus there is no averaging operator applied to it. As in the second-order filter, the density at the cell edges is an average of the densities from the cell centers of the two cells sharing the edge for vertically unstaggered variables (θ). The density will also be averaged in z when the filter is applied to w .

MPAS code: *The second-order and fourth-order horizontal filtering for θ and w are computed in [subroutine atm_compute_dyn_tend](#) in [MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F](#). The computations occur only once in each dynamics timestep, during the first RK3 substep. When the 4th-order filter is applied, the turbulent flux divergences from the 2nd-order filter application (5.5) are saved and used in the second application of the turbulent flux divergence operator (5.6) needed to complete the 4th-order operator.*

5.2 Horizontal Momentum Equation: Horizontal Filters

We cannot easily cast the horizontal filters for the momentum in terms of a flux divergence. Instead, we employ the vector identity to cast the Laplacian of the horizontal velocity in terms of the gradient of divergence across the cell edge minus the gradient of the relative vertical vorticity along the edge:

$$\begin{aligned}\frac{\partial(\tilde{\rho}_d u_i)}{\partial t} &= \dots + \tilde{\rho}_d K \nabla^2 u_i \\ &= \dots + \tilde{\rho}_d K \left(\frac{\partial}{\partial x_i} \nabla_H \cdot \mathbf{v} - \frac{\partial \zeta}{\partial x_j} \right),\end{aligned}\tag{5.7}$$

where the relative vertical vorticity $\zeta = \mathbf{n} \cdot \nabla \times \mathbf{v}$ and $\mathbf{n}$ is a unit vector normal to the ζ coordinate surface.

The fourth-order filter for the horizontal velocity employs two applications of the Laplacian given on the right-hand-side of (5.7). If we define an operator for the Laplacian for the horizontal velocity as

$$L(\mathbf{v}) = \frac{\partial}{\partial x_i} \nabla_H \cdot \mathbf{v} - \frac{\partial}{\partial x_j} \mathbf{n} \cdot \nabla \times \mathbf{v}\tag{5.8}$$

we can express the filter as

$$\begin{aligned}\frac{\partial(\tilde{\rho}_d u_i)}{\partial t} &= \dots - \tilde{\rho}_d \nu_4 L^2(\mathbf{v}) \\ &= \dots - \tilde{\rho}_d \nu_4 \left(\frac{\partial}{\partial x_i} \nabla_H \cdot L(\mathbf{v}) - \frac{\partial}{\partial x_j} \mathbf{n} \cdot \nabla \times L(\mathbf{v}) \right)\end{aligned}\tag{5.9}$$

Discrete Vorticity-Divergence Form of the Laplacian

The discrete form of the velocity divergence used in (5.8) for cell i with edges e_i , edge lengths L_{e_i} , and area A_i , is computed using

$$\nabla_H \cdot \mathbf{v} \rightarrow \frac{1}{A_i} \sum_{n_{e_i}} (L_{e_i} \mathbf{v}_{H,e_i} \cdot \mathbf{n}) = D_i,\tag{5.10}$$

where L_{e_i} is the length of the edge e_i and A_i is the area of cell i . Referring to the horizontal MPAS mesh given in figure 2.1, the gradient of the divergence to update the horizontal velocity u_{13} , where positive u_{13} indicates flow from cell C to cell A , is given as

$$\frac{\partial}{\partial x_i} \nabla_H \cdot \mathbf{v} \rightarrow \frac{D_A - D_C}{|\vec{CA}|},$$

where $|\vec{CA}|$ is the distance from cell centers C to A .

The discrete form of the relative vertical vorticity used in (5.8) is described in section 4.3.2 and is computed using (4.23). The relative vertical vorticity is defined at the vertices of the MPAS horizontal mesh and it is computed by evaluating the circulation about the dual-mesh triangle containing the vertex and then dividing the circulation by the triangle area. Referring

to the horizontal MPAS mesh given in figure 2.1, the gradient of the vorticity along the edge ab for a positive horizontal velocity u_{13} indicating flow from cell C to cell A is

$$\frac{\partial \zeta}{\partial x_j} \rightarrow \frac{\zeta_a - \zeta_b}{|\vec{ab}|}, \quad (5.11)$$

where ζ_a and ζ_b are the relative vertical vorticities defined at the vertices a and b using (4.23) and $|\vec{ab}|$ the distance between the vertices a and b , i.e. L_{e_i} for the cell edge with vertex end points a and b .

Horizontal Divergence Damping

In convection-permitting applications (horizontal cell spacing of a few kilometers or less) we have found that it is advantageous to more strongly filter the horizontal divergence relative to the vertical vorticity in the vorticity-divergence form of the 4th-order filter for the horizontal momentum given in (5.9). This modification to the horizontal filtering of the horizontal momentum equation is accomplished by introducing a coefficient in the filter term, modifying (5.9):

$$\begin{aligned} \frac{\partial(\tilde{\rho}_d u_i)}{\partial t} &= \dots - \tilde{\rho}_d \nu_4 L^2(\mathbf{v}) \\ &= \dots - \tilde{\rho}_d \nu_4 \left(\beta_d \frac{\partial}{\partial x_i} \nabla_H \cdot L(\mathbf{v}) - \frac{\partial}{\partial x_j} \mathbf{n} \cdot \nabla \times L(\mathbf{v}) \right) \end{aligned} \quad (5.12)$$

MPAS code: The second-order and fourth-order horizontal filtering for u are computed in subroutine `atm_compute_dyn_tend` in `MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F`. As with θ and w filter applications, the computations occur only once in each dynamics timestep, during the first RK3 substep. When the 4th-order filter is applied, the turbulent flux divergence from the 2nd-order filter applications is saved and used in the second application of the turbulent flux divergence operator that completes the 4th-order filter evaluation. The coefficient β_d is runtime configurable and is set in the `namelist.atmosphere` as the variable `config_del4u_div_factor`.

5.3 Eddy Viscosities and Hyper-Viscosities

There are two options for specifying the 2nd-order eddy viscosities and the 4th-order hyper-viscosities. Fixed, constant values of the viscosities can be specified using the `2d_fixed` option. Fixed viscosity (with units of $m^2 s^{-1}$) and hyperviscosity (with units of $m^4 s^{-1}$) values are specified for momentum and theta filtering, and the default values are zero, so they must be set when configuring an MPAS application using the `2d_fixed` option. This option is used for idealized simulations, often when a converged solution is sought. The eddy viscosity and hyperviscosity used in the filtering of theta is divided by the Prandtl number which has a default value of 1.

The second option for filtering employs a spatially and temporally varying 2nd-order eddy viscosity that is computed using the horizontal deformation following Smagorinsky (1963).

$$K = c_s^2 l^2 \left[\left(\frac{\partial u}{\partial x} - \frac{\partial v}{\partial y} \right)^2 + \left(\frac{\partial u}{\partial y} + \frac{\partial v}{\partial x} \right)^2 \right]^{1/2}. \quad (5.13)$$

The velocities u and v are orthogonal horizontal velocities where $u \times v$ produce a upward pointing vector. c_s is the Smagorinsky coefficient where the default value in MPAS is $c_s = 0.125$. l is a horizontal length scale and it is set to the nominal cell spacing in the highest resolution region of the MPAS Voronoi mesh. This cell spacing specified as part of the MPAS mesh file description. The computation of the deformation used in the eddy viscosity calculation (5.13) is given in appendix C.

The 4th-order filter is activated automatically when the Smagorinsky filter is specified. The constant hyperviscosity is given as

$$\nu_4(m^4 s^{-1}) = c_4 l^3, \quad (5.14)$$

where c_4 is a user specified coefficient and l is the same length scale used in the Smagorinsky scheme. The default value for the coefficient $c_4 = 0.05$.

The eddy viscosities and hyper-viscosities (5.13) and (5.14) are given for the uniform MPAS meshes. For variable-resolution meshes the eddy viscosities and hyper-viscosities are scaled such that they give the same physical viscosities and hyper-viscosities as would be employed at the nominal *local cell spacing*. For the 4th-order hyper-viscosity (5.14), the length scale l is replaced by

$$l = (\rho_{mesh})^{-1/4}, \quad (5.15)$$

where ρ_{mesh} is the mesh density function used to generate the Voronoi mesh (see Section 2). This is equivalent to scaling the hyper-viscosity computed for the highest-resolution part of the mesh by a coefficient proportional to $(l/l_f)^3$, where l_f is the length scale (nominal cell spacing) on the highest resolution of the variable-resolution mesh. We use the mesh density function as opposed to some sampling of the local cell-center spacing because the Voronoi mesh cell spacing can vary locally on an unstructured mesh whereas the density function is smooth by design. This helps minimize any grid imprinting in the filter operators.

MPAS code: The coefficient c_s can be set in the *namelist.atmosphere* file with the variable *config-smagorinsky-coef*. The default value is 0.125. The variable c_4 also a *namelist.atmosphere* variable named *config-visc4-2dsmag*. It has a default value of 0.05 and units of m/s .

5.4 Offcentering in the Vertical Acoustic Integration

In section 3.4.2 describing the acoustic timestep, the evolution equations for the vertical mass flux Ω (3.37), the coupled potential temperature Θ_m (3.38), and the dry air density $\tilde{\rho}_d$ (3.39), with their corresponding right-hand-side terms (3.40), (3.41), and (3.42), represent a semi-implicit integration of vertically-propagating acoustic and gravity waves. In these equations the terms evaluated at the new time level $(\tau + \Delta\tau)$ are multiplied by the factor $(1 + \epsilon)/2$ and the time τ terms are multiplied with a factor of $(1 - \epsilon)/2$. For $\epsilon > 0$ this will damp acoustic and gravity waves, although given the small acoustic time step it is only the high frequency acoustic waves that experience significant damping.

The off-centering in the semi-implicit time step is perhaps most important for stabilizing the integration of the integration when the coordinate surfaces are sloped. As described in Ikawa (1988) and Dudhia (1995), a linear analysis shows that the stability of the scheme is achieved

when the coordinate surface slope is less than the off-centering parameter in the vertically semi-implicit solution:

$$\left. \frac{\partial z}{\partial x} \right|_{\zeta} < \epsilon,$$

The default value for $\epsilon = 0.1$ in MPAS-Atmosphere. Higher values of the off-centering parameter ϵ may be needed for higher resolution applications with significant terrain features.

MPAS code: *The value for ϵ is runtime configurable and is set in `namelist.atmosphere` through the variable `config-epszm`.*

5.5 Gravity-Wave absorbing Layer

5.5.1 Rayleigh Damping for w

Vertical velocity damping in the gravity-wave absorbing layer is described in section 3.4.3.

5.5.2 2nd-Order Horizontal Filtering

Another option for filtering vertically-propagating gravity waves is to enable a 2nd-order horizontal filter for the horizontal momentum, vertical momentum, and potential temperature, in the layers close to the model lid. The 2nd-order filter for the potential temperature and vertical velocity follows the formulation described in section 5.1.1 using equations (5.1) and (5.2), and the formulation described in section 5.2 for the horizontal momentum using the vorticity-divergence form of the Laplacian. The eddy viscosity used is constant on each coordinate surface level varying linearly from a value of zero at a user-specified level to a maximum value at the model top. When the 2D Smagorinsky filter is active, the eddy viscosity used is chosen to be the maximum of either the fixed value or the value computed in the Smagorinsky formulation.

MPAS code: *The 2nd-order horizontal filter is included as part of the Runge-Kutta time-integration tendencies computed in `subroutine atm_compute_dyn_tend` found in `MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F`. The filter is enabled by setting the `namelist.atmosphere` parameter `config-mpas-cam-coef` to a non-zero value between 0 and 1. The default value is zero (off). The 2nd-order filter is applied starting at the integer `namelist.atmosphere` variable `config-number-cam-damping-levels` number of levels from the model top. The maximum value of the eddy viscosity is $8.333 \times \Delta x \times \text{config-mpas-cam-coef}$ (m^2/s). Setting `config-mpas-cam-coef` to 1 recovers the typical maximum value used in the climate configuration, while 0.2 is the typical value used in other applications.*

5.5.3 Rayleigh Damping of the Horizontal Momentum

While not technically a gravity-wave absorbing filter, a Rayleigh damping filter is available for the horizontal momentum and is applied to a specified number of layers below the model top. For a given time step Δt the damping term has the form

$$(\tilde{\rho}_d u)^{t+\Delta t} = (\tilde{\rho}_d u)^t - \frac{\Delta t}{\Delta t_d} (\tilde{\rho}_d u)^t,$$

where Δt_d is the damping timescale. The damping varies linear between a maximum value at the model top to zero at the layer where the damping begins. The filter is used in longer integrations and helps stabilize the integration when abnormally high horizontal velocities can develop because of the artificial nature of the rigid-lid upper boundary. This filter is also used in numerous hydrostatic models using a constant pressure upper boundary condition. The default damping timescale is long (5 days), but as noted we have found this helps stabilize longer integrations.

MPAS code: *The Rayleigh damping of the horizontal momentum is included as part of the Runge-Kutta time-integration tendencies computed in `subroutine atm_compute_dyn_tend` found in `MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F`. The filter is enabled by setting the `namelist.atmosphere` variable `config_rayleigh_damp_u` to true (false is the default). The number of layers over which the damping layer extends is set in the `namelist.atmosphere` variable `config_number_rayleigh_damp_u_levels`, and the damping timescale (days) is set in the `namelist.atmosphere` variable `config_rayleigh_damp_u_timescale_days`.*

Chapter 6

Diagnostics

Description of diagnostic quantities computed by MPAS.

6.1 Zonal and Meridional Velocity

Description of radial basis function (RBF) formulation to compute the cell-centered zonal and meridional velocities.

6.2 Frontogenesis Function

A frontogenesis function is used in the Community Earth System Model (CESM) within the gravity-wave drag parameterization in the physics of CESM's Community Atmosphere Model (CAM). In continuous form, the frontogenesis function is

$$\frac{1}{2} \left| \frac{\partial \nabla \theta}{\partial t} \right| = \frac{1}{2} \left[- (u_x + v_y) |\nabla \theta|^2 \right. \\ \left. + 2 (u_y + v_x) (\theta_x \theta_y) \right. \\ \left. - (u_x - v_y) (\theta_x^2 - \theta_y^2) \right] \quad (6.1)$$

This frontogenesis function considers only the horizontal component of frontogenesis, and it is based on the formulation described in [Richter et al. \(2010\)](#). The finite-volume computation of components of the function are described in [Appendix C](#).

6.3 Other Diagnostics to Follow

Chapter 7

Regional Configuration

The capability to perform regional simulations with MPAS-Atmosphere in real-data configurations over only a portion of the earth, where lateral boundary conditions are prescribed, was introduced in the in the 8 June 2019 release of MPAS Version 7.0. The regional capability is described in [Skamarock et al. \(2018\)](#). Here we review methodology of the regional configuration from this reference, provide updates that have been introduced since the initial release, and provide specific notes to the implementation in the MPAS code.

This section begins with the definition of the regions of the horizontal MPAS-Atmosphere mesh where the lateral boundary conditions are implemented, followed by a description of where the boundary conditions are applied within the time integration.

7.1 Specified and Relaxation Boundary Regions

Regional MPAS meshes can be constructed in two ways, The first method is to cut the mesh

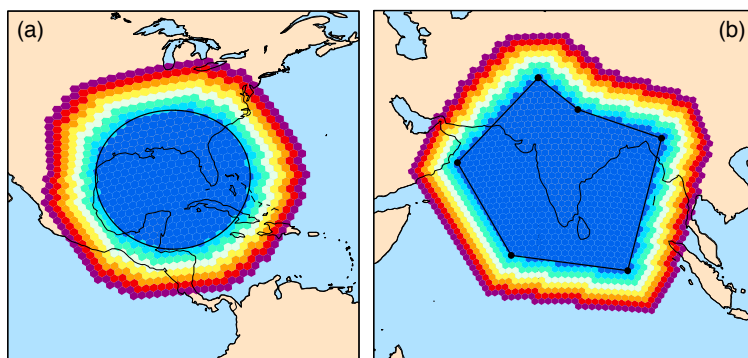


Figure 7.1: [Skamarock et al. \(2018\)](#) Figure 2 - Regional MPAS meshes extracted from a global MPAS mesh by (a) specifying a circular region and (b) specifying a polygon whose faces are great-circle arcs on the sphere. The regional mesh specifications are given by the black lines, and the blue cells fall in the mesh. The boundary-zone cells appended to the outside of the specified mesh are also shown.

from a global mesh (or another regional mesh) using tools described in the MPAS Users Guide. Examples of such meshes are given in figure 7.1 that illustrate the flexibility one has in defining

an MPAS regional mesh. The boundary zone cells that are in the relaxation region and the specified region are those that are not deep blue in color, and these cells make up the boundary region where the lateral boundary conditions are applied. The utility that extracts the regional mesh defines the interior cells (deep blue) and then builds the boundary region cells.

A closer view of the boundary region cells is given in figure 7.2. The interior cells have an index number 0 (not shown in the figure) and deep blue in color, and the boundary region cells are different colors and have indices numbered sequentially beginning from 1, starting with those adjacent to (sharing an edge with) an interior cell to those at the outermost edge of the region (having edges possessing no neighbor). Edges are also labelled where the edge index number is that of the lowest cell number of the cells sharing the edge.

In the C-grid staggering of prognostic variables in MPAS (see chapter 2 figure 2.1), the cell edge normal horizontal velocity is prognosed and all other prognostic variables are defined at cell centers. On the outer two cell and edge boundary zone index values are specified from an external analysis or forecast, usually by spatial and temporal interpolation using MPAS-Atmosphere utilities (see the MPAS Users Guide). This is called the *specified zone*. In figure 7.2 the edges and cells with indices 6 and 7 would have their values specified. The cells and edges with index values 1 through 5 are updated using the standard solution update in the time integration augmented by filtering a relaxation based on the driving analysis/forecast. Cells and edges 1 through 5 are in this *relaxation zone*.

There are two additional relaxation/filter terms applied to cells and edges in the relaxation zone. For a prognostic variable ϕ :

$$\frac{\partial \phi}{\partial t} = RHS_{\phi} + F_1(\phi_{LS} - \phi) - F_2 \Delta x^2 \nabla^2 (\phi_{LS} - \phi). \quad (7.1)$$

The variable ϕ_{LS} represents the analysis or other forecast driving the lateral boundary conditions. The first additional term on the right-hand-side of (7.1) is a Rayleigh damping term with

$$F_1 = \gamma_1(i - 1)/m,$$

where i is the cell or edge index and m is the maximum index of the relaxation zone (5 for the regional mesh shown in figure 7.2). The second term is a 2nd-order filter that is applied to the difference between the driving values and the solution in the relaxation region. The coefficient is set as

$$F_2 = \gamma_2(i - 1)/m,$$

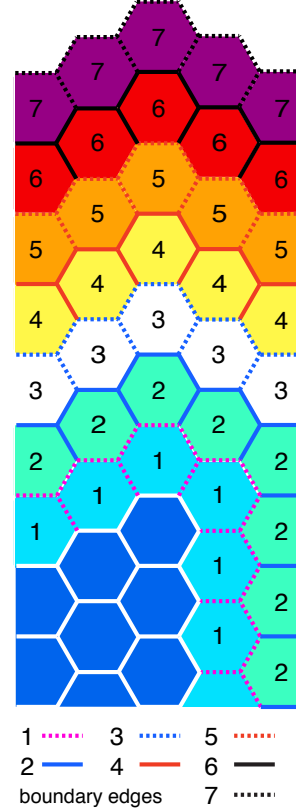


Figure 7.2: Skamarock et al. (2018) Figure 3 - Boundary zone for the MPAS horizontal mesh. The blue cells are the interior mesh, and the labeled cells and edges are in the boundary region.

The default configuration for MPAS sets $\gamma_1 = (0.06\Delta x)^{-1}$ and has units of $1/T$, and the coefficient $\gamma_2 = (0.3\Delta x)^{-1}$ and also has units of $1/T$. The coefficients γ_1 and γ_2 are set based on the local mesh size Δx so that the filters are appropriately set for a variable-resolution mesh. See Skamarock et al. (2018) section 2b for details on the configuration of these coefficients.

7.2 Time Integration

The time integration sequence in MPAS-Atmosphere is described in chapter 3 and summarized in figure 3.1. The additions to the MPAS-A solver to accommodate the regional configuration include code sections where lateral boundary conditions and relaxation zone filters are applied, along with masks and a few places where conditionals are used to change the algorithms for the lateral boundaries.

Figure 7.3 expands figure 3.1 to indicate the locations in the MPAS time integration where lateral boundary adjustments are applied. In the regional boundary zone section (1) in figure 7.3, the tendency for the specified zone increments for u , Θ_m and $\tilde{\rho}_d$ are set, and the filter terms are applied.

```
do dynamics_split_steps
    do rk3_step = 1, 3
        compute large-time-step tendency
        (1) specified zone tendency and relaxation zone
            filtering for  $u$ ,  $\theta_m$  and  $\rho_d$ .
        do acoustic_steps
            update  $u$ 
            update  $\rho$ ,  $\theta$  and  $w$ ,
                including acoustic-step filters
        end acoustic_steps
        (2) specified zone update for  $u$ ;
             $w$  set from LBC in specified zone
    end rk3_step
end dynamics_split_steps

do scalar_rk3_step = 1, 3
    scalar RK3 transport
    (3) specified zone update for scalars,
        filtering in relaxation zone
end scalar_rk3_step

microphysics
(4) specified zone update for  $\theta_m$ ,  $\Theta_m$ , and scalars
```

Figure 7.3: Pseudo code describing the locations of lateral boundary condition applications in the MPAS timestep.

MPAS code: The time integration code is found in *subroutine atm_srk3* in *MPAS-Model/src/core_atmosphere/dynamics/MPAS_atm_time_integration.F*. Boundary zone section (1) is found early in the named *RK3_DYNAMICS* loop and is in an *if (config_apply_lbc)* conditional code section. It consists of call to *subroutine atm_bdy_adjust_dynamics_speczone_tend* and *subroutine atm_bdy_adjust_dynamics_relaxzone_tend* that modify the tendencies of u and θ_m using (7.1). Tendencies and states from the driving analysis are obtained through the functions *mpas_atm_get_bdy_tend* and *mpas_atm_get_bdy_state*. These lateral boundary condition subroutines and functions are found within the time integration file.

The second boundary zone section occurs after the acoustic step within a specific Runge-Kutta substep. In the section (2) the values for u and $\tilde{\rho}u$ are reset to those of the appropriate time level (e.g. u^* , u^{**} , or $u^{t+\Delta t}$, see chapter 3 equations (3.12)-(3.14) and figure 3.3) after the full variables are reconstructed after the acoustic steps. It is also in this part of the dynamics timestep that the lateral boundary conditions for w are applied to set the vertical velocity in the specified zone. In the current release of MPAS-Atmosphere, w in the specified zone is set such that the horizontal gradient of w is zero for the flux computation across an edge between

a relaxation- and specified- zone cell. The current implementation sets the specified zone cell value of w to that of the nearest relaxation zone cell value on that level.

MPAS code: *The code for resetting the u and $\tilde{\rho}u$ values is found in the named **RK3_DYNAMICS** loop after the acoustic-step loop and after the call to subroutine **atm_recover_large_step_variables**, and is in an **if (config_apply_lbcs)** conditional code section. The w values in the specified zone are set after a call to subroutine **atm_compute_solve_diagnostics** and the values are set in a call to subroutine **atm_zero_gradient_w_bdy** in a separate **if (config_apply_lbcs)** conditional code section immediately before the end of the **RK3_DYNAMICS** loop. This subroutine is in the time integration file **MPAS-Model/src/core_atmosphere/dynamics/MPAS_atm_time_integration.F**.*

The third lateral boundary condition section is in the scalar transport Runge-Kutta loop immediately after the call to advance the scalars over a Runge-Kutta substep. Here the values in the specified zone are set and the filtering in the relaxation zone using (7.1) is applied if the values from the driving analysis exist. If the values do not exist then they are assumed to be zero and the specified zone values are set to zero with the filters applied using this value.

MPAS code: *The lateral boundary condition code for scalar transport, section (3), is in a **if (config_apply_lbcs)** conditional code section with in named loop **RK3_SPLIT_TRANSPORT**. In the current implementation individual scalars are checked to see if they exist, and if so the driving analysis state for them is acquired. The relaxation zone filtering and the specified zone values are set in a call to **atm_bdy_adjust_scalars**. All this code is in the time integration file **MPAS-Model/src/core_atmosphere/dynamics/MPAS_atm_time_integration.F**.*

The final boundary condition section in the MPAS time integration resets values for θ_m , Θ_m and the scalars after the call to microphysics. The microphysics is applied in all the columns, even in the specified zone, hence the need to reset the values. If the driving analysis does not contain values for a given scalar then that scalar is set to zero.

MPAS code: *The lateral boundary condition code for scalar transport, section (4), is in a **if (config_apply_lbcs)** conditional code section and appears immediately after the call to **microphysics_driver**. Specified zone values for θ_m and Θ_m are reset in a call to **atm_bdy_reset_speczone_values** and the scalars are reset in a call to subroutine **atm_bdy_set_scalars**. All this code, including the subroutines, is in the time integration file **MPAS-Model/src/core_atmosphere/dynamics/MPAS_atm_time_integration.F**.*

7.2.1 Divergence Damping in the Relaxation Zone

For the horizontal momentum, the 2nd-order horizontal filter applied in (7.1) in the relaxation region takes the form given in (5.7). However, we have introduced a coefficient to increase the divergent component of the filtering, similar to additional filtering applied to the horizontal filter for the 4th-order background filter given in section 5.2 equation (5.12). The resulting 2nd-order filter applied to the horizontal momentum in the relaxation zone takes the form

$$\frac{\partial(\tilde{\rho}_d u_i)}{\partial t} = \dots - \tilde{\rho} \frac{\gamma_2(i-1)}{m} \Delta x^2 \left(\beta_D \frac{\partial}{\partial x_i} \nabla_H \cdot (\mathbf{v}_{LBC} - \mathbf{v}) - \frac{\partial(\zeta_{LBC} - \zeta)}{\partial x_j} \right). \quad (7.2)$$

MPAS code: *The relaxation zone filtering for the horizontal momentum is performed in section (1) of the time integration (see figure 7.3 and the discussion earlier in this section). The coefficient β_d is configurable in `namelist.atmosphere` through the namelist variable `config_relax_zone_divdamp_coef`. Its default value is 6. The filtering is performed in subroutine `atm_bdy_adjust_dynamics_relaxzone_tend` in `MPAS-Model/src/core_atmosphere/dynamics/MPAS_atm_time_integration.F`.*

Chapter 8

Initial Conditions

The method for producing an initial state for MPAS-Atmosphere to integrate involves executing a different MPAS model called *init_atmosphere*. The practical aspects of producing an initial state are covered in the *MPAS-Atmosphere Users Guide*. In this chapter we describe the physical configurations of the different states along with some technical aspects concerning how they are produced. Here are the initial states for the cases that can be produced by the *init_atmosphere* model along with the case numbers used within the *init_atmosphere* model:

- (1, 2, 3): Three variants of the [Jablonowski and Williamson \(2006\)](#) baroclinic wave case.
- (4, 5): A 2D (x, z) squall-line and 3D supercell thunderstorm initialization in Cartesian domains.
- (6): A 2D (x, z) mountain-wave initialization.
- (7): Initialization for full earth atmosphere simulations on the sphere or regional domains on the sphere.
- (8): Produce sea-surface temperature (SST) and surface fields at specified times. These are typically used to update the SSTs and surface fields during long integrations of MPAS-A.
- (9): Produce boundary conditions for regional MPAS-A simulations on the sphere.
- (13): Produce the 3D MPAS-A mesh used in applications outside of MPAS-A integrations, e.g. in NSF NCAR's Community Earth System Model when it is employing the MPAS-A dynamical core.

MPAS code: *The numbers attached to the test cases listed above correspond to the case configuration number in the `namelist.init_atmosphere` configuration file for the `init_atmosphere` core. For example in the full earth atmosphere simulations the configuration variable `config_init_case` = 7.*

8.1 Initialization Commonalities

MPAS-Atmosphere integrates an equation set that uses perturbation variables for density and potential temperature defined relative to a reference state (see chapter 3). The reference state can be specified as a continuous function or a discrete function. The perturbation state density and potential temperature, along with a specified water vapor profile, satisfies an MPAS-A discrete hydrostatic balance.

8.1.1 Reference States

An analytic reference state is used in the [Jablonowski and Williamson \(2006\)](#) baroclinic wave case and in the initialization of the full earth atmosphere, often referred to as a *real-data* initialization. The analytic state is isothermal with a reference temperature $T_o = 250$ Kelvin. The pressure, density and potential temperature are given by

$$\begin{aligned}\bar{p} &= p_o \exp\left(-\frac{gz}{R_d T_o}\right), \\ \bar{\rho} &= \frac{\bar{p}}{R_d T_o}, \\ \bar{\theta} &= T_o \left(\frac{\bar{p}}{p_o}\right)^{-R_d/c_p},\end{aligned}\tag{8.1}$$

where $p_o = 1000$ hPa is a reference pressure.

8.1.2 Hydrostatic Balance

The hydrostatic relation cast in terms of perturbation thermodynamical variables is given in [\(3.22\)](#), and for the case where the only nonzero moist species is water vapor q_v it can be expressed as

$$\frac{\partial p'}{\partial \zeta} = -g (\tilde{\rho}'_d + \tilde{\rho}_d q_v).$$

The discrete formulation used in the initialization is that used for these terms in the MPAS-A vertical momentum equation given in [\(4.29\)](#),

$$p(k) - p(k-1) = -\Delta\zeta(k) \overline{g(\tilde{\rho}'_d + \tilde{\rho}_d q_v)}^k,\tag{8.2}$$

where k indicates the layer number, with layer height increasing with increasing k . The operator $\overline{(\cdot)}^k$ represents either a vertical interpolation to the midpoint of the interval or the average value computed from integrating the quantity over the interval, both options described in [Section 2.2.3](#) with weights defined in [\(2.9\)](#) and [\(2.10\)](#), respectively.

Another form of the hydrostatic relation used in some of the initializations in that cast in terms of the Exner function:

$$\frac{\partial \Pi}{\partial z} = -\frac{g}{c_p \theta^{\frac{1+R_d/c_p}{1+q_v}}}\tag{8.3}$$

where the Exner function is defined as

$$\Pi = \left(\frac{p}{p_o}\right)^{R_d/c_p}$$

8.2 Initialization for Idealized Cases

The idealized atmosphere cases are comprised of cases 1 through 6 as described below.

8.2.1 Idealized Baroclinic Wave Case

The idealized baroclinic wave case is taken from [Jablonowski and Williamson \(2006\)](#) and the cases 1 through 3 involve three different perturbations imposed on the zonally uniform balanced midlatitude northern and southern hemisphere jets prescribed in the initial state. The balanced flow represents a steady-state solution to the equations of motion. However, any zonally nonuniform perturbations will project onto the baroclinically unstable modes of the jets giving rise to baroclinically unstable waves. The perturbations for the three cases are as follows:

(1): No perturbation is applied. In models where the mesh is not zonally uniform (e.g. MPAS), truncation errors arising from the discretization on the horizontal mesh will give rise to baroclinic waves. See [Skamarock et al. \(2012\)](#) figure 8 for an example of an MPAS-Atmosphere result for this case.

(2): The perturbation is that given in equation (10) from [Jablonowski and Williamson \(2006\)](#) for the horizontal wind field u :

$$u'(\lambda, \phi) = u_p \exp \left\{ - \left(\frac{r}{R} \right)^2 \right\}, \quad (8.4)$$

where (λ, ϕ) are the longitude and latitude, $u_p = 1 \text{ m/s}$, r is the great-circle arc distance on the sphere from the point $(\lambda_o, \phi_o) = (20E, 40N)$, and $R = a/10$ where a is the earth radius. There is no vertical dependence to this perturbation.

See [Skamarock et al. \(2012\)](#) figure 7 for an example of an MPAS-Atmosphere result for this case.

(3): The perturbation is designed to be the starting point for growing the most unstable normal mode for the jet which is zonal wavenumber 9:

$$u'(\lambda, \phi) = u_p \cos[k_x(\lambda - \lambda_o)] \sin^2 2\phi, \quad (8.5)$$

where $k_x = 9$ is the zonal wavenumber of the perturbation. There is no vertical dependence to this perturbation. Also, there is not a published solution for this perturbation but the converged normal mode solution for wavenumber 9 solution is shown in [Park et al. \(2013\)](#).

The initialization integrates the hydrostatic balance equation (8.2) from the surface upward to the lid, with a lower boundary condition of $p = 1000 \text{ hPa}$. There is a gradual terrain slope from the equator to the pole to satisfy a zero zonal wind at the surface for the given analytic formula for the jet. This case uses the isothermal reference state (8.1).

MPAS code: The initialization code for this test case is found in *subroutine `init_atm_case_jw`* in the source code file

`MPAS-Model/src/core_init_atmosphere/mpas_init_atm_cases.F`. The wind perturbations in equations (8.4) and (8.5) are integrated between the latitudes of the edge endpoints and projected onto (dotted with) the normal vector in the zonal direction so as to get the average edge-normal wind component on the edge. Additionally, the horizontal winds of the unperturbed jet are computed so that the horizontal mass divergence in the initial state is minimized.

8.2.2 Squall Line and Supercell Case

Both the squall line and supercell thunderstorm case use the same initialization procedure. The lower boundary is a flat surface (there is no terrain) and vertical grid spacing is constant. In these initializations the model top is set to $z = 20$ km. The environment is initialized with the sounding taken from [Weisman and Klemp \(1982\)](#):

$$\theta(z) = \theta_o + (\theta_{tr} - \theta_o) \left(\frac{z}{z_{tr}} \right)^{5/4}, \quad \text{for } z \leq z_{tr} \quad (8.6)$$

$$= \theta_{tr} \exp \left[\frac{g}{c_p T_{tr}} (z - z_{tr}) \right], \quad \text{for } z > z_{tr} \quad (8.7)$$

with the relative humidity defined as

$$H(z) = 1 - \frac{3}{4} \left(\frac{z}{z_{tr}} \right)^{5/4}, \quad \text{for } z \leq z_{tr} \quad (8.8)$$

$$= 0.25, \quad \text{for } z > z_{tr} \quad (8.9)$$

where

$$\theta_o = 300 \text{ K},$$

$$\theta_{tr} = 343 \text{ K},$$

$$T_{tr} = 213 \text{ K},$$

$$z_{tr} = 12 \text{ km}.$$

For the 2D squall-line simulation and the supercell simulation the environmental wind profile is set in the initialization code. It only varies in z and only the x component of the wind field is nonzero:

$$\begin{aligned} u(z) &= u_m \frac{z}{z_{ts}} - u_s \quad \text{for } z \leq z_{ts} \\ &= u_m - u_s, \quad \text{for } z > z_{tr} \end{aligned}$$

where $u_m = 12$ m/s, $u_s = 10$ m/s and $z_{ts} = 2500$ m for the squall line case, and $u_m = 30$ m/s, $u_s = 15$ m/s and $z_{ts} = 5000$ m for the supercell case.

The reference state for these simulations is that given in (8.6) and (8.7) for a dry atmosphere ($H(z) = 0$). The hydrostatic balance for the reference state uses the hydrostatic relation cast in terms of the Exner function (8.3). This equation is also used to compute the mixing ratios based on the relative humidities (8.8) and (8.9). The hydrostatic balance for the full state uses an upward vertical integration of (8.3) to estimate the pressure at the top of the domain, using a reference pressure $p_o = 10^5$ Pa as the boundary condition at the surface, and then uses a downward integration of (8.2) that is used with the state equation to specify the density. This hydrostatic balance includes a warm low-level temperature perturbation (a warm bubble) to initiate convection. The integrations are applied iteratively to arrive at a converged solution that satisfies both the hydrostatic relation and the state equation. Given that the vapor mixing ratio q_v is computed using the Exner-function-based hydrostatic relation, the relative humidity (8.8)

and (8.9) will not be exactly satisfied in the MPAS atmosphere computed using the hydrostatic relation (8.2). We also note that an upper bound of $q_{v_{max}} = 14$ g/kg is set in this initialization.

MPAS code: *The initialization code for this test case is found in [subroutine `init\_atm\_case\_squall\_line`](#) in the source code file [MPAS-Model/src/core_init_atmosphere/mpas_init_atm_cases.F](#). The hydrostatic balance loops and iterations do not assume a flat lower boundary, thus many of the calculations are redundant, particularly for the reference state. Also note that the variable `p` refers to the Exner function in the hydrostatic balancing of the reference state and the variable `pp` refers to a perturbation pressure in the balancing of the MPAS-A perturbation state.*

8.2.3 Mountain Wave Case

As in the squall line and supercell cases, the vertical grid has a constant Δz and the model top is set to $z = 21$ km. The reference state potential temperature is $\bar{\theta}(z) = \theta_o = 288$ K. The initial profile for the potential temperature characterized by constant stability:

$$\theta(z) = \theta_o(1 + N^2 z)$$

where the stability parameter

$$N^2 = \frac{g}{\theta} \frac{\partial \theta}{\partial z}.$$

In the existing initialization code the value for this case is $N^2 = 10^{-4} s^{-2}$.

The surface terrain profile is that described in [Schär et al. \(2002\)](#) and [Klemp et al. \(2003\)](#), and is given by

$$h(x) = h_m \exp \left[- \left(\frac{x - x_c}{x_a} \right)^2 \right] \cos^2 \left[\pi \left(\frac{x - x_c}{x_a} \right) \right]$$

where $h_m = 250$ m and $x_a = 4000$ m.

The hydrostatic balance in this case uses the same procedure as used in the squall line and supercell initialization.

MPAS code: *The initialization code for this test case is found in [subroutine `init\_atm\_case\_mtn\_wave`](#) in the source code file [MPAS-Model/src/core_init_atmosphere/mpas_init_atm_cases.F](#). There are a number of options for terrain profiles and stability configurations that can be changed by editing and recompiling the `init_atmosphere` core.*

8.3 Initialization for Earth-Atmosphere Simulations

The initialization for a real-data full-earth-atmosphere case is much more complex compared to the idealized cases. In the following we describe only the hydrostatic balancing and a few other numerical aspects of the initialization. Other aspects of the initialization scheme are described in the MPAS-Atmosphere Users Guide.

8.3.1 Initialization of the 3D State

Here we only discuss the initialization of the atmospheric state on the MPAS mesh. The specification of that mesh is described in Chapter 2. The reference state used in the initialization is the analytic isothermal state given by (8.1). The hydrostatic balance uses the perturbation state balance (8.2). The surface pressure, and the potential temperature and water vapor mixing ratio in the column, are interpolated to the MPAS-Atmosphere mesh from the analysis. The lowest model level is interpolated from the analysis, and the lowest model level density is computed from the state equation. For each layer above the first layer, the hydrostatic relation (8.2) along with the state equation are evaluated iteratively until the densities and pressures satisfy both the state equation and hydrostatic relation within a certain tolerance which in the current release requires the the change in pressure in each iteration be less than 0.0001 Pa or that the iteration count is less than 30.

MPAS code: *The initialization code for this test case is found in [subroutine `init\_atm\_case\_gfs`](#) in the source code file [MPAS-Model/src/core_init_atmosphere/mpas_init_atm_cases.F](#). Note that most of the subroutine, after the 3D MPAS mesh is created, involves the reading in and interpolating the analysis to the MPAS mesh.*

8.3.2 Lateral Boundary Conditions for Regional Simulations

The lateral boundary condition state for regional simulations is computed in the same manner as the full 3D initialization, except that the 3D mesh is taken from the full state initialization.

Chapter 9

Physics

In preparation.

Chapter 10

Infrastructure

In preparation.

Appendix A

Vertically-Implicit Acoustic Solution

The solution to the coupled system of equations (3.37), (3.38) and (3.39), repeated here for convenience,

$$\Omega''^{\tau+\Delta\tau} + \Delta\tau \frac{1+\epsilon}{2} \left\{ \zeta_z \frac{\rho_d^t}{\rho_m^t} \left[\gamma R_d \pi^t \partial_\zeta (\zeta_z \Theta_m''^{\tau+\Delta\tau}) - g \tilde{\rho}_m^t \frac{R_d}{c_v} \frac{\Theta_m''^{\tau+\Delta\tau}}{\Theta_m^t} \right] + \zeta_z g \tilde{\rho}_d''^{\tau+\Delta\tau} \right\} = \Delta\tau F_{\Omega^\tau}, \quad (3.37)$$

$$\Theta_m''^{\tau+\Delta\tau} = -\Delta\tau \frac{1+\epsilon}{2} \partial_\zeta (\Omega''^{\tau+\Delta\tau} \theta_m^t) + \Delta\tau F_{\Theta_m^\tau}, \quad (3.38)$$

$$\tilde{\rho}_d''^{\tau+\Delta\tau} = -\Delta\tau \frac{1+\epsilon}{2} \partial_\zeta (\Omega''^{\tau+\Delta\tau}) + \Delta\tau F_{\tilde{\rho}^\tau}, \quad (3.39)$$

is needed to complete the acoustic timestep. Given vertical C-grid staggering, the elimination of $\Theta_m''^{\tau+\Delta\tau}$ and $\tilde{\rho}_d''^{\tau+\Delta\tau}$ from (3.37) using (3.38) and (3.39) results in a tridiagonal matrix that must be inverted. In this appendix we describe the setup of the tridiagonal coefficients that define the matrix needed to solve for $\Omega''^{\tau+\Delta\tau}$.

The C-grid vertical staggering of the MPAS mesh is depicted in chapter 2 section 2.2 figure 2.7. We denote the heights at which $\tilde{\rho}_d$, Θ_m , $\mathbf{V}_H$ and Q_j are defined as model levels, and the heights at which W is defined as layer interfaces. The MPAS vertical grid is defined such that the model levels lie exactly 1/2 the distance between the upper and lower interface. Based on this staggering and grid definition, the update equations (3.37)-(3.39) are vertically discretized:

$$\begin{aligned} \Omega''^{\tau+\Delta\tau}(k) + C_{\Omega\Theta z}(k) & \left[\zeta_z(k) \cdot \Theta_m''^{\tau+\Delta\tau}(k) - \zeta_z(k-1) \cdot \Theta_m''^{\tau+\Delta\tau}(k-1) \right] \\ & - C_{\Omega\Theta}(k) \cdot \Theta_m''^{\tau+\Delta\tau}(k) - C_{\Omega\Theta}(k-1) \cdot \Theta_m''^{\tau+\Delta\tau}(k-1) \\ & + C_{\Omega\rho}(k) \cdot \left[\tilde{\rho}_d''^{\tau+\Delta\tau}(k) + \tilde{\rho}_d''^{\tau+\Delta\tau}(k-1) \right] \\ & = \Delta\tau F_{\Omega^\tau}(k), \end{aligned} \quad (A.1)$$

$$\begin{aligned} \Theta_m''^{\tau+\Delta\tau}(k) & = -\frac{1}{\Delta\zeta_w(k)} \left[C_\Theta(k+1) \cdot \Omega''^{\tau+\Delta\tau}(k+1) - C_\Theta(k) \cdot \Omega''^{\tau+\Delta\tau}(k) \right] \\ & + \Delta\tau F_{\Theta_m^\tau}(k), \end{aligned} \quad (A.2)$$

$$\tilde{\rho}_d''^{\tau+\Delta\tau}(k) = -C_\rho(k) \left[\Omega''^{\tau+\Delta\tau}(k+1) - \Omega''^{\tau+\Delta\tau}(k) \right] + \Delta\tau F_{\tilde{\rho}^\tau}(k). \quad (A.3)$$

The levels k indicate layer interfaces in (A.1) and model levels in (A.2) and (A.3). The new coefficients given in blue are defined as

$$C_{\Omega\Theta z}(k) = \Delta\tau \frac{1+\epsilon}{2} \frac{\bar{\zeta}_z^{-k} c_p}{c_v} \left(\frac{\rho_d^t}{\rho_m^t} \right)^k \frac{1}{\Delta\zeta(k)}, \quad \text{interface}, \quad (\text{A.4})$$

$$C_{\Omega\Theta}(k) = \Delta\tau \frac{1+\epsilon}{2} \frac{\zeta_z}{2} \frac{R_d}{c_v} g \tilde{\rho}_d^t \frac{1}{\Theta_M^t}, \quad \text{level}, \quad (\text{A.5})$$

$$C_{\Omega\rho}(k) = \Delta\tau \frac{1+\epsilon}{2} \frac{\bar{\zeta}_z^{-k}}{2} g, \quad \text{interface}, \quad (\text{A.6})$$

$$C_{\Theta}(k) = \Delta\tau \frac{1+\epsilon}{2} \frac{\bar{\Theta}_m^t}{\Theta_m^t}, \quad \text{interface}, \quad (\text{A.7})$$

$$C_{\rho}(k) = \Delta\tau \frac{1+\epsilon}{2} \frac{1}{\Delta\zeta_w(k)}, \quad \text{level}, \quad (\text{A.8})$$

where we have also indicated where they are defined (levels or interfaces). The operators $\bar{\phi}^k$ interpolate values from model levels to layer interfaces, or from layer interfaces to model levels, using the two bounding levels or interfaces as appropriate. Interpolation from the layer interfaces to levels just averages the bounding values.

MPAS code: The coefficients (A.4)-(A.8) are set in subroutine `atm_compute_vert_imp_coefs` in `MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F`. In the MPAS-A code the coefficients are named as follows:

$$\begin{aligned} C_{\Omega\Theta z}(k) &- \text{cofwz} \\ C_{\Omega\Theta}(k) &- \text{cofwt} \\ C_{\Omega\rho}(k) &- \text{cofwr} \\ C_{\Theta}(k) &- \text{coftz} \\ C_{\rho}(k) &- \text{cofrz} \end{aligned}$$

These coefficients are used in subroutine `atm_compute_vert_imp_coefs` to compute more coefficients (see below) for the tridiagonal solver and are used in the acoustic timestep in subroutine `atm_advance_acoustic_step` to evaluate contributions to the the time level τ operators ($F_{\tilde{\rho}^\tau}$, F_{Ω^τ} and F_{Θ^τ}) and in the back substitution to recover Θ_m'' and $\tilde{\rho}''$ at $\tau + \Delta\tau$.

Using (A.2) and (A.3) to eliminate $\Theta_m''^{\tau+\Delta\tau}$ and $\tilde{\rho}''^{\tau+\Delta\tau}$ in (A.1) we arrive at the following

equation for $\Omega''^{\tau+\Delta\tau}$:

$$\begin{aligned}
& \Omega''^{\tau+\Delta\tau}(k-1) \cdot \left[-C_{\Omega\Theta z}(k) \cdot C_{\Theta}(k-1) \cdot \zeta_z(k-1)/\Delta\zeta(k-1) \right. \\
& \quad + C_{\Omega\rho}(k) \cdot C_{\rho}(k-1) \\
& \quad \left. - C_{\Omega\Theta}(k-1) \cdot C_{\Theta}(k-1)/\Delta\zeta(k-1) \right] \\
& + \Omega''^{\tau+\Delta\tau}(k) \cdot \left[1 + C_{\Omega\Theta z}(k) \left(C_{\Theta}(k) \cdot \zeta_z(k)/\Delta\zeta(k) + C_{\Theta}(k) \cdot \zeta_z(k-1)/\Delta\zeta(k-1) \right) \right. \\
& \quad - C_{\Theta}(k) \cdot \left(C_{\Omega\Theta}(k)/\Delta\zeta(k) - C_{\Omega\Theta}(k-1)/\Delta\zeta(k-1) \right) \\
& \quad \left. + C_{\Omega\rho}(k) \cdot \left(C_{\rho}(k) - C_{\rho}(k-1) \right) \right] \\
& + \Omega''^{\tau+\Delta\tau}(k+1) \cdot \left[-C_{\Omega\Theta z}(k) \cdot C_{\Theta}(k+1) \cdot \zeta_z(k)/\Delta\zeta(k) \right. \\
& \quad - C_{\Omega\rho}(k) \cdot C_{\rho}(k) \\
& \quad \left. + C_{\Omega\Theta}(k) \cdot C_{\Theta}(k+1)/\Delta\zeta(k) \right] = \\
& + \Delta\tau \left[F_{\Omega\tau}(k) - \left(C_{\Omega\Theta z}(k) \cdot \zeta_z(k) - C_{\Omega\Theta}(k) \right) F_{\Theta_m\tau}(k) \right. \\
& \quad + \left(C_{\Omega\Theta z}(k) \cdot \zeta_z(k-1) + C_{\Omega\Theta}(k-1) \right) F_{\Theta_m\tau}(k-1) \\
& \quad \left. - C_{\Omega\rho}(k) \left(F_{\rho\tau}(k) + F_{\rho\tau}(k-1) \right) \right]. \tag{A.9}
\end{aligned}$$

This equation can also be written as

$$C_- \cdot \Omega''^{\tau+\Delta\tau}(k-1) + C \cdot \Omega''^{\tau+\Delta\tau}(k) + C_+ \cdot \Omega''^{\tau+\Delta\tau}(k+1) = R, \tag{A.10}$$

where the three coefficients are the three bracketed terms in (A.9). This equation is a tridiagonal system with the diagonal coefficient C and the lower and upper diagonal coefficients C_- and C_+ . In MPAS this system is solved with a standard tridiagonal solver with the boundary condition $\Omega = 0$ at the upper and lower boundaries.

MPAS code: The coefficients for the tridiagonal solver, C , C_- and C_+ , are computed in subroutine `atm_compute_vert_imp_coefs` in

[MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F](#).

In the code, C_- is stored in the array `a_tri`, C is stored in the array `b_tri`, and C_+ is stored in the array `c_tri`. The coefficients are used to precompute other coefficients (`alpha_tri` and `gamma_tri`) that are used along with `a_tri` to perform the implicit solve in subroutine `atm_advance_acoustic_step`.

Appendix B

Least-Squares Polynomial Computation

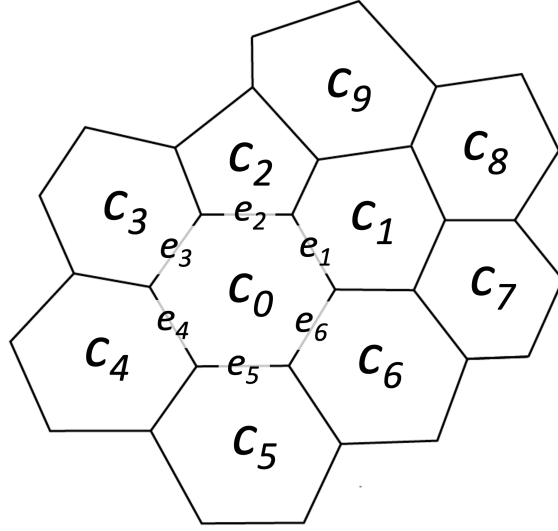


Figure B.1: A limited region in an MPAS horizontal mesh. Adapted from [Skamarock and Gassmann \(2011\)](#) Figure 1.

Consider the Voronoi cells depicted in figure [B.1](#). To compute the flux edge e_1 we fit polynomials to cells c_1 and its neighbor cells, i.e. $(c_1, c_0, c_6, c_7, c_8, c_9, c_2)$, and also to c_0 and its neighbors, i.e. $(c_0, c_1, c_2, c_3, c_4, c_5, c_6)$. More generally, a polynomial fit is needed for each cell, and its 2nd derivative normal to each edge needs to be computed.

To illustrate how the polynomials are computed, we examine the polynomial computation for the cell c_0 . We begin by constructing a planar projection of the cell values on the sphere using a tangent plane that is coincident with the sphere at the cell center point c_0 . We then define a 2 dimensional (x, y) coordinate system on the plane such that the x -axis extends through the first cell on the cell neighbor list for c_0 (see figure [B.2](#)) and define the y -axis by the right-hand rule.

Following [Skamarock and Gassmann \(2011\)](#), the tangent plane is defined such that the angles θ_i are the spherical angles at c_0 between the vectors $\overrightarrow{c_0 c_1}$ and $\overrightarrow{c_0 c_i}$. The length $|\overrightarrow{c_0 c_i}|$ is the great circle arc distance on the sphere between c_0 and c_i . We fit a least-squares-fit polynomial for a

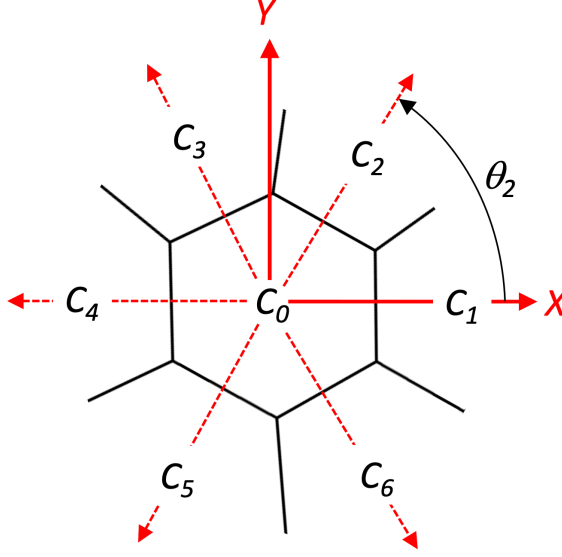


Figure B.2: Tangent plane for cell c_0 .

variable ϕ of the form

$$\phi = a_0 + a_x x + a_y y + a_{xx} x^2 + a_{xy} xy + a_{yy} y^2 \quad (\text{B.1})$$

using the values from the cell c_0 and its neighbors. Defining the cell-centered values as $\mathbf{s} = [\phi_0, \dots, \phi_6]^T$ and the polynomial coefficients $\mathbf{f} = [a_0, \dots, a_{yy}]^T$, we can define the least-squares-fit polynomial following [Strang \(1980\)](#):

$$\mathbf{f} = (\mathbf{P}^T \mathbf{P})^{-1} \mathbf{P}^T \mathbf{s} = \mathbf{B} \mathbf{s}, \quad (\text{B.2})$$

where the matrix $\mathbf{P}$ is an $m \times n$ matrix defined as

$$\mathbf{P} = \begin{pmatrix} 1 & x_0 & y_0 & x_0^2 & x_0 y_0 & y_0^2 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_m & y_m & x_m^2 & x_m y_m & y_m^2 \end{pmatrix}.$$

m is equal to the number of cells used in the polynomial fit (e.g. $m = 7$ for the configuration shown in figure [B.2](#), 6 when the central cell is a pentagon, and 8 when the central cell is a heptagon) and n is the number of coefficients in the polynomial ([B.1](#)), in this case 6.

The matrix $\mathbf{B} = (\mathbf{P}^T \mathbf{P})^{-1} \mathbf{P}^T$ is an $n \times m$ matrix that is used to compute the coefficients of the polynomial given the cell values. In the horizontal flux divergence ([4.7](#)), we need the second derivatives of the polynomials in the direction normal to each cell face. Employing the chain rule, the 2nd derivative of the polynomial ([B.1](#)) in the x' direction, where x' is a rotation of the coordinate x through an angle θ_{e_i} , is

$$\frac{\partial^2 \phi}{\partial x'^2} = 2 (a_{xx} \cos^2 \theta_{e_i} + a_{xy} \cos \theta_{e_i} \sin \theta_{e_i} + a_{yy} \sin^2 \theta_{e_i}), \quad (\text{B.3})$$

and we can write this using the $\mathbf{B}$ matrix as

$$\frac{\partial^2 \phi}{\partial x'^2} = \sum_{i=1}^m [\phi_i (2\mathbf{B}_{4,i} \cos^2 \theta_{e_i} + 2\mathbf{B}_{5,i} \cos \theta_{e_i} \sin \theta_{e_i} + 2\mathbf{B}_{6,i} \sin^2 \theta_{e_i})] . \quad (\text{B.4})$$

For each cell we compute and store the coefficients for each edge e on the cell (the term in parentheses in (B.4)) and use these in the construction of the flux (4.7).

MPAS code: *The computation of the least-squares-fit 2nd derivative weights occurs within the `init_atmosphere` core in `MPAS-Model/src/core_init_atmosphere/mpas_atm_advection.F`. The 2nd derivative coefficients are stored in the array `deriv_two(source_cell,cells,edges)`, where `edges` are all the edges on the mesh, `cells` are the two cells that share the edge, and `source_cells` are the cells used in the least-squares fit polynomial. These operations are performed in subroutine `atm_initialize_advection_rk`.*

Appendix C

Gradient Operators

Horizontal gradients on the MPAS mesh are needed in many places in MPAS, for example in the calculation of the horizontal deformation and in the calculation of the horizontal components of the frontogenesis function. In this technical note we have already described horizontal gradients across cell edges (cell-centered quantities differenced between cells sharing the edge), gradients along cell edges (e.g. vorticity differenced between vertices), and the horizontal flux divergence operator applied to momentum, coupled potential temperature, and scalar mass. We describe the finite-volume computation of arbitrary horizontal gradients in the first section in the first section of this appendix and their use in specific operators in the latter sections of this appendix.

C.1 Horizontal Derivatives

Within the finite-volume formulation of MPAS, we define gradient operators as the cell-average of the gradient. Using Green's theorem,

$$\iint_R \left(\frac{\partial N}{\partial x} - \frac{\partial M}{\partial y} \right) dx dy = \oint_C (M dx + N dy),$$

with the components

$$\iint_R \frac{\partial N}{\partial x} dx dy = \oint_C N dy \tag{C.1}$$

and

$$\iint_R -\frac{\partial M}{\partial y} dx dy = \oint_C M dx, \tag{C.2}$$

we transform cell-integrated values of derivatives into line integrals of the differentiated quantities. Specifically, we take the line integral in the counterclockwise direction in this application, and the integral is over a closed curve C , completely around the cell. Dividing the discrete evaluation of (C.1) or (C.2) by the area recovers the cell-averaged gradient.

To compute the discrete horizontal derivatives, we begin by defining a tangent plane to the cell center and map the vertices of the cell to the tangent plane, as described in figure C.1. We define the plane and the locations of the vertices (x_i, y_i) on the tangent plane for vertices $i = 1, \dots, n$, where n is the number of vertices (and the number of edges). The vertices are

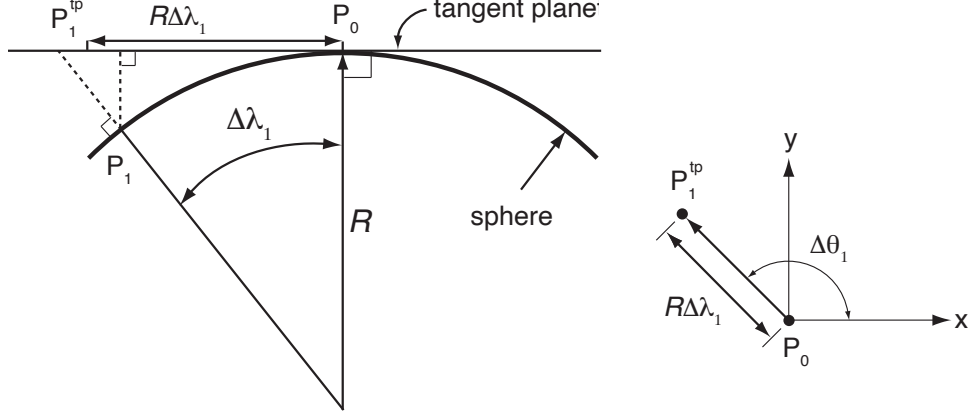


Figure C.1: MPAS tangent-plane schematic.

ordered such that they progress counterclockwise around the cell; likewise edges are ordered counterclockwise around the cell with edge e_i spanning vertices (x_i, y_i) and (x_{i+1}, y_{i+1}) as depicted in figure C.2. The line integrals take place in the (x, y) coordinate system defined in figure C.2, where the x direction is unconstrained, e.g. we do not require any vertex to lie on the x axis nor do we require any normal velocity to lie on the axis. In the MPAS implementation, we define the positive direction x as tangent to latitude circles at the cell edge points and increasing in the direction of increasing longitude, and the direction y as tangent to longitudes and increasing with increasing latitude, i.e. the local latitude-longitude coordinate. Thus the x derivative is the longitudinal derivative and the y derivative is the latitudinal derivative.

Referring to figure C.2, we define the length increments for the edge from vertex i to vertex $i + 1$ as

$$\begin{aligned}\Delta x &= -l_e \sin \theta_e, \\ \Delta y &= l_e \cos \theta_e,\end{aligned}$$

where

$$l_e = \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2}$$

is the edge length on the tangent plane for edge e . We can now define the gradient operators for cell-centered quantities as

$$A_c \phi_x = \sum_{e=1}^n l_e \cos \theta_e \bar{\phi}^e, \quad (\text{C.3})$$

$$A_c \phi_y = \sum_{e=1}^n l_e \sin \theta_e \bar{\phi}^e, \quad (\text{C.4})$$

where $\bar{\phi}^e$ denotes an average of the cell-centered values of ϕ from the cells sharing the edge and n is the number of edges on the cell. Next we define coefficients:

$$c_{x,c,e} = \frac{l_e \cos \theta_e}{A_c} \quad (\text{C.5})$$

$$c_{y,c,e} = \frac{l_e \sin \theta_e}{A_c} \quad (\text{C.6})$$

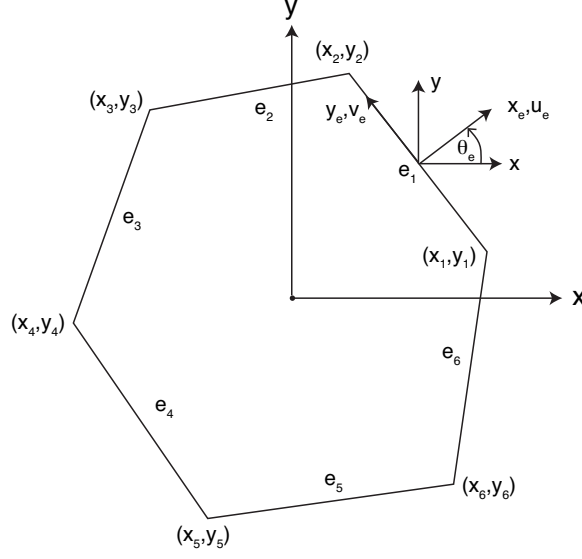


Figure C.2: Horizontal depiction of the tangent plane and cell-edge velocity fields for an arbitrary MPAS mesh cell.

To compute the line integrals (C.1) and (C.2) for the horizontal velocities (u, v) in the tangent plane coordinate, we rotate the edge velocities into the coordinate:

$$\begin{aligned} u &= u_e \cos \theta_e - v_e \sin \theta_e, \\ v &= u_e \sin \theta_e + v_e \cos \theta_e. \end{aligned}$$

The edge velocity u_e is recovered from the prognosed momentum $\tilde{\rho}_d u_e$, and the edge tangential velocity is diagnosed as described in section 4.3.2. Note that in MPAS the normal velocity may or may not be positive outward from a given cell, and when pointing inward the signs are shifted given the 180 degree rotation of both u_e and v_e . Given these definitions, we can compute the cell-averaged derivatives of the velocity components (u, v) on the tangent plane:

$$\begin{aligned} A_c u_x &= \sum_{e=1}^n l_e \cos \theta_e (u_e \cos \theta_e - v_e \sin \theta_e) \\ &= \sum_{e=1}^n l_e (u_e \cos^2 \theta_e - v_e \cos \theta_e \sin \theta_e) \end{aligned} \tag{C.7}$$

$$\begin{aligned} A_c u_y &= \sum_{e=1}^n l_e \sin \theta_e (u_e \cos \theta_e - v_e \sin \theta_e) \\ &= \sum_{e=1}^n l_e (u_e \sin \theta_e \cos \theta_e - v_e \sin^2 \theta_e) \end{aligned} \tag{C.8}$$

$$\begin{aligned}
A_c v_x &= \sum_{e=1}^n l_e \cos \theta_e (u_e \sin \theta_e + v_e \cos \theta_e) \\
&= \sum_{e=1}^n l_e (u_e \cos \theta_e \sin \theta_e + v_e \cos^2 \theta_e)
\end{aligned} \tag{C.9}$$

$$\begin{aligned}
A_c v_y &= \sum_{e=1}^n l_e \sin \theta_e (u_e \sin \theta_e + v_e \cos \theta_e) \\
&= \sum_{e=1}^n l_e (u_e \sin^2 \theta_e + v_e \sin \theta_e \cos \theta_e)
\end{aligned} \tag{C.10}$$

where A_c is the cell area and n is the number of edges on each cell. There are only three distinct coefficients multiplying velocities u_e and v_e in (C.7)-(C.10): $\cos^2 \theta_e$, $\sin^2 \theta_e$, and $\cos \theta_e \sin \theta_e$, all three multiplied by l_e .

C.2 Horizontal deformation for the Smagorinsky scheme

The 2D (horizontal) Smagorinsky viscosity calculation uses the horizontal deformation term in the formula for the horizontal eddy viscosity K :

$$K = c_s^2 l^2 \left[\left(\frac{\partial u}{\partial x} - \frac{\partial v}{\partial y} \right)^2 + \left(\frac{\partial u}{\partial y} + \frac{\partial v}{\partial x} \right)^2 \right]^{1/2}. \tag{C.11}$$

Defining line-integral coefficients $a_{c,e}$, $b_{c,e}$, and $c_{c,e}$ for cell c and edge e as

$$\begin{aligned}
a_{c,e} &= [l_e \cos^2 \theta_e]_c / A_c, \\
b_{c,e} &= [l_e \sin^2 \theta_e]_c / A_c, \\
c_{c,e} &= [l_e \sin \theta_e \cos \theta_e]_c / A_c,
\end{aligned}$$

we can re-write the derivatives for cell c as

$$u_x = \sum_{e=1}^n (u_e a_{c,e} - v_e c_{c,e}), \tag{C.12}$$

$$u_y = \sum_{e=1}^n (u_e c_{c,e} - v_e b_{c,e}), \tag{C.13}$$

$$v_x = \sum_{e=1}^n (u_e c_{c,e} + v_e a_{c,e}), \tag{C.14}$$

$$v_y = \sum_{e=1}^n (u_e b_{c,e} + v_e c_{c,e}). \tag{C.15}$$

In the Smagorinsky eddy viscosity (C.11), the terms in the deformation operator can be written in terms of the discrete velocity derivatives (C.12)-(C.15):

$$\left(\frac{\partial u}{\partial x} - \frac{\partial v}{\partial y}\right) \rightarrow \sum_{e=1}^n ((a_{c,e} - b_{c,e})u_e - 2c_{c,e}v_e) \quad (\text{C.16})$$

$$\left(\frac{\partial u}{\partial y} + \frac{\partial v}{\partial x}\right) \rightarrow \sum_{e=1}^n ((2c_{c,e}u_e + (a_{c,e} - b_{c,e})v_e) \quad (\text{C.17})$$

In MPAS we pre-compute the coefficients $(a_{c,e} - b_{c,e})$ and $2c_{c,e}$ for all edges on each cell used in computing the horizontal derivatives of the velocities and store them for use during the integration.

MPAS code: The two coefficients in (C.16) and (C.17), $(a_{c,e} - b_{c,e})$ and $2c_{c,e}$, are computed in subroutine `atm_initialize_deformation_weights` in [MPAS-Model/src/core_init_atmosphere/mpas_atm_advection.F](#) in subroutine `atm_initialize_deformation_weights`. The coefficients are stored in the arrays `defc_a` $(a_{c,e} - b_{c,e})$ and `defc_b` $(2c_{c,e})$. The Smagorinsky viscosity (C.11) is computed in subroutine `atm_compute_dyn_tend` in [MPAS-Model/src/core_atmosphere/dynamics/mpas_atm_time_integration.F](#) as are the 2nd-order filter tendencies for the dynamics variables u , w , and θ_m for the RK3 time integration.

C.3 Frontogenesis Function

The frontogenesis function is given in section 6.2 and we repeat it here:

$$\begin{aligned} \frac{1}{2} \left| \frac{\partial \nabla \theta}{\partial t} \right| = \frac{1}{2} \left[- (u_x + v_y) |\nabla \theta|^2 \right. \\ \left. + 2 (u_y + v_x) (\theta_x \theta_y) \right. \\ \left. - (u_x - v_y) (\theta_x^2 - \theta_y^2) \right]. \end{aligned} \quad (\text{C.18})$$

There are three quantities involving the velocity needed to evaluate this function: $(u_x + v_y)$, $(u_y + v_x)$, and $(u_x - v_y)$. The first is the velocity divergence, and this is computed using the standard divergence operator for a cell:

$$(u_x + v_y) = \nabla_\zeta \cdot \mathbf{v} = \frac{1}{A_i} \sum_{n_{e_i}} (L_{e_i} \mathbf{v}_{H,e_i} \cdot \mathbf{n}), \quad (\text{C.19})$$

where n_e are the number of edges for the cell and L_{e_i} is the length of each edge. In MPAS the velocity divergence is computed on the sphere (i.e. L_{e_i} are the spherical arc lengths of the edge as opposed to the length of the edges on the tangent plane in figure C.2).

The second term, $(u_y + v_x)$, is the shearing deformation, and it is computed as in the Smagorinsky deformation using C.17. The third term, $(u_x - v_y)$, is the stretching deformation term and it is computed as in the Smagorinsky deformation using C.16.

Derivatives of the potential temperature (θ_x, θ_y) are also needed in the frontogenesis function (C.18). These are computed using (C.3) and (C.4).

MPAS code: *The coefficients involving hearing deformation and the stretching deformation are computed stored as described in the case of the Smagorinsky eddy viscosity calculation. The coefficients for the scalar gradient calculations, (C.5) and (C.6), are computed in subroutine atm_initialize_deformation_weights in [MPAS-Model/src/core_init_atmosphere/mpas_atm_advection.F](#) in subroutine atm_initialize_deformation_weights. The coefficients are stored in the arrays cell_gradient_coef_x ($c_{x_{c,e}}$) and cell_gradient_coef_y ($c_{y_{c,e}}$). They are not presently used in MPAS-A but are used in the frontogenesis computation in CAM-MPAS.*

Bibliography

- Dudhia, J., 1995: Reply. *Monthly Weather Review*, **123** (8), 2573–2575.
- Hundsdoerfer, W. B., B. Koren, M. van Loon, and J. G. Verwer, 1995: A positive finite difference advection scheme. *J. Comp. Phys.*, **117**, 35–46.
- Ikawa, M., 1988: Comparison of some schemes for nonhydrostatic models with orography. *J. Meteor. Soc. Japan*, **66**, 753–776.
- Jablonowski, C., and D. L. Williamson, 2006: A baroclinic instability test case for atmospheric model dynamical cores. *Quart. J. Roy. Meteor. Soc.*, **132**, 2943–2975.
- Klemp, J. B., 2011: A terrain following coordinate with smoothed coordinate surfaces. *Mon. Wea. Rev.*, **139**, 2163–2169, doi:10.1175/MWR-D-10-05046.1.
- Klemp, J. B., J. Dudhia, and A. D. Hassiotis, 2008: An upper gravity-wave absorbing layer for NWP applications. *Mon. Wea. Rev.*, **136**, 3987–4004, doi:10.1175/2008MWR2596.1.
- Klemp, J. B., and W. C. Skamarock, 2021: Adapting the MPAS Dynamical Core for Applications Extending Into the Thermosphere. *Journal of Advances in Modeling Earth Systems*, n/a–n/a, doi:10.1029/2021MS002499.
- Klemp, J. B., W. C. Skamarock, and J. Dudhia, 2007: Conservative split-explicit time integration methods for the compressible nonhydrostatic equations. *Mon. Wea. Rev.*, **135**, 2897–2913.
- Klemp, J. B., W. C. Skamarock, and O. Fuhrer, 2003: Numerical consistency of metric terms in terrain-following coordinates. *Mon. Wea. Rev.*, **131**, 1229–1239.
- Klemp, J. B., W. C. Skamarock, and S. Park, Ha, 2018: Damping Acoustic Modes in Compressible Horizontally Explicit Vertically Implicit (HEVI) and Split-Explicit Time Integration Schemes. *Monthly Weather Review*, n/a–n/a, doi:10.1175/MWR-D-17-0384.1.
- Ooyama, K. V., 1990: A thermodynamic foundation for modeling the moist atmosphere. *J. Atmos. Sci.*, **47**, 2580–2593, doi:10.1175/JAS-D-20-0124.1.
- Park, S.-H., W. C. Skamarock, J. B. Klemp, L. D. Fowler, and M. G. Duda, 2013: Evaluation of Global Atmospheric Solvers Using Extensions of the Jablonowski and Williamson Baroclinic Wave Test Case. *Monthly Weather Review*, **141** (9), 3116–3129, doi:10.1175/MWR-D-12-00096.1.

- Richter, J. H., F. Sassi, and R. R. Garcia, 2010: Toward a physically based gravity wave source parameterization in a general circulation model. *Journal of the Atmospheric Sciences*, **67** (1), 136–156, doi:<https://doi.org/10.1175/2009JAS3112.1>.
- Ringler, T., L. Ju, and M. Gunzburger, 2008: A multiresolution method for climate system modeling: application of spherical centroidal Voronoi tessellations. *Ocean Dyn.*, **58**, 475–498.
- Ringler, T., J. Thuburn, J. B. Klemp, and W. C. Skamarock, 2010: A unified approach to energy conservation and potential vorticity dynamics for arbitrarily-structured C-grids. *J. Comp. Phys.*, **229**, 3065–3090.
- Sadourny, R., and C. Basdevant, 1985: Parameterization of subgrid scale barotropic and baroclinic eddies in quasi-geostrophic models - anticipated potential vorticity method. *J. Atmos. Sci.*, **42**, 1353–1363.
- Schär, C., D. Leuenberger, O. Fuhrer, D. Lüthi, and C. Girard, 2002: A new terrain-following vertical coordinate formulation for atmospheric prediction models. *Mon. Wea. Rev.*, **130**, 2459–2480.
- Skamarock, W. C., M. G. Duda, S. Ha, , and S.-H. Park, 2018: Limited-Area Atmospheric Modeling Using an Unstructured Mesh. *Mon. Wea. Rev.*, **146**, 3445–3460, doi:10.1175/MWR-D-18-0155.1.
- Skamarock, W. C., and A. Gassmann, 2011: Conservative transport schemes for spherical geodesic grids: High-order flux operators for ODE-based time integration. *Mon. Wea. Rev.*, **139**, 2962–2975, doi:10.1175/MWR-D-10-05056.1.
- Skamarock, W. C., J. B. Klemp, M. G. Duda, L. D. Fowler, S.-H. Park, and T. Ringler, 2012: A multiscale nonhydrostatic atmospheric model using centroidal Voronoi tessellations and C-grid staggering. *Mon. Wea. Rev.*, **140**, 3090–3105, doi:10.1175/MWR-D-11-00215.1.
- Skamarock, W. C., H. Ong, and J. B. Klemp, 2021a: A fully compressible nonhydrostatic deep-atmosphere equations solver for mpas. *Monthly Weather Review*, **149** (2), 571 – 583, doi:10.1175/MWR-D-20-0286.1, URL <https://journals.ametsoc.org/view/journals/mwre/149/2/MWR-D-20-0286.1.xml>.
- Skamarock, W. C., and Coauthors, 2021b: *A Description of the Advanced Research WRF Version 4.3*. NCAR Technical Note NCAR/TN-556+STR, 165 pp., doi:10.5065/1dfh-6p97, available at <https://www2.mmm.ucar.edu/wrf/users/docs/technote/contents.html>.
- Smagorinsky, J., 1963: General circulation experiments with the primitive equations. *Mon. Wea. Rev.*, **91**, 99–164.
- Strang, G., 1980: *Linear Algebra and Its Applications*. 2nd ed., Academic Press, 414 pp.
- Thuburn, J., T. Ringler, W. C. Skamarock, and J. B. Klemp, 2009: Numerical representation of geostrophic modes on arbitrarily structured C-grids. *J. Comp. Phys.*, **228**, 8321–8335.
- Weisman, M. L., and J. B. Klemp, 1982: The dependence of numerically simulated convective storms on vertical wind shear and buoyancy. *Mon. Wea. Rev.*, **110**, 504–520.

- Wicker, L. J., and W. C. Skamarock, 2002: Time-splitting methods for elastic models using forward time schemes. *Mon. Wea. Rev.*, **130**, 2088–2097.
- Zalesak, S. T., 1979: Fully multidimensional flux-corrected transport algorithms for fluids. *J. Comp. Phys.*, **31**, 335–362.